

BÁO CÁO ĐỒ ÁN

MÔN: THỰC HÀNH LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG

HK2 - NĂM HỌC: 2025-2026

ĐỒ ÁN NHÓM 8:

ỨNG DỤNG QUẢN LÝ SIÊU THỊ QUY MÔ VỪA VÀ NHỎ BẰNG NGÔN NGỮ C++

LỚP: 23DTV_CLC1

GVHD: Ths. Cao Trần Bảo Thương

STT	Họ và tên	MSSV
1	Hồ Trọng Hải	23207050
2	Bùi Anh Phúc	23207094
3	Lý Anh Thư	23207114
4	Đỗ Trường Tín	23207135
5	Mai Lý Khải Triều	23207136

Lời cảm ơn

Lời đầu tiên, nhóm chúng em xin gửi lời cảm ơn chân thành đến thầy/cô Khoa Điện tử - Viễn thông đã tạo điều kiện thuận lợi và môi trường học tập tốt nhất để chúng em tiếp cận với học phần Thực hành Lập trình Hướng đối tượng.

Đặc biệt, chúng em xin bày tỏ lòng biết ơn sâu sắc đến thầy Cao Trần Bảo Thương. Trong suốt quá trình học tập, thầy không chỉ truyền đạt nền tảng kiến thức lý thuyết vững chắc mà còn tận tình hướng dẫn chúng em phương pháp tư duy giải quyết vấn đề. Sự chỉ dẫn của thầy đã giúp nhóm hoàn thành tốt các yêu cầu trong dự án này.

Dù cố gắng hết sức tuy nhiên do vốn kiến thức tích lũy còn hạn chế nên bản báo cáo đồ án của nhóm em sẽ không thể tránh khỏi những sai sót. Chúng em mong được thầy xem xét và đóng góp ý kiến để bài báo cáo của nhóm được hoàn thiện hơn.

Chúng em xin chân thành cảm ơn!

TPHCM, ngày 30/04/2026

MỤC LỤC

TÓM TẮT.....	4
I. GIỚI THIỆU	4
II. CƠ SỞ LÝ THUYẾT	5
2.1. Tổng quan về Lập trình Hướng đối tượng	5
2.2.1. Tính trừu tượng.....	5
2.2.2. Tính đóng gói	5
2.2.3. Tính kế thừa.....	6
2.2.4. Tính đa hình.....	6
2.3. Phương thức	6
2.4. Các mối quan hệ giữa các lớp đối tượng.....	7
2.5. Kiến trúc phần mềm MVC-lite (Model-View-Controller)	8
2.6. Lập trình hướng sự kiện (Event-Driven) và Win32 API	8
2.7. Xử lý lưu trữ tệp tin và chuẩn dữ liệu CSV	9
2.8. Quản trị bộ nhớ động và Thư viện STL.....	9
2.9. Kỹ thuật tổ chức mã nguồn và Khai báo sớm.....	9
2.10. Thư viện đồ họa GDI+ (Graphics Device Interface Plus)	10
III. ỨNG DỤNG	11
3.1. Lưu đồ giải thuật thuật toán.....	11
3.1.1. Nhóm 1: Lưu đồ Tổng quát Vận hành Hệ thống	11
3.1.2. Nhóm 2: Lưu đồ Xử lý Sự kiện Hệ thống	12
3.1.3. Nhóm 3: Lưu đồ Thao tác Giao diện.....	13
3.1.4. Nhóm 4: Lưu đồ Nghiệp vụ	18
3.2. Chương trình C++	28
3.3. Kết quả giao diện	87
3.4. Kiểm thử.....	92
3.4.2.1. Test case 1: Kiểm thử tính năng Power	94
3.4.2.2. Test case 2: Kiểm thử với tính năng Upload dữ liệu *.txt đúng cấu trúc	95
3.4.2.3. Test case 3: Kiểm thử với tính năng Upload dữ liệu *.txt sai cấu trúc	96
3.4.2.4. Test case 4: Kiểm thử với tính năng Upload dữ liệu *.csv đúng cấu trúc	96
3.4.2.5. Test case 5: Kiểm thử với tính năng Upload dữ liệu *.csv sai cấu trúc	97

3.4.2.6. Test case 6: Kiểm thử tính năng Mở Ca với dữ liệu chưa được nạp	97
3.4.2.7. Test case 7: Kiểm thử tính năng Mở Ca với dữ liệu được nạp đúng.....	98
3.4.2.8. Test case 8: Kiểm thử tính năng Kết Ca	99
3.4.2.9. Test case 9: Kiểm thử tính năng Bán Hàng	100
3.4.2.10. Test case 10: Kiểm thử tính năng Nhập Kho	103
3.4.2.11. Test case 11: Kiểm thử với tính năng Xuất Kho	105
3.4.2.12. Test case 12: kiểm thử tính năng ghi chú	106
<u>IV. KẾT LUẬN</u>	106
V. TÀI LIỆU THAM KHẢO	107
<u>DANH MỤC BẢNG</u>	108
<u>DANH MỤC HÌNH</u>	108

TÓM TẮT

Ngành bán lẻ và hoạt động quản lý siêu thị quy mô vừa và nhỏ luôn đòi hỏi các hệ thống điểm bán hàng vận hành chính xác, đồng bộ hóa dữ liệu nhanh chóng và tối ưu hóa thao tác trực quan cho người sử dụng. Các thao tác thủ công trong quản lý giao dịch, kiểm soát kho bãi và ghi chép sổ sách thường tiềm ẩn rủi ro sai sót dữ liệu. Đồng thời, việc thiếu vắng một kiến trúc phần mềm đồng nhất khiến quá trình kiểm chứng thông tin đầu vào, phân luồng nghiệp vụ và lưu vết lịch sử hệ thống gặp nhiều khó khăn, dễ dẫn đến mất đồng bộ giữa giao diện và cơ sở dữ liệu cốt lõi. Báo cáo trình bày giải pháp phát triển ứng dụng quản lý siêu thị quy mô vừa và nhỏ trên nền tảng ngôn ngữ C++ áp dụng kiến trúc MVC-lite nhằm phân tách hoàn toàn tầng giao diện và tầng xử lý logic. Đồng thời triển khai triệt để bốn tính chất cốt lõi của Lập trình Hướng đối tượng nhằm thiết lập một cấu trúc mã nguồn vững chắc. Quá trình triển khai kiến tạo ra một ứng dụng hoạt động ổn định, bảo chứng tính chính xác của luồng dữ liệu trước khi hệ thống đồng bộ vào cơ sở dữ liệu.

I. GIỚI THIỆU

Sự chuyển đổi số trong ngành bán lẻ đặt ra yêu cầu cấp thiết về các ứng dụng quản lý điểm bán hàng có khả năng vận hành chính xác, đồng bộ hóa dữ liệu nhanh chóng và tối ưu hóa trải nghiệm người dùng. Đáp ứng nhu cầu thực tiễn đó, ứng dụng quản lý siêu thị quy mô vừa và nhỏ được phát triển hoàn toàn trên nền tảng ngôn ngữ C++. Mục tiêu cốt lõi của hệ thống là tự động hóa quy trình quản lý giao dịch, loại bỏ rủi ro sai sót từ các thao tác thủ công và bảo chứng tính toàn vẹn tuyệt đối cho cơ sở dữ liệu lưu trữ. Ứng dụng vận hành dựa trên kiến trúc MVC-lite, thiết lập ranh giới độc lập giữa tầng giao diện người dùng (GUI) và tầng xử lý logic. Nền tảng kiến trúc này áp đặt một luồng xử lý dữ liệu khép kín và nghiêm ngặt: tiếp nhận thông tin, kiểm chứng tính hợp lệ, kết xuất lên bộ nhớ tạm và đồng bộ hóa bền vững xuống hệ thống tệp tin kèm theo cơ chế lưu vết nhật ký tự động theo thời gian thực. Hệ thống cung cấp một chu trình quản lý toàn diện, bao quát từ khâu nạp dữ liệu ban đầu, kiểm soát phiên làm việc (Mở/Kết ca), đến phân hệ xử lý giao dịch thương mại (Nhập kho, Xuất kho, Bán hàng) và hệ thống sổ ghi chú. Đặc biệt, cấu trúc mã nguồn khai thác triệt để bốn đặc tính cốt lõi của Lập trình Hướng đối tượng bao gồm: Đóng gói, Kế thừa và Đa hình, Trừu tượng.

- Đóng gói: Mọi thao tác kiểm duyệt, tính toán và lưu trữ phải đi qua các phương thức chuẩn hóa của lớp điều khiển nhằm đảm bảo an toàn bộ nhớ.
- Kế thừa: Cấu trúc mã nguồn thiết lập một lớp nền Transaction chứa các thuộc tính và phương thức quản lý mảng dữ liệu tạm, làm nền tảng mở rộng trực tiếp cho các phân hệ giao dịch cụ thể bao gồm Invoice, Import, và Export.

- Đa hình: Cơ chế lưu trữ đa hình cho phép hệ thống tự động định tuyến và xuất dữ liệu vào các tệp đích tương ứng thông qua một lệnh gọi kích hoạt duy nhất từ người dùng.
- Trừu tượng: Phần mềm che giấu toàn bộ độ phức tạp của thuật toán xử lý cấp thấp mang lại trải nghiệm tương tác liền mạch, đơn giản hóa cho người dùng cuối tại tầng giao diện.

Cách tiếp cận này không chỉ giải quyết trọn vẹn các yêu cầu nghiệp vụ hiện tại mà còn thiết lập một bộ khung phần mềm linh hoạt, có tính mở rộng cao và tuân thủ các tiêu chuẩn kỹ thuật phần mềm hiện đại.

II. CƠ SỞ LÝ THUYẾT

2.1. Tổng quan về Lập trình Hướng đối tượng

Lập trình hướng đối tượng (Object-Oriented Programming – OOP) là một phương pháp lập trình trong đó chương trình được tổ chức xoay quanh các đối tượng (object) thay vì chỉ là các hàm hay thủ tục. Mỗi đối tượng là một thực thể bao gồm dữ liệu (thuộc tính) và hành vi (phương thức). Trong OOP, các đối tượng được tạo ra từ lớp (class) – là một khuôn mẫu định nghĩa cấu trúc và hành vi chung. Bốn đặc tính cốt lõi của Lập trình Hướng đối tượng bao gồm: Đóng gói, Kế thừa và Đa hình, Trừu tượng.

2.2. Tính chất của lập trình hướng đối tượng

2.2.1. Tính trừu tượng

Tính trừu tượng là quá trình tập trung vào những đặc điểm cốt lõi, cần thiết của đối tượng để mô phỏng thế giới thực, đồng thời ẩn đi những chi tiết cài đặt phức tạp không liên quan. Việc áp dụng tính trừu tượng giúp đơn giản hóa bài toán thông qua hai hành động chính:

- Loại bỏ sự phức tạp: Chỉ giữ lại các thuộc tính (dữ liệu) và phương thức (hành động) thực sự quan trọng đối với ngữ cảnh của hệ thống.
- Xây dựng cấu trúc lớp: Từ những đặc điểm cốt lõi đã chọn lọc, ta tiến hành định nghĩa các thuộc tính và phương thức để hình thành nên lớp (class) đại diện cho đối tượng đó.

2.2.2. Tính đóng gói

Tính đóng gói là kỹ thuật che giấu các chi tiết bên trong của một đối tượng để bảo vệ sự toàn vẹn của dữ liệu. Quá trình này bao gồm việc hợp nhất dữ liệu và các phương thức xử

lý dữ liệu đó vào trong một Lớp (Class). Cơ chế thực hiện: Kiểm soát quyền truy cập thông qua các phạm vi gồm private, protected, và public. Trong đó, các thuộc tính thường được để ở mức private để đảm bảo chỉ các phương thức nội bộ mới có quyền thay đổi trạng thái của đối tượng.

2.2.3. Tính kế thừa

Tính kế thừa cho phép xây dựng một lớp mới (lớp con) dựa trên các đặc tính của một lớp đã tồn tại (lớp cha). Lớp con không chỉ thừa hưởng các thuộc tính và phương thức từ lớp cha mà còn có khả năng mở rộng hoặc định nghĩa lại chúng.

Cơ chế hoạt động:

- Tái sử dụng: Lớp con sử dụng lại mã nguồn từ lớp cha mà không cần viết lại.
- Ghi đè (Overriding): Lớp con có thể thay đổi cách hoạt động của một phương thức để phù hợp với đặc điểm riêng của nó.

2.2.4. Tính đa hình

Tính đa hình cho phép các đối tượng thuộc các lớp khác nhau (nhưng có cùng một lớp cha) có thể thực thi cùng một tác vụ theo những cách thức riêng biệt. Nói cách khác, một phương thức có thể có nhiều hình thái thực hiện khác nhau tùy thuộc vào đối tượng cụ thể đang gọi nó.

Tính đa hình được thể hiện qua hai dạng chính:

- Nạp chồng (Overloading) - Đa hình tĩnh: Xảy ra trong cùng một lớp. Các hàm có cùng tên nhưng khác nhau về số lượng hoặc kiểu dữ liệu của tham số nhằm xử lý các dữ liệu đầu vào khác nhau một cách linh hoạt mà vẫn giữ được tên gọi gọi nhớ.
- Ghi đè (Overriding) - Đa hình động: Xảy ra trong mối quan hệ kế thừa (giữa lớp cha và lớp con). Lớp con định nghĩa lại một phương thức đã có ở lớp cha để thực hiện hành động chuyên biệt hơn. Cho phép chương trình quyết định phương thức nào được gọi tại thời điểm thực thi (runtime) dựa trên đối tượng thực tế.

2.3. Phương thức

Phương thức là hàm được định nghĩa bên trong lớp, dùng để mô tả hành vi của đối tượng.

- **Phương thức set/get:** dùng để truy cập vào các thành phần có phạm vi truy cập private trong lớp. Là phương thức dùng để thiết lập (set) và lấy (get) giá trị, được

áp dụng cho các thuộc tính ở từ khóa private (dạng thuộc tính ẩn) trong một lớp nhằm đảm bảo tính bảo mật và kiểm soát quyền truy cập vào dữ liệu của lớp

- **Phương thức khởi tạo:** là một phương thức đặc biệt được sử dụng để khởi tạo một đối tượng mới và được gọi ngay sau khi bộ nhớ được cấp phát cho đối tượng. Tự động chạy khi chúng ta khai báo đối tượng hoặc cấp phát vùng nhớ cho đối tượng (cho con trở class) nhằm thiết lập các giá trị ban đầu cho đối tượng, khởi tạo các tài nguyên cần thiết.
- **Phương thức hủy:** là các phương thức được gọi khi một đối tượng được hủy ra khỏi bộ nhớ. Được gọi tự động khi đối tượng ra khỏi phạm vi hoặc bị xóa nhằm giải phóng tài nguyên cần thiết, thực hiện các công việc dọn dẹp sau khi đối tượng không còn sử dụng.
- **Phương thức hằng (const):** là các đối tượng hoặc phương thức không thay đổi giá trị của thuộc tính của đối tượng. Không cho phép thay đổi giá trị của thuộc tính nhằm bảo vệ dữ liệu của đối tượng, đảm bảo tính nhất quán của dữ liệu.

2.4. Các mối quan hệ giữa các lớp đối tượng

Trong mô hình lập trình hướng đối tượng (OOP), các lớp không tồn tại biệt lập mà tương tác với nhau thông qua các mối quan hệ cấu trúc và hành vi. Việc xác định rõ ràng các mối quan hệ này là cơ sở để xây dựng kiến trúc phần mềm có tính module hóa cao và dễ bảo trì.

- **Quan hệ Phụ thuộc:** là mối quan hệ yếu nhất giữa các lớp. Quan hệ phụ thuộc xảy ra khi một lớp sử dụng đối tượng của lớp khác làm tham số trong phương thức hoặc biến cục bộ trong một khoảng thời gian ngắn. Sự thay đổi trong định nghĩa của lớp bị phụ thuộc có thể ảnh hưởng đến lớp đang sử dụng nó.
- **Quan hệ Hiệp hội:** mô tả một kết nối ngữ nghĩa giữa hai lớp, cho biết các đối tượng của chúng có liên kết với nhau. Hiệp hội có thể là một chiều hoặc đa chiều và thường được xác định bởi số lượng tham gia như 1-1, 1-n, hoặc n-n.
- **Quan hệ Thu nạp và Hợp thành:** là các dạng đặc biệt của quan hệ hiệp hội, thể hiện mối quan hệ "toàn thể - bộ phận" (has-a). Trong đó Thu nạp là một quan hệ yếu trong đó bộ phận có thể tồn tại độc lập với toàn thể. Hợp thành là một quan hệ mạnh mẽ, trong đó bộ phận không thể tồn tại nếu thực thể toàn thể bị hủy bỏ.
- **Quan hệ Kế thừa:** mối quan hệ này thể hiện sự phân cấp giữa lớp cha (base class) và lớp con (derived class). Lớp con kế thừa các đặc tính của lớp cha và có thể mở rộng hoặc chuyên biệt hóa chúng. Đây là nền tảng để thực hiện tính tái sử dụng mã nguồn và đa hình động trong hệ thống.
- **Quan hệ Hiện thực hóa:** mối quan hệ này xảy ra giữa một lớp và một giao diện (Interface). Lớp cam kết thực hiện đầy đủ các phương thức được định nghĩa trong

giao diện đó. Đây là cơ chế quan trọng để tách biệt giữa "định nghĩa hành vi" và "cài đặt cụ thể", giúp hệ thống linh hoạt hơn trước các thay đổi.

2.5. Kiến trúc phần mềm MVC-lite (Model-View-Controller)

Kiến trúc MVC (Model - View - Controller) là một mẫu thiết kế phần mềm kinh điển nhằm phân tách rạch ròi giữa tầng hiển thị và tầng xử lý dữ liệu. Trong đồ án này, hệ thống áp dụng biến thể MVC-lite được tinh gọn để phù hợp với quy mô ứng dụng nội bộ:

- **Model (Tầng dữ liệu):** Được đại diện bởi lớp DataManager, chịu trách nhiệm quản lý cấu trúc dữ liệu trên RAM (như các mảng chứa sản phẩm, nhân viên. Model không trực tiếp giao tiếp với người dùng mà chỉ phản hồi các truy vấn từ Controller.
- **View (Tầng giao diện):** Được xây dựng dựa trên Win32 API, bao gồm các màn hình tương tác, nút bấm và bảng danh sách tạm (ListView). Theo đúng thiết kế, tầng GUI tuyệt đối không xử lý logic. Nó chỉ nhận input từ người dùng và truyền tín hiệu hàm xuống Controller.
- **Controller (Tầng điều khiển):** Đóng vai trò bộ não trung tâm, được hiện thực hóa qua đối tượng appCtrl thuộc lớp Controller. Lớp này tiếp nhận dữ liệu từ View, thực thi các bước kiểm duyệt (Data Validation) cực kỳ khắt khe trước khi ra lệnh cập nhật Model hoặc xuất dữ liệu ra hệ thống tệp tin

2.6. Lập trình hướng sự kiện (Event-Driven) và Win32 API

Lập trình hướng sự kiện là mô hình trong đó luồng thực thi của chương trình được quyết định bởi các sự kiện (events) như thao tác nhấp chuột hay gõ phím. Đồ án ứng dụng mạnh mẽ mô hình này thông qua thư viện Win32 API của hệ điều hành Windows để kiến tạo giao diện đồ họa (GUI) chuẩn mực.

- **Cơ chế xử lý thông điệp:** Toàn bộ tương tác của người dùng được hệ điều hành bắt lại và đẩy vào hàm xử lý thông điệp trung tâm WndProc. Thông qua sự kiện WM_COMMAND, phần mềm nhận diện chính xác định danh của từng nút bấm để kích hoạt đúng tiến trình nghiệp vụ.
- **Giao diện máy POS và bàn phím ảo:** Để mô phỏng chân thực trải nghiệm máy POS siêu thị, ứng dụng tự thiết kế một hệ thống bàn phím ảo trên màn hình. Thông qua việc bắt sự kiện EN_SETFOCUS, hệ thống tự động nhận diện và trả đúng vào các ô nhập liệu (Text Box) đang được kích hoạt, cho phép luân chuyển chuỗi ký tự từ bàn phím ảo vào màn hình một cách mượt mà và chính xác.

2.7. Xử lý lưu trữ tệp tin và chuẩn dữ liệu CSV

Do đặc thù của ứng dụng quản lý siêu thị quy mô nhỏ, thay vì phụ thuộc vào các hệ quản trị cơ sở dữ liệu công kênh (như SQL Server), phần mềm áp dụng giải pháp lưu trữ tệp tin trực tiếp, đảm bảo tính gọn nhẹ và khả năng truy xuất độc lập.

- **Kỹ thuật xử lý tệp:** Ứng dụng tích hợp thư viện <fstream> chuẩn của C++ (ifstream cho luồng đọc, ofstream cho luồng ghi). Cơ chế Auto-save (lưu tự động ngầm) được kích hoạt ngay khi một giao dịch hoàn tất và vượt qua bộ lọc kiểm chứng, ghi xuất trực tiếp xuống đĩa cứng
- **Định dạng CSV (Comma-Separated Values):** Toàn bộ dữ liệu nghiệp vụ được số hóa dưới chuẩn CSV. Việc lựa chọn CSV mang lại lợi thế vượt trội về dung lượng lưu trữ tối thiểu, tốc độ nạp/xuất cực nhanh, đồng thời tạo điều kiện thuận lợi để người quản lý dễ dàng mở và tổng hợp báo cáo bằng phần mềm Microsoft Excel.

2.8. Quản trị bộ nhớ động và Thư viện STL

Việc cấp phát bộ nhớ tĩnh (Static Array) truyền thống thường dẫn đến nguy cơ tràn bộ nhớ (Overflow) hoặc lãng phí tài nguyên khi số lượng hàng hóa biến động. Để khắc phục, hệ thống triển khai toàn diện Thư viện Template Chuẩn (Standard Template Library - STL) của C++, đặc biệt là cấu trúc dữ liệu động std::vector.

- Quản lý bộ nhớ trung tâm: Lớp DataManager khởi tạo các tập hợp vector<Product> và vector<User> để nạp và quản lý toàn bộ danh mục sản phẩm từ tệp tin khi khởi động hệ thống, cho phép truy xuất thông tin với tốc độ tối ưu.
- Bộ nhớ đệm trong giao dịch: Kế thừa vào siêu lớp Transaction, hệ thống ứng dụng một vector<Item> đóng vai trò như bộ nhớ tạm (Cache) để chứa các mặt hàng đang được quét mã. Cấu trúc vector có khả năng tự động co giãn kích thước một cách an toàn khi số lượng dòng hiển thị trên bảng ListView tăng lên, đảm bảo duy trì độ ổn định tuyệt đối xuyên suốt các phiên giao dịch mua bán liên tục.

2.9. Kỹ thuật tổ chức mã nguồn và Khai báo sớm

Trong các hệ thống phần mềm hướng đối tượng phức tạp, các lớp (class) thường xuyên phải tương tác chéo với nhau, dẫn đến rủi ro phát sinh lỗi phụ thuộc vòng (Circular Dependency). Đồ án đã giải quyết triệt để vấn đề này thông qua hai kỹ thuật tiên biên dịch trong C++:

- **Chỉ thị bảo vệ (Header Guards):** Hệ thống sử dụng từ khóa `#pragma once` tại dòng đầu tiên của mọi tệp tiêu đề (header files). Kỹ thuật này ép buộc trình biên dịch chỉ nạp mỗi tệp đúng một lần duy nhất xuyên suốt quá trình build, loại bỏ hoàn toàn lỗi định nghĩa lặp (redefinition error).
- **Khai báo sớm (Forward Declaration):** Tại các lớp có sự tham chiếu chéo như `SystemManager` gọi `Controller` và ngược lại, thay vì nạp (include) toàn bộ thư viện của nhau, hệ thống sử dụng kỹ thuật khai báo hờ (ví dụ: `class Controller;`). Kỹ thuật này thông báo trước cho trình biên dịch về sự tồn tại của lớp, cho phép các đối tượng móc nối với nhau một cách an toàn mà không làm kẹt vòng lặp biên dịch.

2.10. Thư viện đồ họa GDI+ (Graphics Device Interface Plus)

Bên cạnh các hộp thoại và nút bấm cơ bản của Win32 API, giao diện máy POS đòi hỏi khả năng hiển thị bộ nhận diện thương hiệu (Logo) tại màn hình chờ. Để đáp ứng, hệ thống đã tích hợp thêm thư viện đồ họa 2D nâng cao là GDI+ (`<gdiplus.h>`). Thông qua quá trình nạp token khởi tạo (`GdiplusStartup`), hệ thống sử dụng các đối tượng đồ họa như `Graphics` và `SolidBrush` để xuất ra các khối hình học. Đặc biệt, thuật toán khử răng cưa (`SmoothingModeAntiAlias`) được kích hoạt để đảm bảo các nét vẽ trên GUI có độ mượt mà và sắc nét tối đa trên màn hình.

2.11. Quản trị bộ nhớ chia sẻ bằng Con trỏ

Để đảm bảo tính nhất quán dữ liệu (Data Consistency), hệ thống tuân thủ nguyên tắc: Không nhân bản (copy) mảng dữ liệu gốc ra nhiều vùng nhớ khác nhau. Thay vào đó, khái niệm Con trỏ (Pointer) trong C++ được ứng dụng làm cơ chế liên kết trung tâm. Cụ thể, các phân hệ nghiệp vụ độc lập như `UploadManager`, `NoteManager`, và `ShiftManager` đều được khai báo một biến con trỏ tham chiếu (ví dụ: `DataManager* db = nullptr;`). Thông qua hàm truyền con trỏ (Dependency Injection), các lớp này trỏ trực tiếp đến vùng nhớ chứa `vector<Product>` và `vector<User>` của bộ não trung tâm. Cơ chế này đảm bảo khi nghiệp vụ Bán hàng trừ đi 1 lượng tồn kho, nghiệp vụ Xuất kho ngay lập tức thấy được sự thay đổi, ngăn chặn triệt để tình trạng bất đồng bộ dữ liệu.

2.12. Nguyên lý kiểm chứng tính toàn vẹn dữ liệu

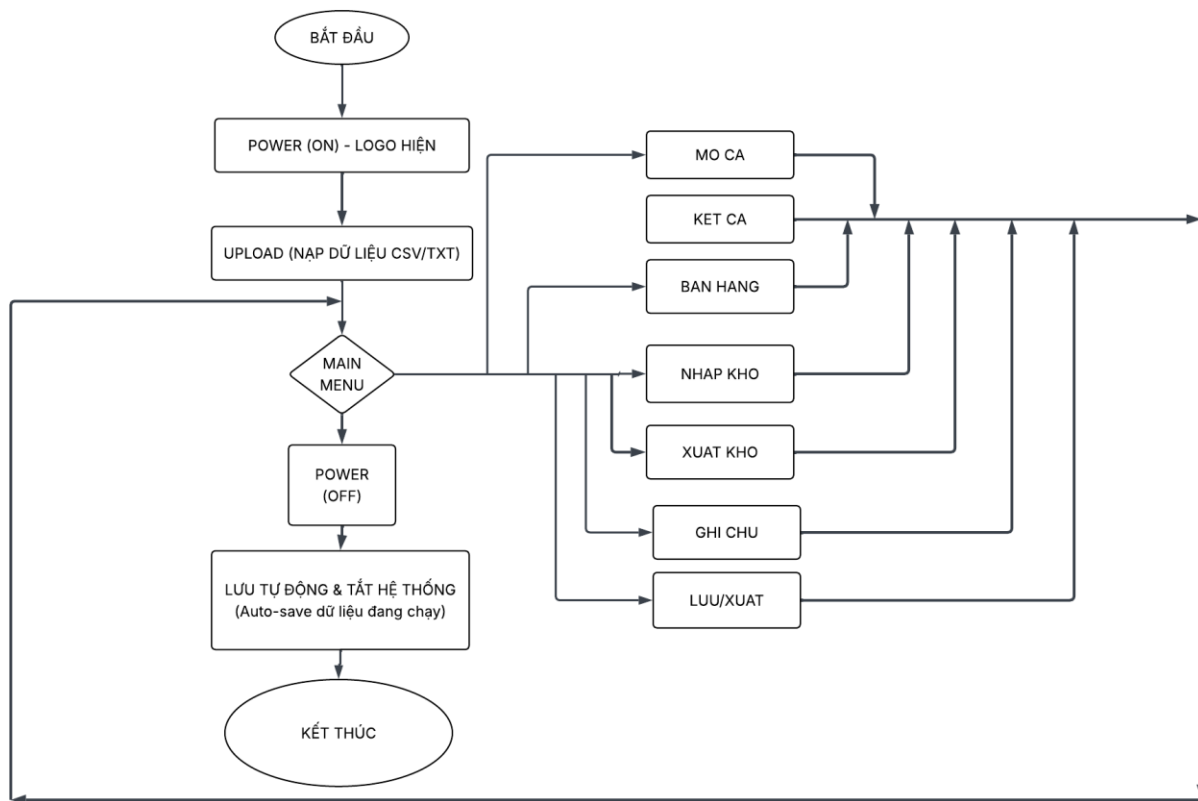
Xuyên suốt tài liệu thiết kế nghiệp vụ của đồ án, một quy tắc bất di bất dịch được thiết lập: "Không validate → không hiển thị, không lưu file". Theo đó, nguyên lý Data Validation được lập trình cứng vào mọi phương thức nhập liệu của tầng Controller. Mọi luồng dữ liệu thô đầu vào đều phải vượt qua bộ lọc kiểm duyệt cực kỳ khắt khe nhằm đối chiếu: định dạng kiểu dữ liệu rõ ràng, kiểm tra mã thực tế trong cơ sở dữ liệu, và giới hạn

chặn mốc tồn kho. Chỉ khi đạt chuẩn 100%, dữ liệu mới được phép nạp vào mảng cấu trúc bộ nhớ tạm Item để hiển thị lên bảng điều khiển, giúp phần mềm chống lại mọi sự cố tràn bộ nhớ hoặc hư hỏng file do người dùng thao tác lỗi.

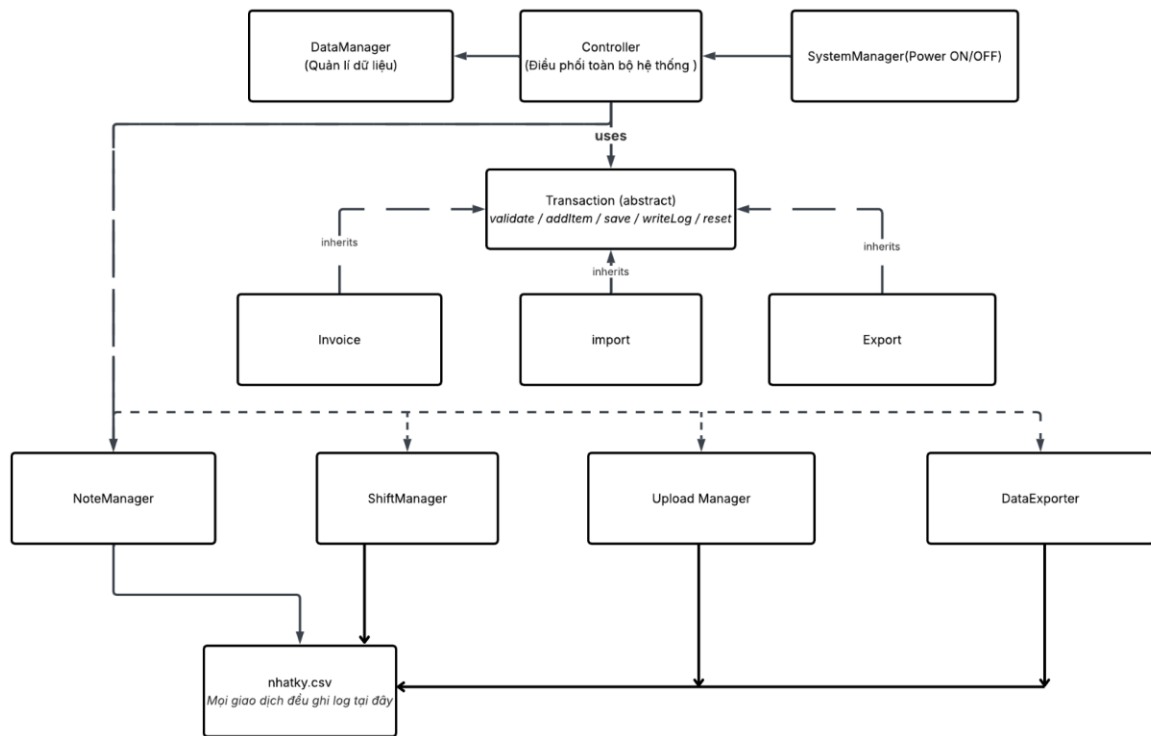
III. ỨNG DỤNG

3.1. Lưu đồ giải thuật thuật toán

3.1.1. Nhóm 1: Lưu đồ Tổng quát Vận hành Hệ thống



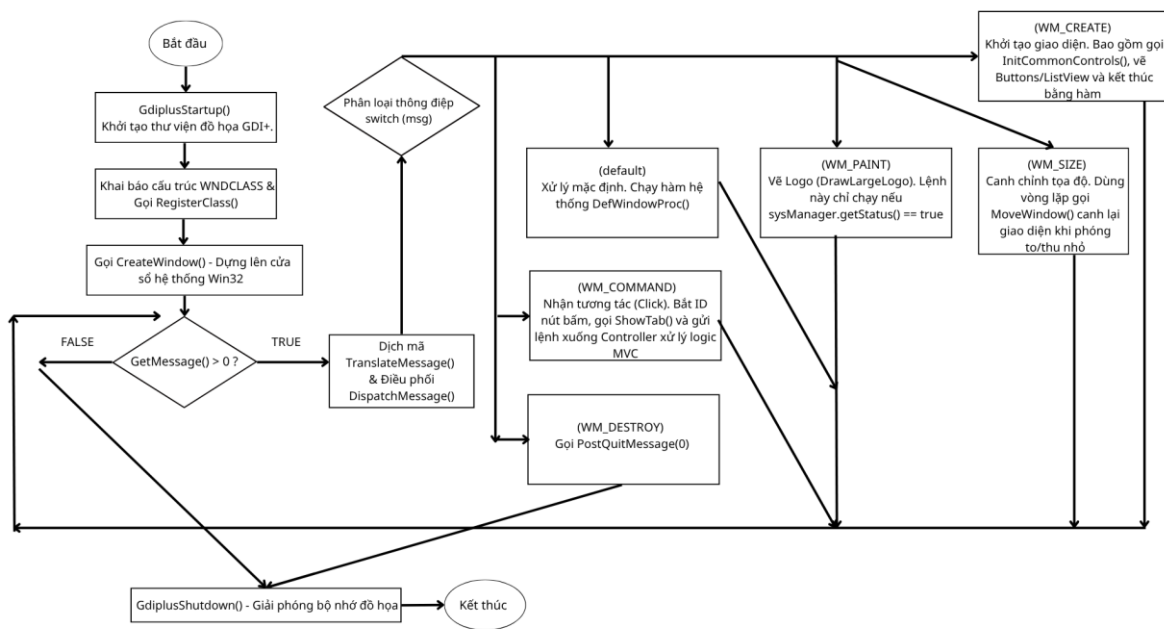
Hình 3.1. Lưu đồ Tổng quát Vận hành Hệ thống



Hình 3.2. Sơ đồ Tổ chức Phân tầng Mã nguồn Hệ thống

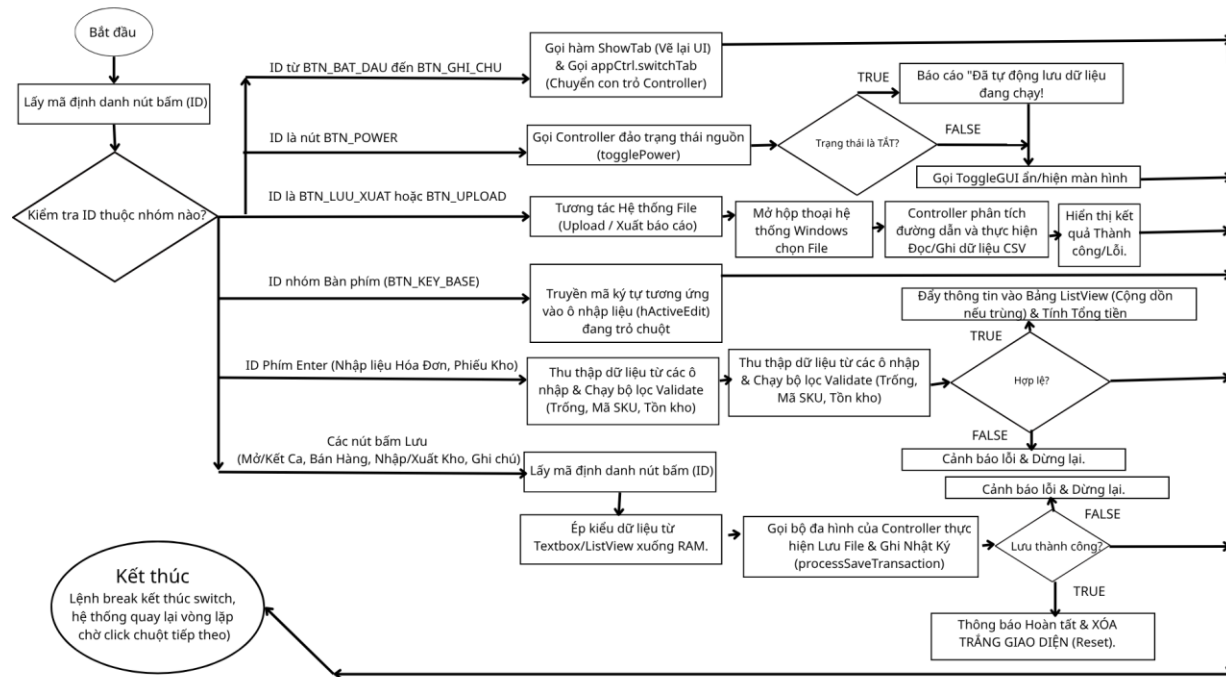
3.1.2. Nhóm 2: Lưu đồ Xử lý Sự kiện Hệ thống

3.1.2.1. Lưu đồ Xử lý giao diện toàn hệ thống



Hình 3.3. Lưu đồ Xử lý Giao diện toàn hệ thống

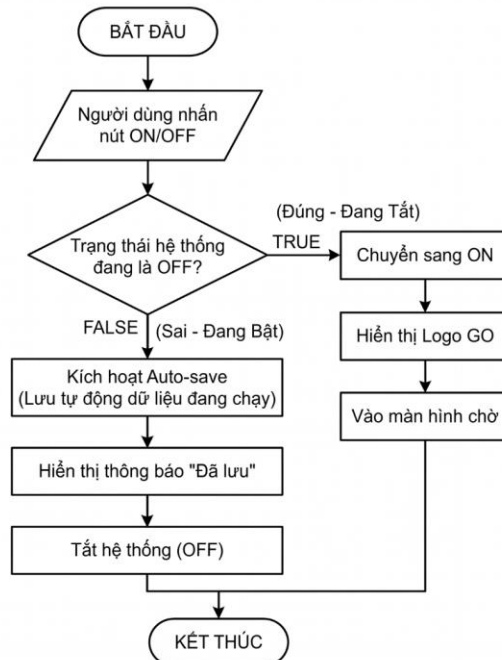
3.1.2.2 Lưu đồ Xử lý Sự kiện Giao diện (WM_COMMAND)



Hình 3.4. Lưu đồ Xử lý Sự kiện Giao diện

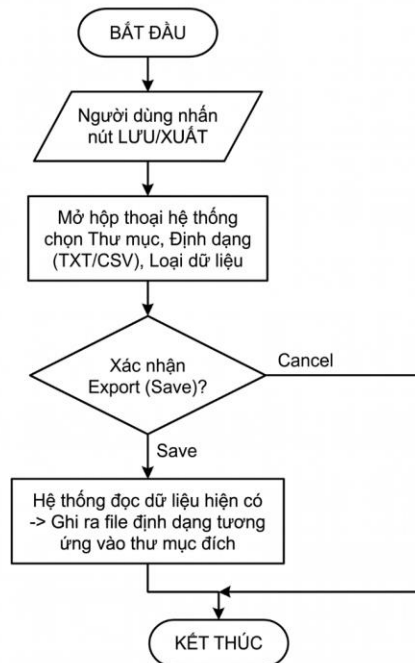
3.1.3. Nhóm 3: Lưu đồ Thao tác Giao diện

3.1.3.1. Lưu đồ Power (ON/OFF)



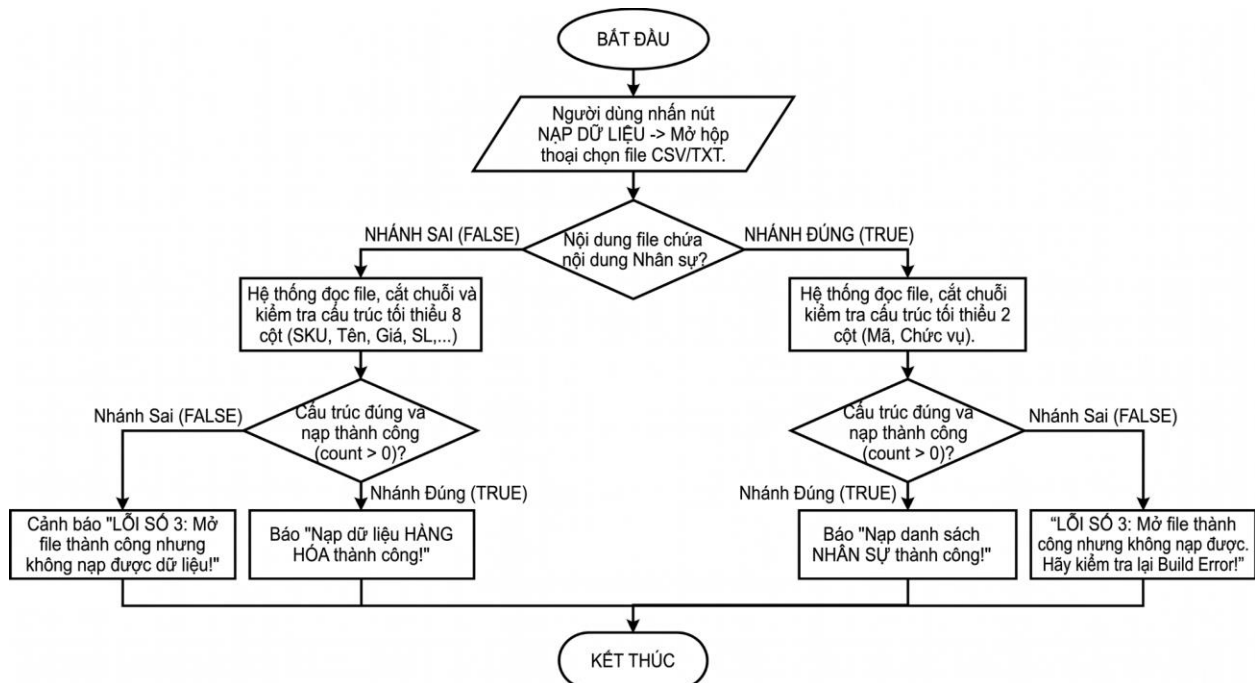
Hình 3.5. Lưu đồ Tab Power (ON/OFF)

3.1.3.2. Lưu đồ File (LƯU/XUẤT)



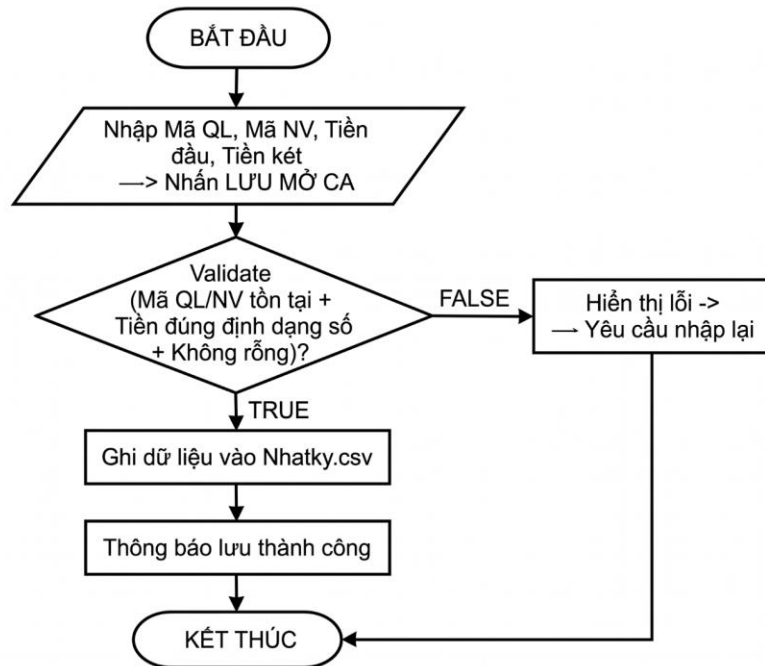
Hình 3.6. Lưu đồ File (LƯU/XUẤT)

3.1.3.3. Lưu đồ Upload (NẠP DỮ LIỆU)



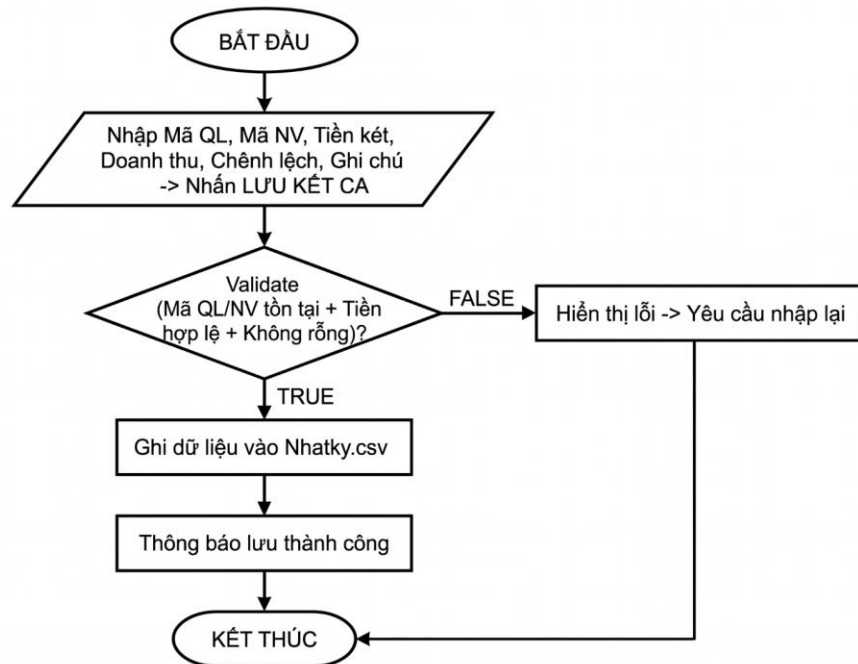
Hình 3.7. Lưu đồ Upload (NẠP DỮ LIỆU)

3.1.3.4. Lưu đồ Mở Ca (BẮT ĐẦU)



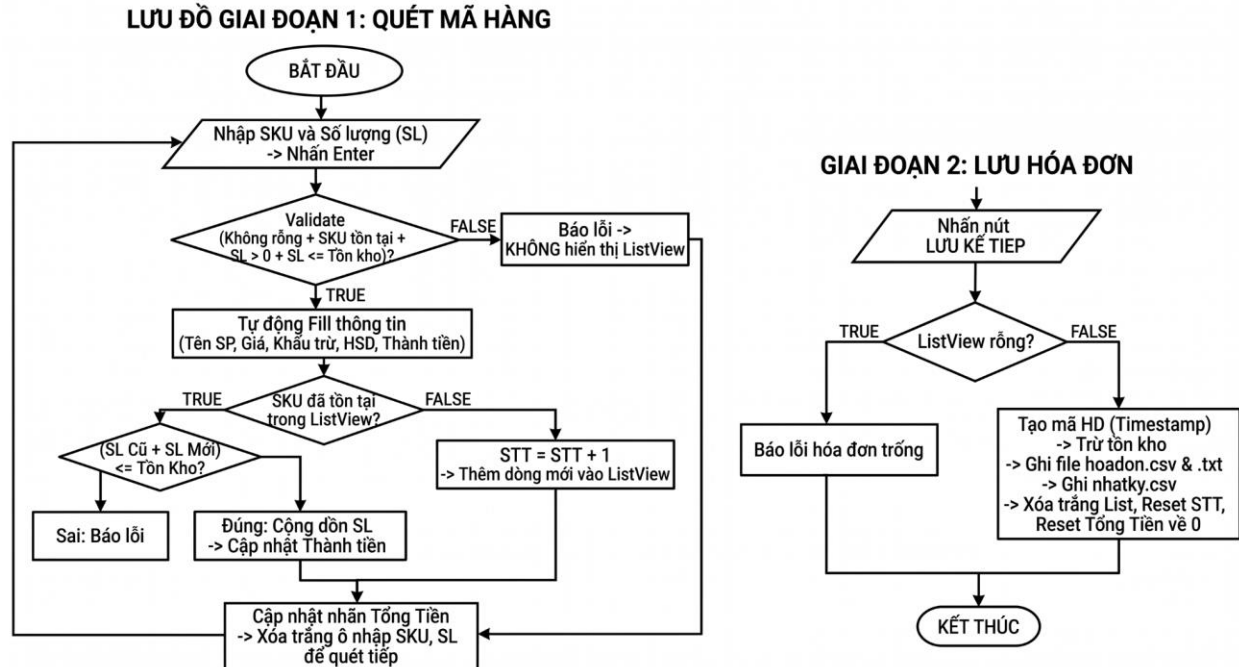
Hình 3.8. Lưu đồ Mở Ca (BẮT ĐẦU)

3.1.3.5. Lưu đồ Kết Ca (KẾT THÚC)



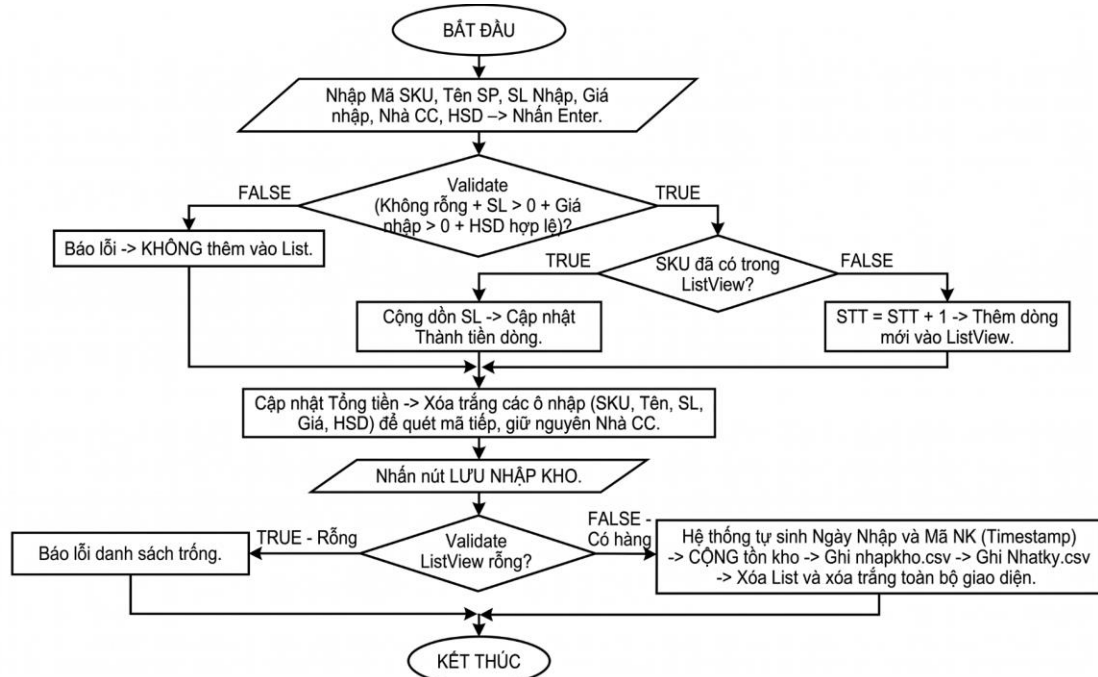
Hình 3.9. Lưu đồ Kết Ca (KẾT THÚC)

3.1.3.6. Lưu đồ Bán Hàng (BÁN HÀNG)



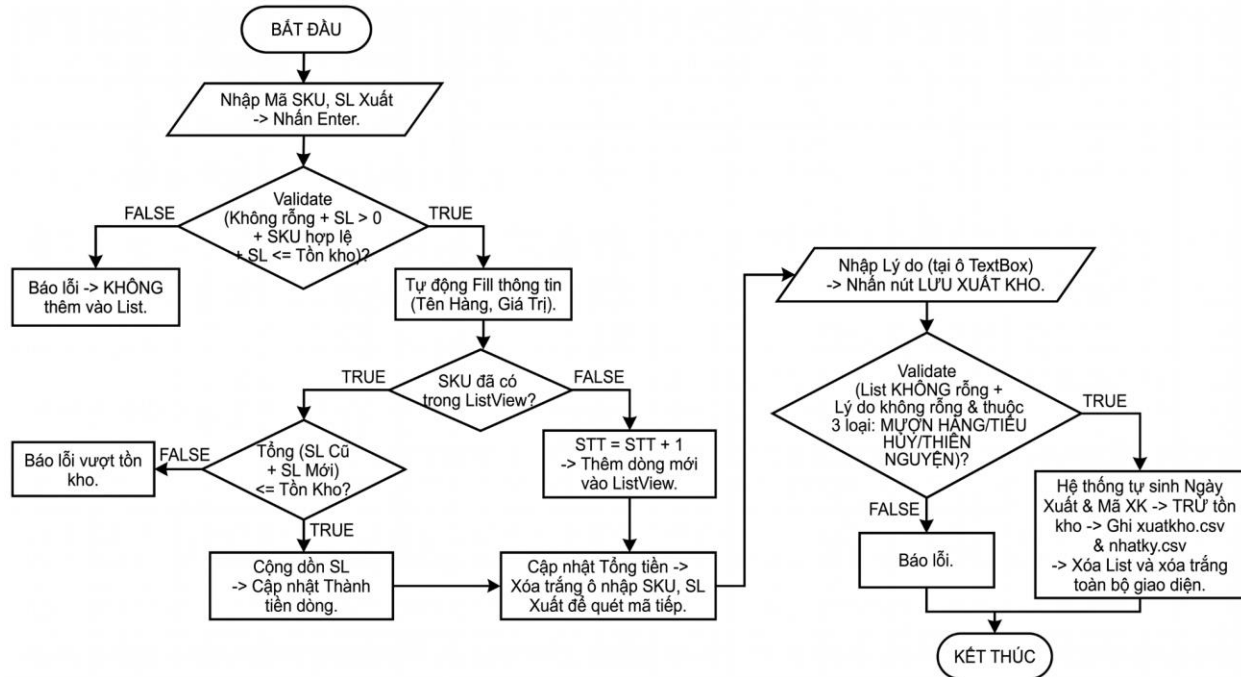
Hình 3.10. Lưu đồ Bán Hàng (BÁN HÀNG)

3.1.3.7. Lưu đồ Nhập Kho (NHẬP KHO)



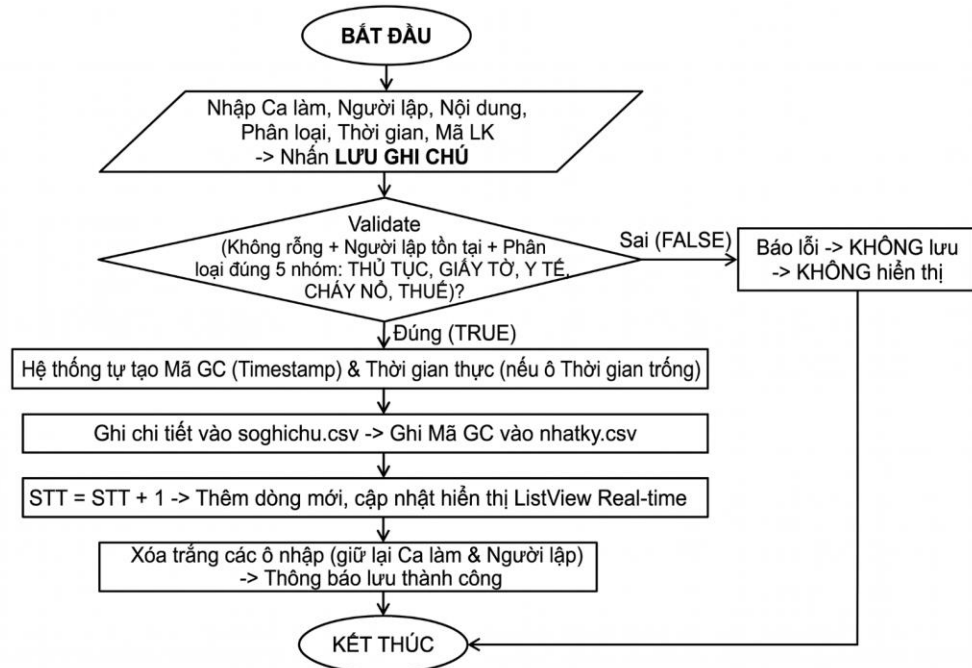
Hình 3.11. Lưu đồ Nhập Kho (NHẬP KHO)

3.1.3.8. Lưu đồ Xuất Kho (XUẤT KHO)



Hình 3.12. Lưu đồ Xuất Kho (XUẤT KHO)

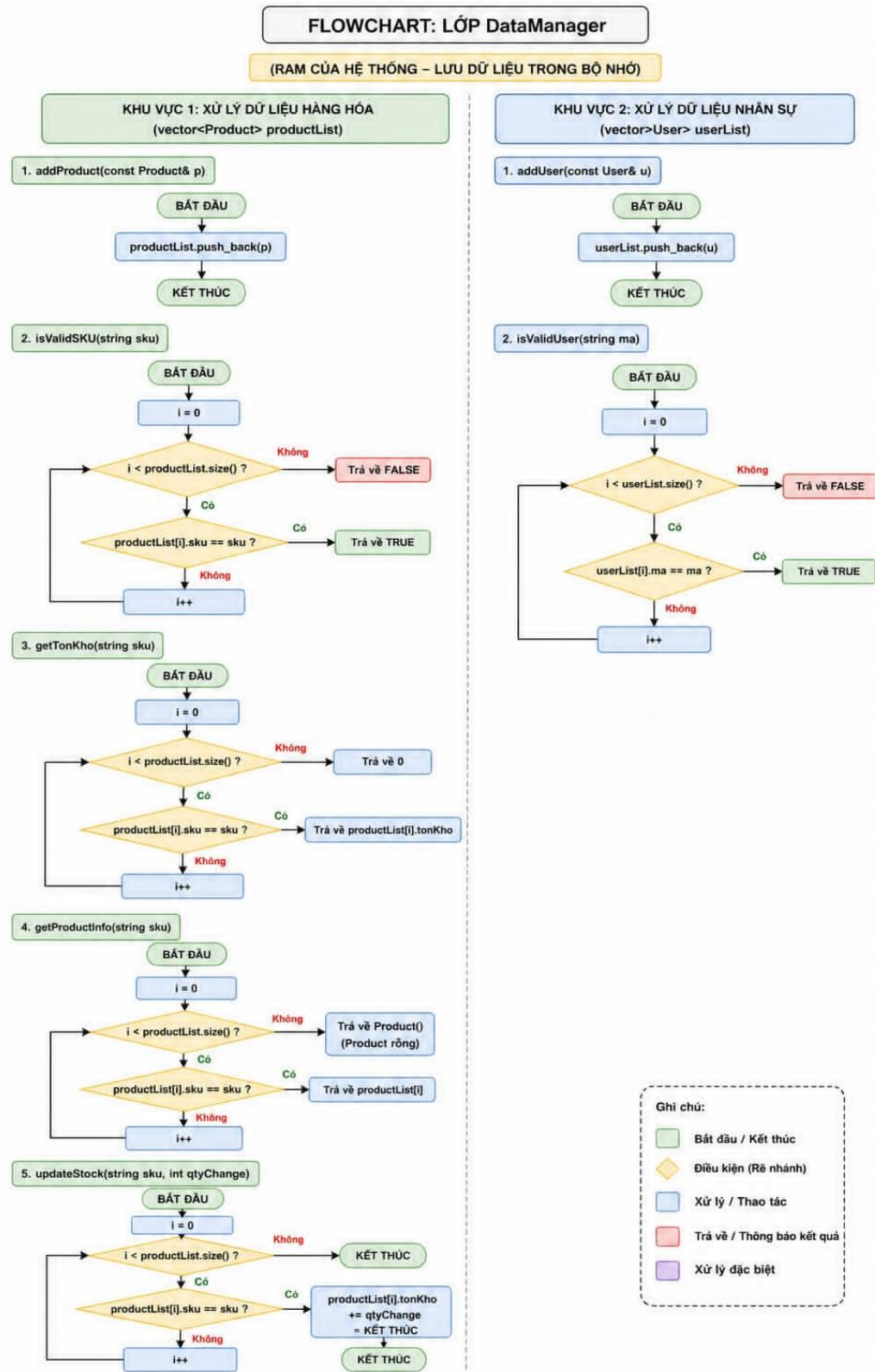
3.1.3.9. Lưu đồ Ghi Chú (GHI CHÚ)



Hình 3.13. Lưu đồ Ghi Chú (GHI CHÚ)

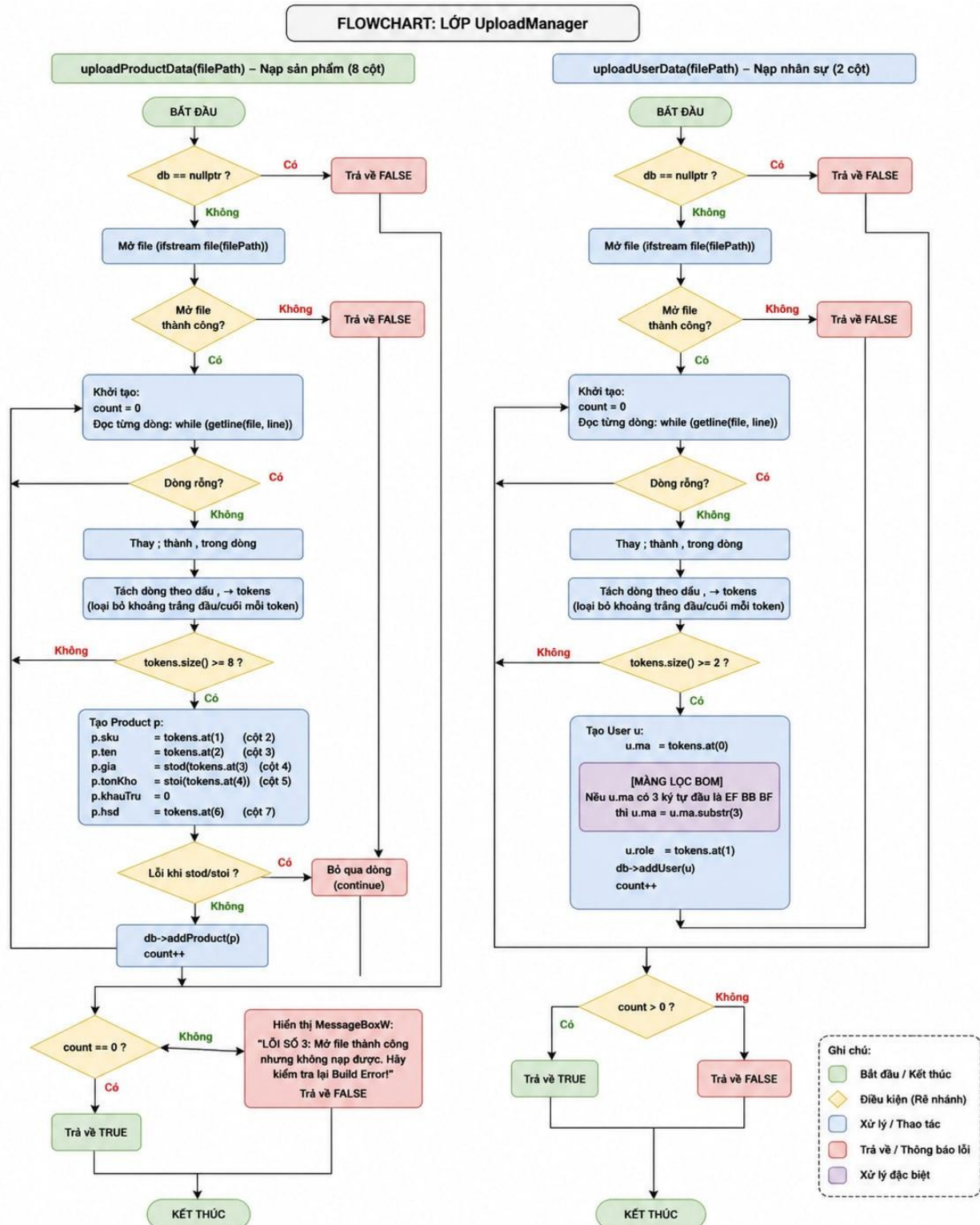
3.1.4. Nhóm 4: Lưu đồ Nghiệp vụ

3.1.4.1. Lưu đồ Quản trị Dữ liệu Trung tâm (DataManager)



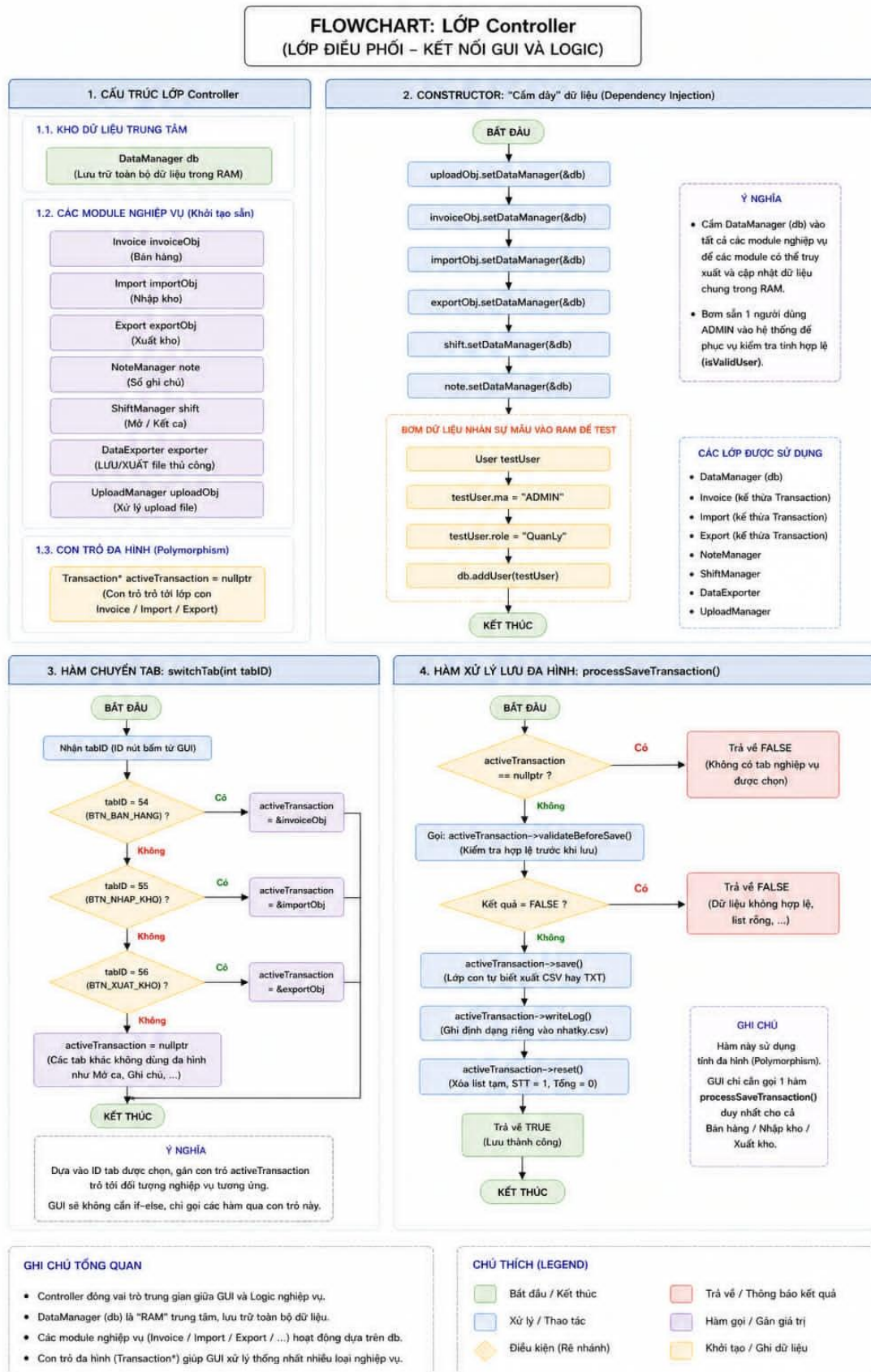
Hình 3.14. Lưu đồ Quản trị Dữ liệu Trung tâm

3.1.4.2. Lưu đồ Giải thuật Nạp dữ liệu đầu vào (UploadManager)



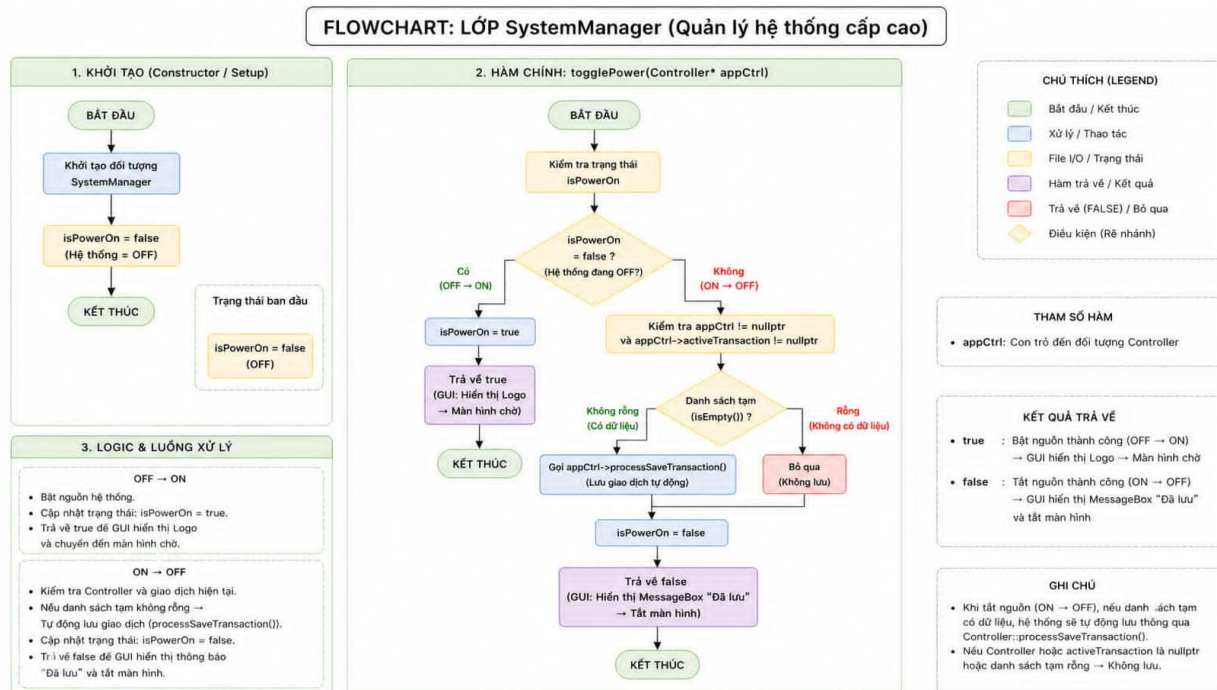
Hình 3.15. Lưu đồ Giải thuật Nạp dữ liệu đầu vào

3.1.4.3. Lưu đồ Controller (Controller)



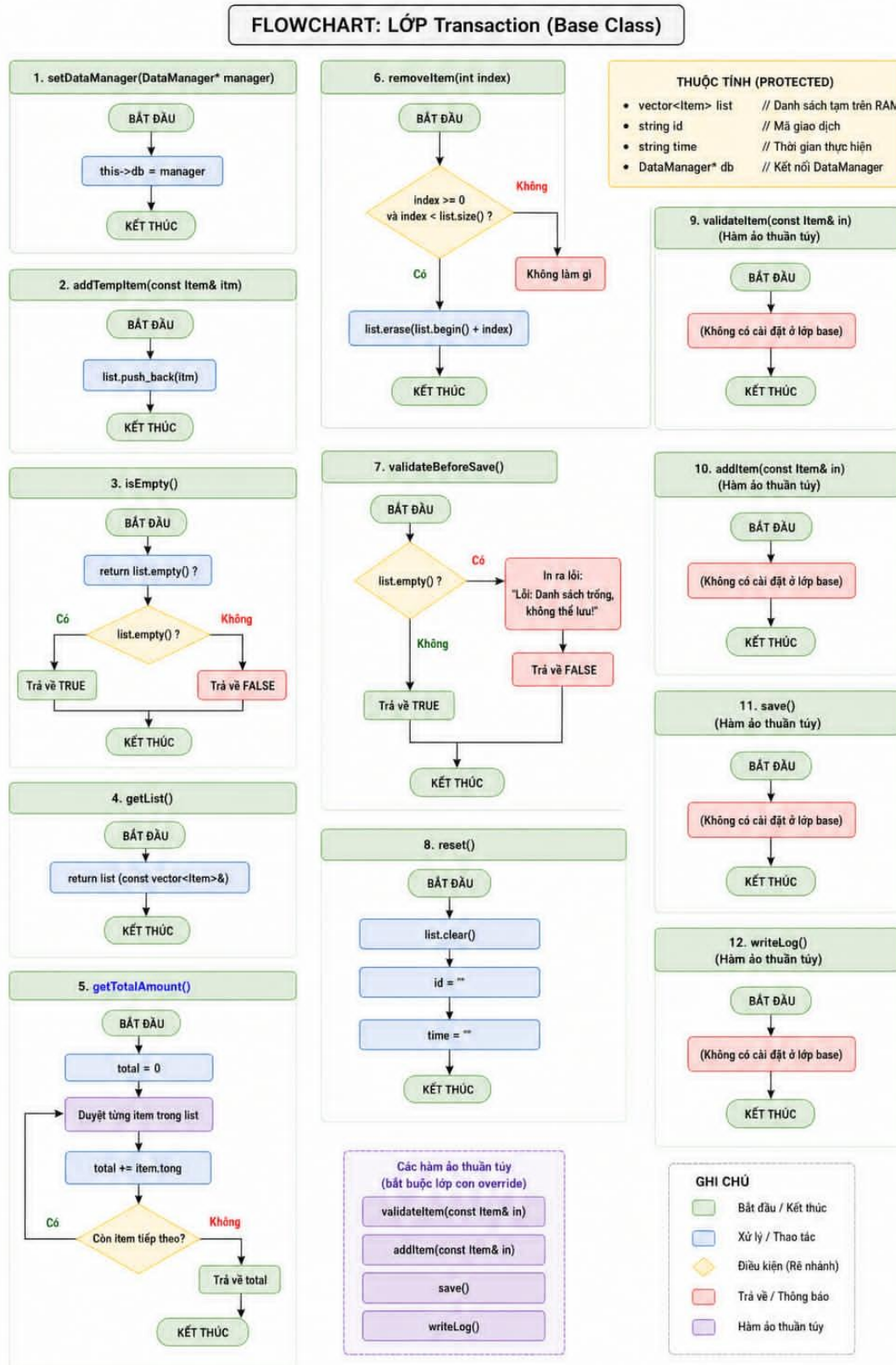
Hình 3.16. Lưu đồ Controller

3.1.4.4 Lưu đồ Quản lý hệ thống (SystemManager)



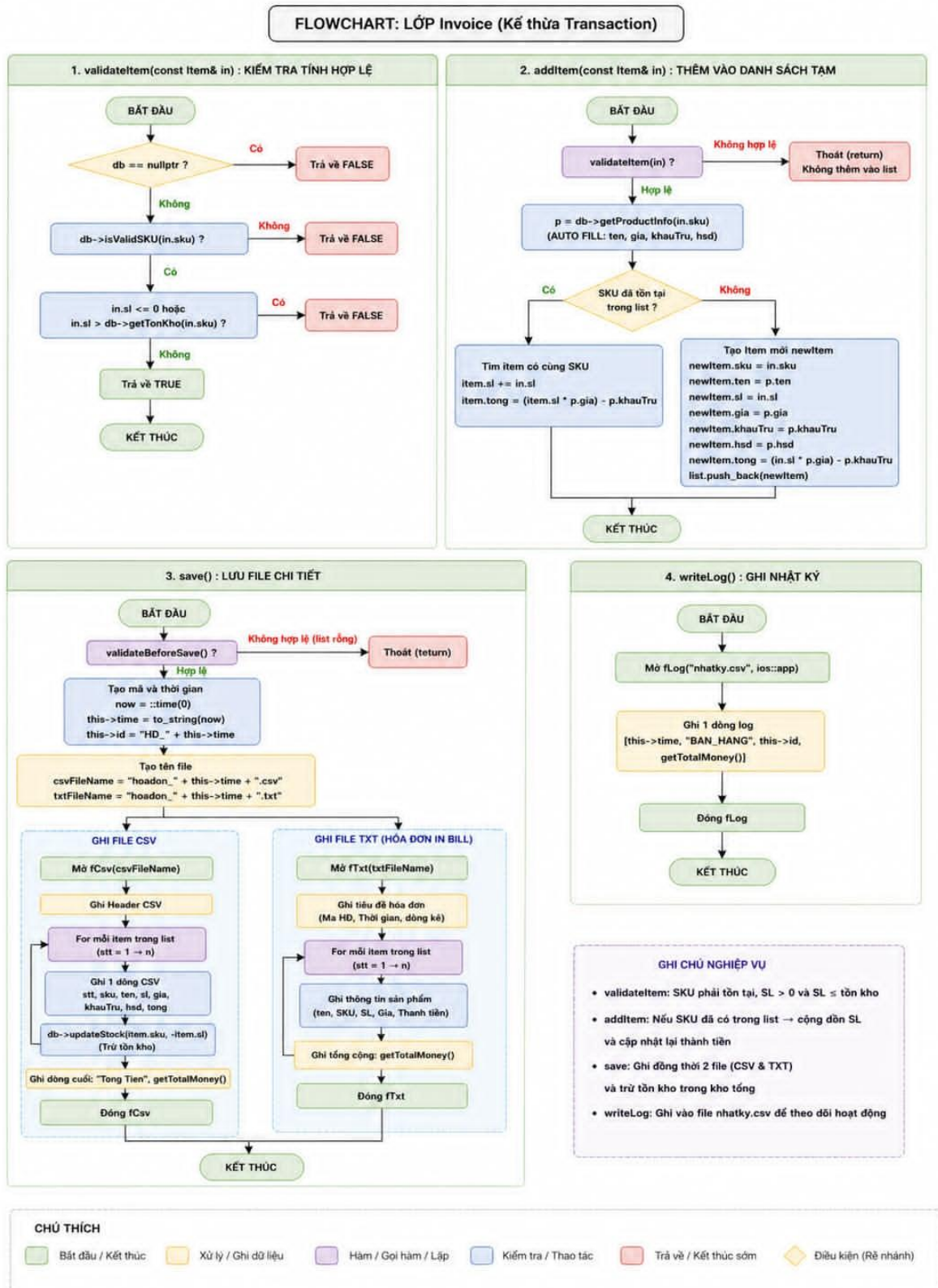
Hình 3.17. Lưu đồ Quản lý hệ thống

3.1.4.5. Lưu đồ Cơ chế Giao dịch chung (Transaction)



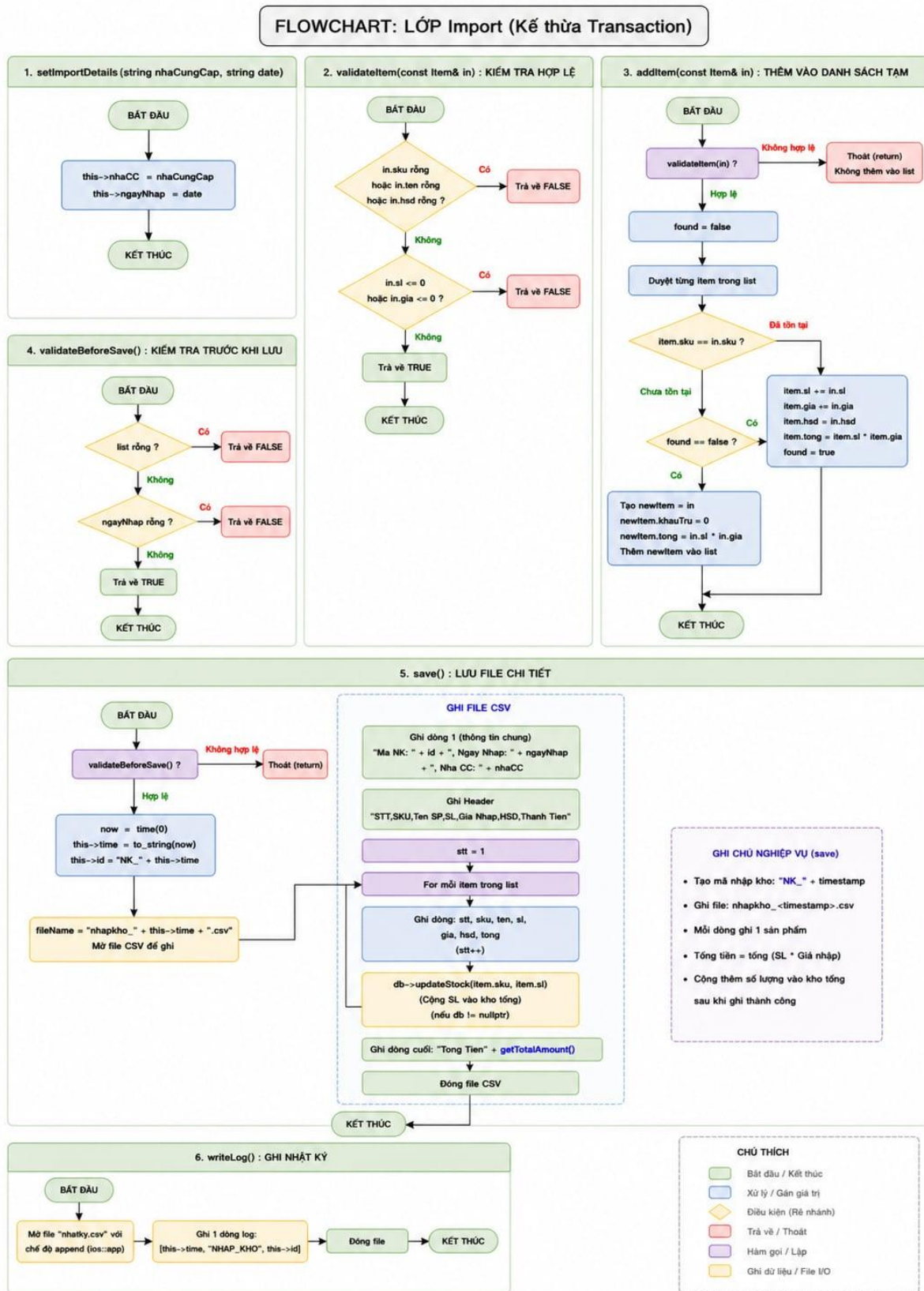
Hình 3.18. Lưu đồ Cơ chế Giao dịch chung

3.1.4.6. Lưu đồ Nghiệp vụ Bán hàng (Invoice)



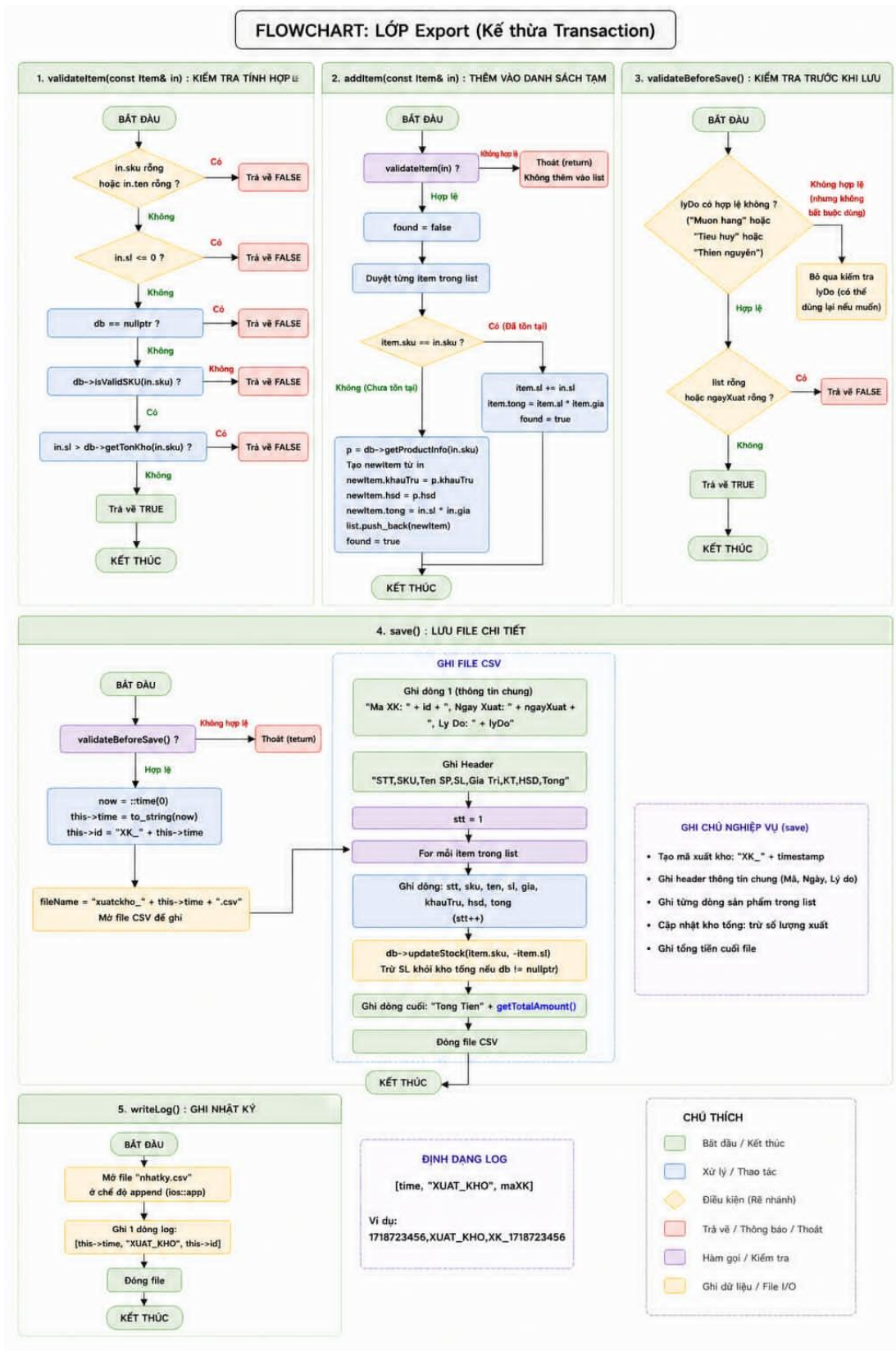
Hình 3.19. Lưu đồ Nghiệp vụ Bán hàng

3.1.4.7. Lưu đồ Nghiệp vụ Nhập kho (Import)



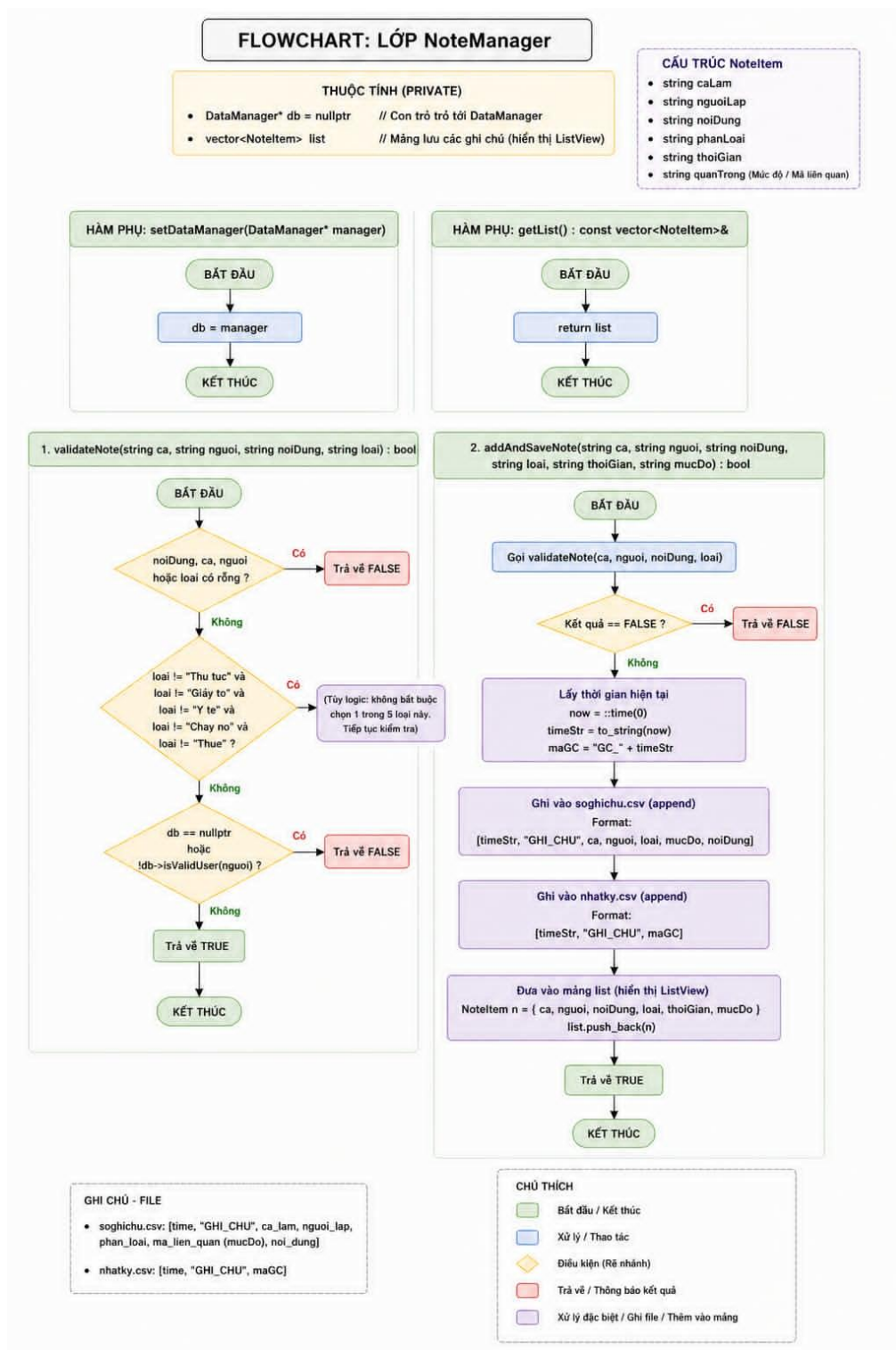
Hình 3.20. Lưu đồ Nghiệp vụ Nhập kho

3.1.4.8. Lưu đồ Nghiệp vụ Xuất kho (Export)



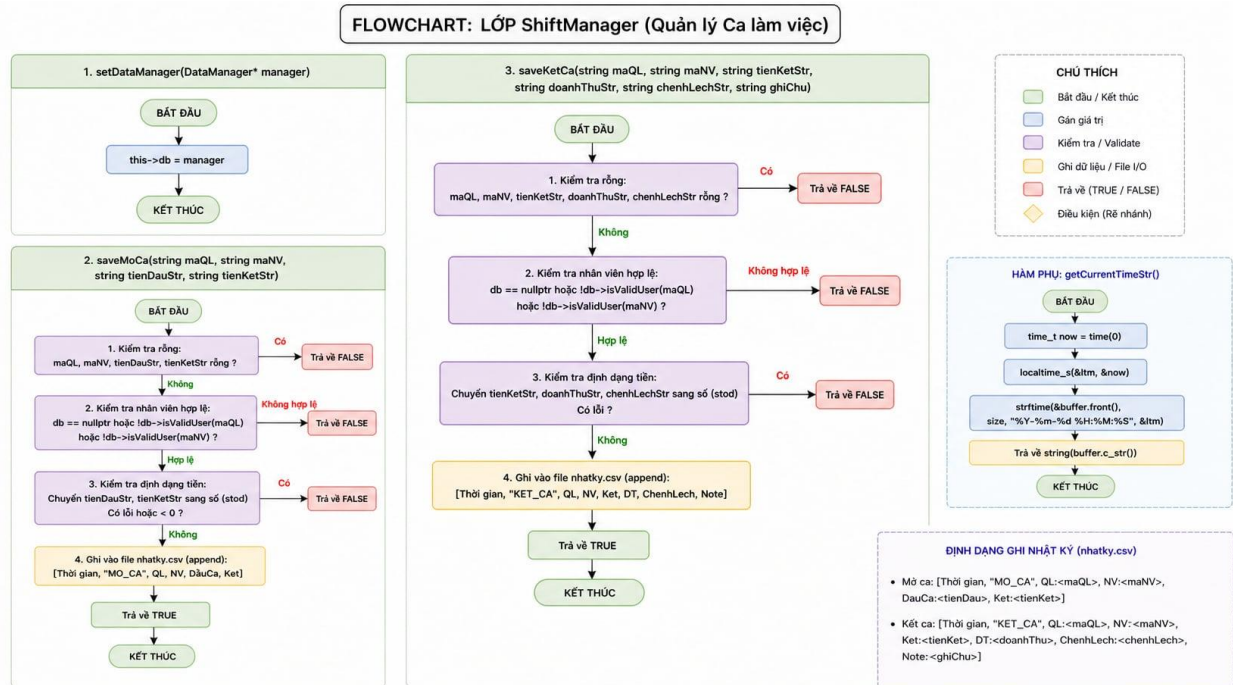
Hình 3.21. Lưu đồ Nghiệp vụ Xuất kho

3.1.4.9. Lưu đồ Xử lý Sổ ghi chú thời gian thực (NoteManager)



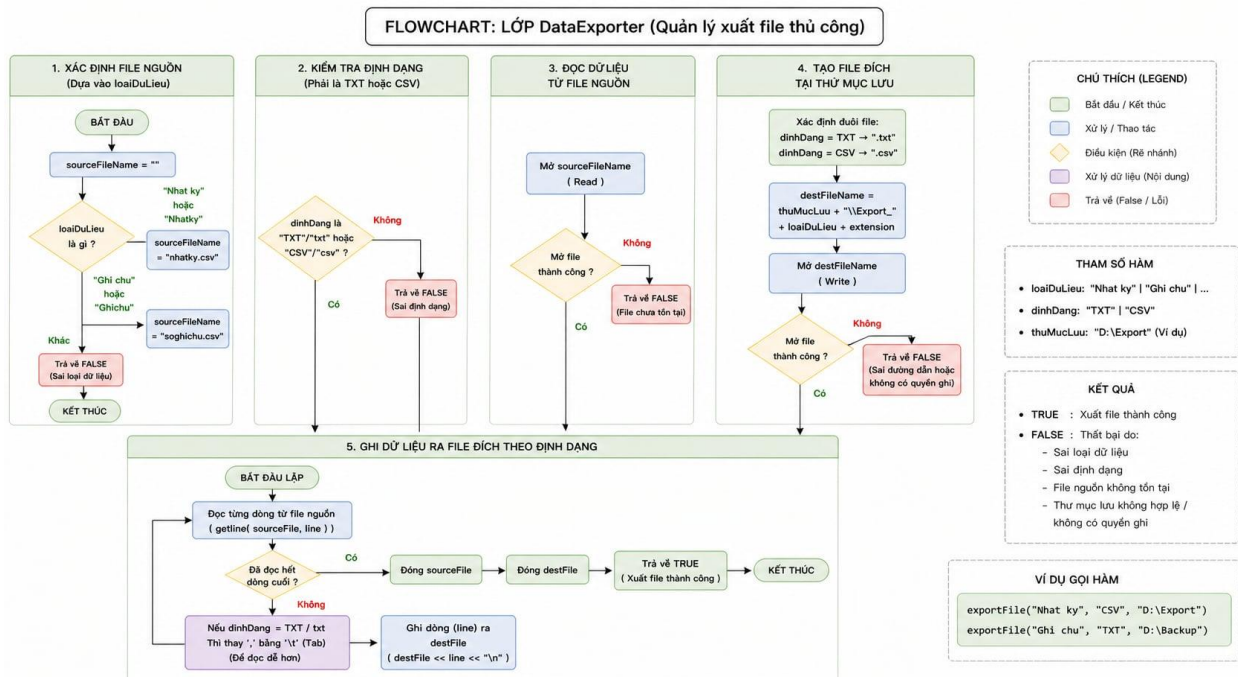
Hình 3.22. Lưu đồ Xử lý Sổ ghi chú thời gian thực

3.1.4.10. Lưu đồ Quản lý Ca làm việc (ShiftManager)



Hình 3.23. Lưu đồ Quản lý Ca làm việc

3.1.4.11. Lưu đồ xuất báo cáo (DataExporter)



Hình 3.24. Lưu đồ xuất báo cáo

3.2. Chương trình C++

Bảng 3.1. Chương trình GUI.cpp

```
#include <windows.h> // Thư viện cốt lõi của Windows (quản lý cửa sổ, thông điệp)
#include <gdiplus.h> // Thư viện đồ họa nâng cao (để vẽ giao diện, logo, màu sắc đẹp hơn)
#include <vector>
#include <string>
#include <comctl.h> // Thư viện các trình điều khiển chung (như ListView, Button hiện đại)

// --- NHÚNG TOÀN BỘ CÁC MODULE NGHIỆP VỤ ĐÃ VIẾT ---
#include "11b_DataManager.h" // Quản lý kho dữ liệu RAM
#include "0b_UploadManager.h" // Nạp dữ liệu từ file
#include "1b_Transaction.h" // Lớp cha Giao dịch
#include "2b_Invoice.h" // Bán hàng
#include "3b_Import.h" // Nhập kho
#include "4b_Export.h" // Xuất kho
#include "6b_NoteManager.h" // Ghi chú
#include "7b_ShiftManager.h" // Quản lý ca làm việc
#include "8a_DataExporter.h" // Xuất báo cáo file
#include "12b_SystemManager.h" // Quản lý nguồn (Bật/Tắt hệ thống)
#include "10b_Controller.h" // Bộ điều khiển trung tâm (Bộ não)

// --- KHỞI TẠO ĐỐI TƯỢNG TOÀN CỤC ---
// Các đối tượng này tồn tại xuyên suốt vòng đời của ứng dụng
Controller appCtrl; // Đối tượng điều khiển mọi logic nghiệp vụ
SystemManager sysManager; // Đối tượng quản lý trạng thái đóng/mở máy

// --- CẤU HÌNH LIÊN KẾT THƯ VIỆN HỆ THỐNG ---
// Lệnh pragma giúp trình biên dịch tự động tìm và nạp các file thư viện .lib cần thiết
#pragma comment (lib, "Gdiplus.lib") // Liên kết thư viện đồ họa GDI+
#pragma comment (lib, "comctl32.lib") // Liên kết thư viện Common Controls (ListView,
ProgressBar...)

using namespace std;
using namespace Gdiplus; // Sử dụng không gian tên của GDI+ để gọi các hàm đồ họa ngắn gọn hơn

// ===== ID ĐỊNH DANH =====
#define BTN_POWER 51
#define BTN_BAT_DAU 52
#define BTN_KET_THUC 53
#define BTN_BAN_HANG 54
#define BTN_NHAP_KHO 55
#define BTN_XUAT_KHO 56
```

```

#define BTN_GHI_CHU 57
#define BTN_LUU_XUAT 58
#define BTN_UPLOAD 59
#define BTN_KEY_BASE 1000

#define TXT_BH_SKU 401
#define TXT_BH_TEN 402
#define TXT_BH_SL 403
#define TXT_BH_GIA 404
#define TXT_BH_KHAUTRU 405
#define TXT_BH_HSD 406
#define BTN_BH_LUU_KE_TIEP 407
#define LV_BH_LIST 408

#define TXT_NH_SKU 501
#define TXT_NH_TEN 502
#define TXT_NH_SL 503
#define TXT_NH_GIA 504
#define TXT_NH_NCC 505
#define TXT_NH_HSD 506
#define BTN_NH_LUU 507
#define LV_NH_LIST 508

#define TXT_XH_SKU 601
#define TXT_XH_TEN 602
#define TXT_XH_SL 603
#define TXT_XH_GIA 604
#define TXT_XH_LYDO 605
#define TXT_XH_NGAY 606
#define BTN_XH_LUU 607
#define LV_XH_LIST 608

#define TXT_GC_CA 701
#define TXT_GC_NGUOILAP 702
#define TXT_GC_NOIDUNG 703
#define TXT_GC_PHANLOAI 704
#define TXT_GC_THOIGIAN 705
#define TXT_GC_MALK 706
#define BTN_GC_LUU 707
#define LV_GC_LIST 708

#define TXT_MOCA_QL 201
#define TXT_MOCA_NV 202
#define TXT_MOCA_TIENDAU 203

```

```

#define TXT_MOCA_TIENKET_DAU 204
#define BTN_MOCA_LUU 205
#define TXT_KETCA_QL 301
#define TXT_KETCA_NV 302
#define TXT_KETCA_TIENKET_CUOI 303
#define TXT_KETCA_DOANHTHU 304
#define TXT_KETCA_CHENHLECH 305
#define TXT_KETCA_GHICHU 306
#define BTN_KETCA_LUU 307

const wchar_t* layout[] = {
    L"1", L"2", L"3", L"4", L"5", L"6", L"7", L"8", L"9", L"0",
    L"Q", L"W", L"E", L"R", L"T", L"Y", L"U", L"I", L"O", L"P",
    L"A", L"S", L"D", L"F", L"G", L"H", L"J", L"K", L"L", L"Enter",
    L"Z", L"X", L"C", L"V", L"B", L"N", L"M", L"<-", L"Up", L"Dn", L"Lt", L"Rt"
};

// ===== BIEN TOAN CUC UI =====

HWND hActiveEdit, hScreenGroup;
HWND hSidebar[10], hSidebarBtns[10];
HWND hKeys[42], hSpaceGroup, hSpaceBtn, hDoanhThuLabel;
HWND hListViewBH, hListViewNH, hListViewXH, hListViewGC;
vector<HWND> hBHInputs, hNHInputs, hXHInputs, hGCInputs, hMoCaInputs, hKetCaInputs;

// ===== HAM TIEN ICH =====

/**
 * Hàm RenderListViewColumns: Thiết lập tiêu đề và độ rộng các cột cho bảng (ListView)
 * @param hListView: Handle của bảng cần cấu hình
 * @param cols: Danh sách các tiêu đề cột (dạng chuỗi wide string)
 * @param widths: Danh sách độ rộng tương ứng cho từng cột (đơn vị pixel)
 */
void RenderListViewColumns(HWND hListView, const vector<wstring>& cols, const vector<int>& widths) {

    // 1. Kích hoạt tính năng mở rộng: Hiện đường kẻ bảng (Gridlines) và cho phép chọn cả dòng khi
    nhấn chuột
    ListView_SetExtendedListViewStyle(hListView, LVS_EX_GRIDLINES |
    LVS_EX_FULLROWSELECT);

    // 2. Khởi tạo cấu trúc LVCOLUMN để định nghĩa thông số cột
    LVCOLUMN lvc = { 0 };

```

```

// Mask xác định các thuộc tính sẽ được thiết lập: Định dạng, Độ rộng, Văn bản và Chỉ số cột phụ
lvc.mask = LVCF_FMT | LVCF_WIDTH | LVCF_TEXT | LVCF_SUBITEM;
lvc.fmt = LVCFMT_LEFT; // Căn lề trái cho nội dung trong cột

// 3. Duyệt qua danh sách để chèn từng cột vào bảng
for (size_t i = 0; i < cols.size(); i++) {
    lvc.iSubItem = (int)i;          // Chỉ số của cột (từ 0)
    lvc.cx = widths[i];            // Thiết lập độ rộng pixel từ vector widths
    lvc.pszText = (LPWSTR)cols[i].c_str(); // Gán nội dung tiêu đề từ vector cols

    // Thực hiện lệnh chèn cột vào ListView tại vị trí i
    ListView_InsertColumn(hListView, (int)i, &lvc);
}
}

void ShowTab(int tab) {
    for (HWND h : hBHInputs) ShowWindow(h, SW_HIDE);
    for (HWND h : hNHInputs) ShowWindow(h, SW_HIDE);
    for (HWND h : hXHInputs) ShowWindow(h, SW_HIDE);
    for (HWND h : hGCInputs) ShowWindow(h, SW_HIDE);
    for (HWND h : hMoCaInputs) ShowWindow(h, SW_HIDE);
    for (HWND h : hKetCaInputs) ShowWindow(h, SW_HIDE);

    ShowWindow(hListViewBH, SW_HIDE); ShowWindow(hListViewNH, SW_HIDE);
    ShowWindow(hListViewXH, SW_HIDE); ShowWindow(hListViewGC, SW_HIDE);

    switch (tab) {
    case BTN_BAN_HANG:
        SetWindowText(hScreenGroup, L"MAN HINH: BAN HANG");
        for (HWND h : hBHInputs) ShowWindow(h, SW_SHOW);
        ShowWindow(hListViewBH, SW_SHOW); break;
    case BTN_NHAP_KHO:
        SetWindowText(hScreenGroup, L"MAN HINH: NHAP KHO");
        for (HWND h : hNHInputs) ShowWindow(h, SW_SHOW); ShowWindow(hListViewNH,
SW_SHOW); break;
    case BTN_XUAT_KHO:
        SetWindowText(hScreenGroup, L"MAN HINH: XUAT KHO");
        for (HWND h : hXHInputs) ShowWindow(h, SW_SHOW); ShowWindow(hListViewXH,
SW_SHOW); break;
    case BTN_GHI_CHU:
        SetWindowText(hScreenGroup, L"MAN HINH: SO GHI CHU");
        for (HWND h : hGCInputs) ShowWindow(h, SW_SHOW); ShowWindow(hListViewGC,

```



```

SW_SHOW); break;
case BTN_BAT_DAU:
    SetWindowText(hScreenGroup, L"MAN HINH: MO CA");
    for (HWND h : hMoCaInputs) ShowWindow(h, SW_SHOW); break;
case BTN_KET_THUC:
    SetWindowText(hScreenGroup, L"MAN HINH: KET CA");
    for (HWND h : hKetCaInputs) ShowWindow(h, SW_SHOW); break;
case -1: SetWindowText(hScreenGroup, L""); break;
}
}

/**
 * Hàm ToggleGUI: Ẩn hoặc hiện toàn bộ các thành phần giao diện chính
 * @param hWnd: Handle của cửa sổ chính
 * @param show: true để hiện giao diện, false để ẩn giao diện
 */
void ToggleGUI(HWND hWnd, bool show) {
    // 1. Xác định lệnh thực hiện: SW_SHOW (Hiện thị) hoặc SW_HIDE (Ẩn)
    int cmd = show ? SW_SHOW : SW_HIDE;

    // 2. Duyệt qua mảng Sidebar: Ẩn/Hiện các Panel và các nút bấm điều hướng (từ chỉ số 1 đến 8)
    for (int i = 1; i < 9; i++) {
        ShowWindow(hSidebar[i], cmd);
        ShowWindow(hSidebarBtns[i], cmd);
    }

    // 3. Duyệt qua mảng bàn phím ảo: Ẩn/Hiện toàn bộ 42 phím bấm
    for (int i = 0; i < 42; i++) ShowWindow(hKeys[i], cmd);

    // 4. Ẩn/Hiện các thành phần hỗ trợ khác:
    ShowWindow(hSpaceGroup, cmd); // Nhóm phím cách (Space)
    ShowWindow(hSpaceBtn, cmd); // Nút phím cách
    ShowWindow(hDoanhThuLabel, cmd); // Nhãn hiển thị doanh thu
    ShowWindow(hScreenGroup, cmd); // Vùng hiển thị nội dung chính (Screen)

    // 5. Logic bổ sung khi TẮT giao diện:
    // Gọi ShowTab(-1) để đóng tất cả các tab đang mở, đưa về trạng thái trống
    if (!show) ShowTab(-1);

    // 6. Yêu cầu hệ thống vẽ lại cửa sổ chính để cập nhật thay đổi hình ảnh ngay lập tức
    InvalidateRect(hWnd, NULL, TRUE);
}

```

```

void DrawLargeLogo(Graphics& g, int centerX, int centerY, int baseSize) {
    float size = (float)baseSize * 4.0f;
    g.SetSmoothingMode(SmoothingModeAntiAlias);
    SolidBrush redBrush(Color(255, 230, 0, 0));
    g.FillEllipse(&redBrush, (float)centerX - size / 2.0f, (float)centerY - size / 2.0f, size, size);

    FontFamily fontFamily(L"Segoe UI");
    Font goFont(&fontFamily, size / 2.2f, FontStyleBold, UnitPixel);
    SolidBrush whiteBrush(Color(255, 255, 255, 255));

    StringFormat format;
    format.SetAlignment(StringAlignmentCenter);
    format.SetLineAlignment(StringAlignmentCenter);

    RectF rectGo((float)centerX - size / 2.0f, (float)centerY - size / 2.0f, size, size);
    g.DrawString(L"GO", -1, &goFont, rectGo, &format, &whiteBrush);
}

// ===== WNDPROC =====
LRESULT CALLBACK WndProc(HWND hWnd, UINT msg, WPARAM wp, LPARAM lp) {
    switch (msg) {
        case WM_CREATE: {
            InitCommonControls();
            const wchar_t* sbT[] = { L"Power", L"File", L"Upload", L"Mo Ca", L"Ket Ca", L"Ban Hang",
L"Nhap Kho", L"Xuat Kho", L"Ghi Chu" };
            const wchar_t* btnT[] = { L"ON/OFF", L"LUU/XUAT", L"NAP DU LIEU", L"BAT DAU",
L"KET THUC", L"BAN HANG", L"NHAP KHO", L"XUAT KHO", L"GHI CHU" };
            const HMENU ids[] = { (HMENU)BTN_POWER, (HMENU)BTN_LUU_XUAT,
(HMENU)BTN_UPLOAD, (HMENU)BTN_BAT_DAU, (HMENU)BTN_KET_THUC,
(HMENU)BTN_BAN_HANG, (HMENU)BTN_NHAP_KHO, (HMENU)BTN_XUAT_KHO,
(HMENU)BTN_GHI_CHU };

            for (int i = 0; i < 9; i++) {
                hSidebar[i] = CreateWindow(L"BUTTON", sbT[i], WS_CHILD | WS_VISIBLE |
BS_GROUPBOX, 0, 0, 0, 0, hWnd, NULL, NULL, NULL);
                hSidebarBtns[i] = CreateWindow(L"BUTTON", btnT[i], WS_CHILD | WS_VISIBLE, 0, 0,
0, 0, hWnd, ids[i], NULL, NULL);
            }
            hScreenGroup = CreateWindow(L"BUTTON", L"MAN HINH: CHO", WS_CHILD |
WS_VISIBLE | BS_GROUPBOX, 0, 0, 0, 0, hWnd, NULL, NULL, NULL);

// =====TAB BAN HANG=====
            const wchar_t* bhl[] = { L"Ma SKU:", L"Ten Hang:", L"So Luong:", L"Don Gia:", L"Khau
Tru:", L"HSD:" };

```

```

HMENU bhi[] = { (HMENU)TXT_BH_SKU, (HMENU)TXT_BH_TEN,
(HMENU)TXT_BH_SL, (HMENU)TXT_BH_GIA, (HMENU)TXT_BH_KHAUTRU,
(HMENU)TXT_BH_HSD };
for (int i = 0; i < 6; i++) {
    hBHInputs.push_back(CreateWindow(L"STATIC", bhl[i], WS_CHILD, 0, 0, 0, 0, hWnd,
NULL, NULL, NULL));
    hBHInputs.push_back(CreateWindow(L"EDIT", L"", WS_CHILD | WS_BORDER | (i == 1 ||
i >= 3 ? ES_READONLY : 0), 0, 0, 0, 0, hWnd, bhi[i], NULL, NULL));
}
hBHInputs.push_back(CreateWindow(L"BUTTON", L"LUU KE TIEP", WS_CHILD, 0, 0, 0,
0, hWnd, (HMENU)BTN_BH_LUU_KE_TIEP, NULL, NULL));

// =====TAB NHAP KHO=====
const wchar_t* nhl[] = { L"Ma SKU:", L"Ten Hang:", L"SL Nhap:", L"Gia Nhap:", L"Nha
CC:", L"HSD:" };
HMENU nhi[] = { (HMENU)TXT_NH_SKU, (HMENU)TXT_NH_TEN,
(HMENU)TXT_NH_SL, (HMENU)TXT_NH_GIA, (HMENU)TXT_NH_NCC,
(HMENU)TXT_NH_HSD };
for (int i = 0; i < 6; i++) {
    hNHInputs.push_back(CreateWindow(L"STATIC", nhl[i], WS_CHILD, 0, 0, 0, 0, hWnd,
NULL, NULL, NULL));
    hNHInputs.push_back(CreateWindow(L"EDIT", L"", WS_CHILD | WS_BORDER, 0, 0, 0,
0, hWnd, nhi[i], NULL, NULL));
}
hNHInputs.push_back(CreateWindow(L"BUTTON", L"LUU NHAP KHO", WS_CHILD, 0, 0,
0, 0, hWnd, (HMENU)BTN_NH_LUU, NULL, NULL));
// =====TAB XUAT KHO=====
const wchar_t* xhl[] = { L"Ma SKU:", L"Ten Hang:", L"SL Xuat:", L"Gia Tri:", L"Ly Do:",
L"Ngay Xuat:" };
HMENU xhi[] = { (HMENU)TXT_XH_SKU, (HMENU)TXT_XH_TEN,
(HMENU)TXT_XH_SL, (HMENU)TXT_XH_GIA, (HMENU)TXT_XH_LYDO,
(HMENU)TXT_XH_NGAY };

for (int i = 0; i < 6; i++) {
    hXHInputs.push_back(CreateWindow(L"STATIC", xhl[i], WS_CHILD, 0, 0, 0, 0, hWnd,
NULL, NULL, NULL));

    // Dùng EDIT thường cho tất cả các ô để bàn phím ảo POS hoạt động mượt mà
    int style = WS_CHILD | WS_BORDER | ES_AUTOHSCROLL;

    // Tự động KHÓA "Chỉ đọc" cho ô Tên Hàng (i==1) và Giá Trị (i==3)
    // vì 2 ô này hệ thống sẽ tự động điền khi gõ xong SKU
    if (i == 1 || i == 3) {
        style |= ES_READONLY;
    }
}

```

```

    }

    hXHInputs.push_back(CreateWindow(L"EDIT", L"", style, 0, 0, 0, 0, hWnd, xhi[i], NULL,
    NULL));
    }
    hXHInputs.push_back(CreateWindow(L"BUTTON", L"LUU XUAT KHO", WS_CHILD, 0, 0,
    0, 0, hWnd, (HMENU)BTN_XH_LUU, NULL, NULL));

    // =====TAB GHI CHU=====
    const wchar_t* gcl[] = { L"Ca Lam:", L"Nguoi Lap:", L"Noi Dung:", L"Phan Loai:", L"Thoi
    Gian:", L"Ma LK:" };
    HMENU gci[] = { (HMENU)TXT_GC_CA, (HMENU)TXT_GC_NGUOILAP,
    (HMENU)TXT_GC_NOIDUNG, (HMENU)TXT_GC_PHANLOAI,
    (HMENU)TXT_GC_THOIGIAN, (HMENU)TXT_GC_MALK };

    for (int i = 0; i < 6; i++) {
        hGCInputs.push_back(CreateWindow(L"STATIC", gcl[i], WS_CHILD, 0, 0, 0, 0, hWnd,
        NULL, NULL, NULL));

        // XÓA BỎ COMBOBOX! Dùng EDIT thường cho TẤT CẢ các ô

        hGCInputs.push_back(CreateWindow(L"EDIT", L"", WS_CHILD | WS_BORDER |
        ES_AUTOHSCROLL, 0, 0, 0, 0, hWnd, gci[i], NULL, NULL));
    }

    hGCInputs.push_back(CreateWindow(L"BUTTON", L"LUU GHI CHU", WS_CHILD, 0, 0, 0,
    0, hWnd, (HMENU)BTN_GC_LUU, NULL, NULL));
    // KHOI TAO LISTVIEWS VOI DUONG LUOI (GRIDLINES)
    hListViewBH = CreateWindow(WC_LISTVIEW, L"", WS_CHILD | WS_BORDER |
    LVS_REPORT, 0, 0, 0, 0, hWnd, (HMENU)LV_BH_LIST, NULL, NULL);
    hListViewNH = CreateWindow(WC_LISTVIEW, L"", WS_CHILD | WS_BORDER |
    LVS_REPORT, 0, 0, 0, 0, hWnd, (HMENU)LV_NH_LIST, NULL, NULL);
    hListViewXH = CreateWindow(WC_LISTVIEW, L"", WS_CHILD | WS_BORDER |
    LVS_REPORT, 0, 0, 0, 0, hWnd, (HMENU)LV_XH_LIST, NULL, NULL);
    hListViewGC = CreateWindow(WC_LISTVIEW, L"", WS_CHILD | WS_BORDER |
    LVS_REPORT, 0, 0, 0, 0, hWnd, (HMENU)LV_GC_LIST, NULL, NULL);

    RenderListViewColumns(hListViewBH, { L"STT", L"SKU", L"Ten Hang", L"SL", L"Gia",
    L"KT", L"HSD", L"Tong" }, { 40, 70, 150, 50, 80, 50, 90, 100 });
    RenderListViewColumns(hListViewNH, { L"STT", L"SKU", L"Ten Hang", L"SL", L"Gia",
    L"KT", L"HSD", L"Tong" }, { 40, 70, 150, 50, 80, 50, 90, 100 });
    RenderListViewColumns(hListViewXH, { L"STT", L"SKU", L"Ten Hang", L"SL", L"Gia",
    L"KT", L"HSD", L"Tong" }, { 40, 70, 150, 50, 80, 50, 90, 100 });
    RenderListViewColumns(hListViewGC, { L"STT", L"Ca", L"Nguoi lap", L"Loai", L"Time",

```

```

L"Noi dung", L"Ma LK" }, { 40, 60, 100, 80, 80, 150, 80 });

// ===== MO/KET CA =====
const wchar_t* mcl[] = { L"Ma QL:", L"Ma NV:", L"Tien dau:", L"Tien ket:" };
HMENU mci[] = { (HMENU)TXT_MOCA_QL, (HMENU)TXT_MOCA_NV,
(HMENU)TXT_MOCA_TIENDAUI, (HMENU)TXT_MOCA_TIENKET_DAU };
for (int i = 0; i < 4; i++) {
    hMoCaInputs.push_back(CreateWindow(L"STATIC", mcl[i], WS_CHILD, 0, 0, 0, 0, hWnd,
NULL, NULL, NULL));
    hMoCaInputs.push_back(CreateWindow(L"EDIT", L"", WS_CHILD | WS_BORDER, 0, 0,
0, 0, hWnd, mci[i], NULL, NULL));
}
hMoCaInputs.push_back(CreateWindow(L"BUTTON", L"LUU MO CA", WS_CHILD, 0, 0, 0,
0, hWnd, (HMENU)BTN_MOCA_LUU, NULL, NULL));

const wchar_t* kcl[] = { L"Ma QL:", L"Ma NV:", L"Tien ket:", L"Doanh thu:", L"Chenh lech:",
L"Ghi chu:" };
HMENU kci[] = { (HMENU)TXT_KETCA_QL, (HMENU)TXT_KETCA_NV,
(HMENU)TXT_KETCA_TIENKET_CUOI, (HMENU)TXT_KETCA_DOANHTHU,
(HMENU)TXT_KETCA_CHENHLECH, (HMENU)TXT_KETCA_GHICHU };
for (int i = 0; i < 6; i++) {
    hKetCaInputs.push_back(CreateWindow(L"STATIC", kcl[i], WS_CHILD, 0, 0, 0, 0, hWnd,
NULL, NULL, NULL));
    hKetCaInputs.push_back(CreateWindow(L"EDIT", L"", WS_CHILD | WS_BORDER, 0, 0,
0, 0, hWnd, kci[i], NULL, NULL));
}
hKetCaInputs.push_back(CreateWindow(L"BUTTON", L"LUU KET CA", WS_CHILD, 0, 0,
0, 0, hWnd, (HMENU)BTN_KETCA_LUU, NULL, NULL));

for (int i = 0; i < 42; i++) hKeys[i] = CreateWindow(L"BUTTON", layout[i], WS_CHILD |
WS_VISIBLE, 0, 0, 0, 0, hWnd, (HMENU)(size_t)(BTN_KEY_BASE + i), NULL, NULL);
hSpaceGroup = CreateWindow(L"BUTTON", L"Space", WS_CHILD | WS_VISIBLE |
BS_GROUPBOX, 0, 0, 0, 0, hWnd, NULL, NULL, NULL);
hSpaceBtn = CreateWindow(L"BUTTON", L"SPACE", WS_CHILD | WS_VISIBLE, 0, 0, 0, 0,
hWnd, (HMENU)(size_t)(BTN_KEY_BASE + 99), NULL, NULL);
hDoanhThuLabel = CreateWindow(L"STATIC", L"DOANH THU: 0", WS_CHILD |
WS_VISIBLE, 0, 0, 0, 0, hWnd, NULL, NULL, NULL);

ToggleGUI(hWnd, false);
break;
}
case WM_SIZE: {
    int w = LOWORD(lp), h = HIWORD(lp), sbW = 165;
    for (int i = 0; i < 9; i++) {

```

```

        MoveWindow(hSidebar[i], 5, i * 70 + 5, sbW - 10, 65, TRUE);
        MoveWindow(hSidebarBtns[i], 15, i * 70 + 25, sbW - 30, 35, TRUE);
    }
    int mX = sbW + 5, mW = w - mX - 5, scrH = 450;
    MoveWindow(hScreenGroup, mX, 5, mW, scrH, TRUE);

    auto arrange = [&](vector<HWND>& inputs, HWND lv) {
        for (int i = 0; i < 6; i++) {
            MoveWindow(inputs[i * 2], mX + 15, 30 + i * 42, 100, 20, TRUE);
            MoveWindow(inputs[i * 2 + 1], mX + 15, 50 + i * 42, 200, 25, TRUE);
        }
        MoveWindow(inputs.back(), mX + 15, scrH - 50, 200, 40, TRUE);
        MoveWindow(lv, mX + 230, 25, mW - 245, scrH - 35, TRUE);
    };

    arrange(hBHInputs, hListViewBH); arrange(hNHInputs, hListViewNH); arrange(hXHInputs,
hListViewXH);
    arrange(hGCInputs, hListViewGC);

    for (int i = 0; i < 4; i++) {
        MoveWindow(hMoCaInputs[i * 2], mX + 20, 35 + i * 35, 100, 20, TRUE);
        MoveWindow(hMoCaInputs[i * 2 + 1], mX + 130, 33 + i * 35, 180, 25, TRUE);
    }
    MoveWindow(hMoCaInputs.back(), mX + mW - 140, scrH - 55, 120, 40, TRUE);

    for (int i = 0; i < 6; i++) {
        MoveWindow(hKetCaInputs[i * 2], mX + 20, 30 + i * 30, 100, 20, TRUE);
        MoveWindow(hKetCaInputs[i * 2 + 1], mX + 130, 28 + i * 30, 180, 25, TRUE);
    }
    MoveWindow(hKetCaInputs.back(), mX + mW - 140, scrH - 55, 120, 40, TRUE);

    int kY = scrH + 10, kw = mW / 10, kh = (h - kY - 95) / 5;
    for (int i = 0; i < 42; i++) MoveWindow(hKeys[i], mX + (i % 10) * kw, kY + (i / 10) * kh, kw -
2, kh - 2, TRUE);
    MoveWindow(hSpaceGroup, mX, h - 85, mW, 55, TRUE);
    MoveWindow(hSpaceBtn, mX + 10, h - 65, mW - 20, 30, TRUE);
    MoveWindow(hDoanhThuLabel, w - 200, h - 25, 180, 20, TRUE);
    break;
}
case WM_COMMAND: {
    int id = LOWORD(wp);

    // =====
    if (id >= BTN_BAT_DAU && id <= BTN_GHI_CHU) {

```

```

ShowTab(id);          // 1. GUI tự hiển thị màn hình mới
appCtrl.switchTab(id); // 2. Controller tự đổi con trỏ đa hình tương ứng
}

// =====
if (id == BTN_POWER) {
    // 1. Kiểm tra trạng thái máy TRƯỚC KHI bấm
    bool wasOn = sysManager.getStatus();

    // 2. Gọi hàm bật/tắt (hệ thống tự động lưu ngầm bên trong 12b nếu đang ON)
    bool isTurnedOn = sysManager.togglePower(&appCtrl);

    // 3. Nếu đang ON mà chuyển thành OFF -> Hiển thị "Đã lưu" theo đúng Lưu đồ [1]
    if (wasOn == true && isTurnedOn == false) {
        MessageBox(hWnd, L"Đã lưu tự động tắt cả dữ liệu đang chạy!", L"Hệ Thống", MB_OK |
        MB_ICONINFORMATION);
    }

    // 4. Cập nhật lại giao diện (Ẩn hết nếu tắt, hiện nếu bật)
    ToggleGUI(hWnd, sysManager.getStatus());
}

// =====BTN_LUU_XUAT=====
if (id == BTN_LUU_XUAT) {
    OPENFILENAME ofn;
    wchar_t szFile[MAX_PATH] = L"NhatKy";

    ZeroMemory(&ofn, sizeof(ofn));
    ofn.lStructSize = sizeof(ofn);
    ofn.hwndOwner = hWnd;
    ofn.lpstrFile = szFile;
    ofn.nMaxFile = sizeof(szFile) / sizeof(wchar_t); // Khai báo đúng kích thước buffer

    // Giới hạn người dùng CHỈ ĐƯỢC CHỌN TXT hoặc CSV theo Lưu đồ đồ án
    ofn.lpstrFilter = L"File CSV (*.csv)\0*.csv\0File TXT (*.txt)\0*.txt\0";
    ofn.nFilterIndex = 1; // Mặc định hiển thị ưu tiên .csv

    // Tiêu đề hướng dẫn người dùng trực quan trên hộp thoại
    ofn.lpstrTitle = L"BUỚC 1: Chọn Thư mục (X) | BUỚC 2: Nhập đúng ô File Name
    (NhatKy, HoaDon, NhapKho, XuatKho, GhiChu)";
    ofn.Flags = OFN_PATHMUSTEXIST | OFN_OVERWRITEPROMPT;

    // Nếu người dùng nhấn "Save / Xác nhận Export"
    if (GetSaveFileName(&ofn)) {

```



```

// 1. Chuyển đổi đường dẫn file từ WCHAR sang string tiêu chuẩn
wstring ws(szFile);
string fullPath(ws.begin(), ws.end());

// 2. Xác định "Định dạng" dựa vào tùy chọn (Filter) của người dùng
string dinhDang = (ofn.nFilterIndex == 1) ? "CSV" : "TXT";

// 3. Phân tách đường dẫn để lấy "Thư mục (X)" và "Loại dữ liệu"
string thuMuc = "";
string loiDuLieu = "";

size_t lastSlash = fullPath.find_last_of("\\");
size_t lastDot = fullPath.find_last_of(".");

if (lastSlash != string::npos && lastDot != string::npos) {
    thuMuc = fullPath.substr(0, lastSlash); // Tách lấy đoạn C:\Export
    loiDuLieu = fullPath.substr(lastSlash + 1, lastDot - lastSlash - 1); // Tách lấy từ NhatKy
}
else {
    thuMuc = fullPath; // Backup an toàn nếu không có dấu chấm
    loiDuLieu = "NhatKy";
}

// 4. GUI GỌI HÀM LOGIC (MVC-Lite): Truyền đúng 3 tham số vào lớp DataExporter
if (appCtrl.exporter.exportFile(loiDuLieu, dinhDang, thuMuc)) {
    MessageBox(hWnd, L"Xuất dữ liệu thành công ra thư mục đã chọn!", L"Hoàn tất",
MB_OK | MB_ICONINFORMATION);
}
else {
    MessageBox(hWnd, L"Lỗi: Không có dữ liệu hoặc sai Loại Dữ Liệu (Phải là: NhatKy,
HoaDon, NhapKho...)!", L"Lỗi Xuất File", MB_ICONERROR);
}
}

//
=====BTN_UPLOAD=====

if (id == BTN_UPLOAD) {
    OPENFILENAME ofn;
    wchar_t szFile[MAX_PATH] = { 0 };
    ZeroMemory(&ofn, sizeof(ofn));
    ofn.lStructSize = sizeof(ofn);
    ofn.hwndOwner = hWnd;

```

```

ofn.lpstrFile = szFile;
ofn.nMaxFile = MAX_PATH;
ofn.lpstrFilter = L"Data Files (*.csv; *.txt)\0*.csv;*.txt\0All Files (*.*)\0*.*\0";
ofn.nFilterIndex = 1;
ofn.Flags = OFN_PATHMUSTEXIST | OFN_FILEMUSTEXIST;

if (GetOpenFileName(&ofn)) {
    wstring ws(szFile); // Lấy đường dẫn thực tế từ hộp thoại

    // 1. Nếu tên file có chứa chữ "NhanSu" -> Gọi hàm nạp file 2 cột
    if (ws.find(L"NhanSu") != wstring::npos) {
        if (appCtrl.uploadObj.uploadUserData(ws)) {
            MessageBoxW(hWnd, L"Nạp danh sách NHÂN SỰ thành công!", L"Thông Báo",
MB_OK | MB_ICONINFORMATION);
        }
    }
    // 2. Ngược lại (nếu chọn file List.txt) -> Gọi hàm nạp file 8 cột
    else {
        if (appCtrl.uploadObj.uploadProductData(ws)) {
            MessageBoxW(hWnd, L"Nạp dữ liệu HÀNG HÓA thành công!", L"Thông Báo",
MB_OK | MB_ICONINFORMATION);
        }
    }
}

// Hàm lambda phụ trợ giúp lấy chữ từ ô nhập nhanh gọn
auto GetStrFromHWND = [](HWND h) {
    // [CHÚ Ý]: Tự tay gõ ngoặc vuông bọc số 256 nếu web vẫn xóa nhé!
    wchar_t buf[256] = { 0 };
    GetWindowTextW(h, buf, 256);
    wstring ws(buf);
    return string(ws.begin(), ws.end());
};

// =====BTN_MOCA_LUU=====
if (id == BTN_MOCA_LUU) {
    // Đã fix chuẩn vị trí các ô trống theo quy luật xen kẽ
    string ql = GetStrFromHWND(hMoCaInputs.at(1)); // Ô Mã QL
    string nv = GetStrFromHWND(hMoCaInputs.at(3)); // Ô Mã NV
    string tienDau = GetStrFromHWND(hMoCaInputs.at(5)); // Ô Tiền đầu
    string tienKet = GetStrFromHWND(hMoCaInputs.at(7)); // Ô Tiền kết

    if (appCtrl.shift.saveMoCa(ql, nv, tienDau, tienKet)) {
        MessageBoxW(hWnd, L"Mở ca thành công! Đã ghi nhật ký.", L"Thành công", MB_OK |
MB_ICONINFORMATION);
    }
}

```

```

    }
    else {
        MessageBoxW(hWnd, L"LỖI: Sai định dạng số, bỏ trống, hoặc Mã QL/NV không tồn tại!
Yêu cầu nhập lại.", L"Lỗi Validate", MB_OK | MB_ICONERROR);
    }
}

// =====BTN_KETCA_LUU=====

if (id == BTN_KETCA_LUU) {
    // Lấy 6 chuỗi dữ liệu từ các ô trống (Edit Control) theo quy luật số lẻ
    string ql = GetStrFromHWND(hKetCaInputs.at(1)); // Ô Mã QL
    string nv = GetStrFromHWND(hKetCaInputs.at(3)); // Ô Mã NV
    string tienKet = GetStrFromHWND(hKetCaInputs.at(5)); // Ô Tiền kết
    string doanhThu = GetStrFromHWND(hKetCaInputs.at(7)); // Ô Doanh thu
    string chenhLech = GetStrFromHWND(hKetCaInputs.at(9)); // Ô Chênh lệch
    string ghiChu = GetStrFromHWND(hKetCaInputs.at(11)); // Ô Ghi chú

    // Truyền xuống Controller xử lý [1]
    if (appCtrl.shift.saveKetCa(ql, nv, tienKet, doanhThu, chenhLech, ghiChu)) {
        MessageBoxW(hWnd, L"Kết ca thành công! Đã ghi nhật ký.", L"Thành công", MB_OK |
MB_ICONINFORMATION);
    }
    else {
        MessageBoxW(hWnd, L"LỖI: Sai định dạng số, bỏ trống, hoặc Mã QL/NV không tồn tại!
Yêu cầu nhập lại.", L"Lỗi Validate", MB_OK | MB_ICONERROR);
    }
}

// =====HÀM LAMBDA TRỢ GIÚP TÍNH TỔNG TIỀN=====

auto UpdateTongTien = [](HWND hLV, HWND hLabel) {
    int count = ListView_GetItemCount(hLV);
    double total = 0.0;
    for (int i = 0; i < count; i++) {
        // Tự gõ tay cập ngoặc vuông nếu web bị nuốt ký tự nhé
        wchar_t itemTong[256] = { 0 };
        ListView_GetItemText(hLV, i, 7, itemTong, 256); // Cột 7 là Tổng
        total += _wtof(itemTong);
    }
    wstring txt = L"TỔNG TIỀN: " + to_wstring(total);
    SetWindowTextW(hLabel, txt.c_str());
};

// =====SỰ KIỆN: NHẬP & THÊM VÀO LISTVIEW=====

```

```

// VÀ SỬA NÓ THÀNH NHƯ THỂ NÀY:
if (id == 1029 && IsWindowVisible(hListViewBH)) {
    string sku = GetStrFromHWND(hBHInputs.at(1)); // Lấy ô SKU
    string slStr = GetStrFromHWND(hBHInputs.at(5)); // Lấy ô Số Lượng

    // 1. Validate: Không rỗng
    if (sku.empty() || slStr.empty()) {
        MessageBoxW(hWnd, L"LỖI: Vui lòng nhập SKU và Số lượng!", L"Lỗi Validate",
MB_OK | MB_ICONERROR);
        return 0; // Trả về ngay, KHÔNG cho hiển thị ListView
    }

    // 2. Validate: Tồn tại SKU & Đúng định dạng số
    if (!appCtrl.db.isValidSKU(sku)) {
        MessageBoxW(hWnd, L"LỖI: SKU không tồn tại trong hệ thống!", L"Lỗi Validate",
MB_OK | MB_ICONERROR);
        return 0;
    }

    int sl = 0;
    try { sl = stoi(slStr); }
    catch (...) {
        MessageBoxW(hWnd, L"LỖI: Số lượng phải là chữ số hợp lệ!", L"Lỗi Validate", MB_OK
| MB_ICONERROR);
        return 0;
    }

    Product p = appCtrl.db.getProductInfo(sku);

    // 3. Validate: Số lượng > 0 và <= Tồn kho
    if (sl <= 0 || sl > p.tonKho) {
        MessageBoxW(hWnd, L"LỖI: Số lượng không hợp lệ (Phải > 0 và <= Tồn kho)!", L"Lỗi
Validate", MB_OK | MB_ICONERROR);
        return 0;
    }

    // 4. Nếu HỢP LỆ -> Tự động Fill thông tin lên giao diện (HSD, Khấu trừ Auto)
    SetWindowTextA(hBHInputs.at(3), p.ten.c_str());
    SetWindowTextA(hBHInputs.at(7), to_string(p.gia).c_str());
    SetWindowTextA(hBHInputs.at(9), to_string(p.khauTru).c_str());
    SetWindowTextA(hBHInputs.at(11), p.hsd.c_str());

    double thanhTien = (sl * p.gia) - p.khauTru;

```

```

// 5. Kiểm tra xem SKU đã có trong ListView chưa?
int itemCount = ListView_GetItemCount(hListViewBH);
bool isExist = false;

for (int i = 0; i < itemCount; i++) {
    wchar_t itemSKU[256] = { 0 };
    ListView_GetItemText(hListViewBH, i, 1, itemSKU, 256);
    wstring wsSKU(itemSKU);
    string currentSKU(wsSKU.begin(), wsSKU.end());

    if (currentSKU == sku) {
        // ĐÃ TỒN TẠI -> Cộng dồn Số lượng & Thành tiền
        wchar_t itemSL[256] = { 0 };
        ListView_GetItemText(hListViewBH, i, 3, itemSL, 256);
        int oldSL = _wtoi(itemSL);
        int newSL = oldSL + sl;

        // Validate kiểm tra lại tồn kho một lần nữa khi cộng dồn
        if (newSL > p.tonKho) {
            MessageBoxW(hWnd, L"LỖI: Tổng số lượng cộng dồn vượt quá tồn kho cho phép!",
L"Lỗi", MB_OK | MB_ICONERROR);
            return 0;
        }

        double newTong = (newSL * p.gia) - p.khauTru;

        // Cập nhật lại giao diện dòng đã có
        wstring wsNewSL = to_wstring(newSL);
        wstring wsNewTong = to_wstring(newTong);
        ListView_SetItemText(hListViewBH, i, 3, (LPWSTR)wsNewSL.c_str());
        ListView_SetItemText(hListViewBH, i, 7, (LPWSTR)wsNewTong.c_str());

        isExist = true;
        break;
    }
}

// 6. CHƯA TỒN TẠI -> Tạo dòng mới với STT tịnh tiến
if (!isExist) {
    int newRow = itemCount;
    wstring wsSTT = to_wstring(newRow + 1);
    wstring wSku = wstring(sku.begin(), sku.end());
    wstring wTen = wstring(p.ten.begin(), p.ten.end());
    wstring wSl = to_wstring(sl);

```

```
wstring wGia = to_wstring(p.gia);
wstring wKt = to_wstring(p.khauTru);
wstring wHsd = wstring(p.hsd.begin(), p.hsd.end());
wstring wTong = to_wstring(thanhTien);
```

```
LVITEMW lvi = { 0 };
lvi.mask = LVIF_TEXT;
lvi.iItem = newRow;
lvi.iSubItem = 0;
lvi.pszText = (LPWSTR)wsSTT.c_str();
ListView_InsertItem(hListViewBH, &lvi);
```

```
ListView_SetItemText(hListViewBH, newRow, 1, (LPWSTR)wSku.c_str());
ListView_SetItemText(hListViewBH, newRow, 2, (LPWSTR)wTen.c_str());
ListView_SetItemText(hListViewBH, newRow, 3, (LPWSTR)wSl.c_str());
ListView_SetItemText(hListViewBH, newRow, 4, (LPWSTR)wGia.c_str());
ListView_SetItemText(hListViewBH, newRow, 5, (LPWSTR)wKt.c_str());
ListView_SetItemText(hListViewBH, newRow, 6, (LPWSTR)wHsd.c_str());
ListView_SetItemText(hListViewBH, newRow, 7, (LPWSTR)wTong.c_str());
```

```
}
```

```
// 7. Cập nhật lại Tổng tiền (Gọi hàm Lambda ở trên)
UpdateTongTien(hListViewBH, hDoanhThuLabel);
```

```
// Xóa rỗng 2 ô nhập liệu để người dùng scan mã tiếp theo
SetWindowTextW(hBHInputs.at(1), L"");
SetWindowTextW(hBHInputs.at(5), L"");
```

```
}
```

```
// =====SỰ KIỆN: XÓA DÒNG TRONG LISTVIEW=====
```

```
if (id == 1037 && IsWindowVisible(hListViewBH)) {
    // Lấy dòng đang được chọn (bôi xanh)
    int selIndex = ListView_GetNextItem(hListViewBH, -1, LVNI_SELECTED);
    if (selIndex != -1) {
        // Xóa dòng
        ListView_DeleteItem(hListViewBH, selIndex);

        // Cập nhật lại toàn bộ Cột STT (Để luôn là 1, 2, 3...)
        int count = ListView_GetItemCount(hListViewBH);
        for (int i = 0; i < count; i++) {
            wstring wsSTT = to_wstring(i + 1);
            ListView_SetItemText(hListViewBH, i, 0, (LPWSTR)wsSTT.c_str());
        }
    }
}
```

```

        // Tính lại Tổng tiền
        UpdateTongTien(hListViewBH, hDoanhThuLabel);
    }
    else {
        MessageBoxW(hWnd, L"Vui lòng chọn 1 sản phẩm trong bảng để xóa!", L"Thông báo",
MB_OK | MB_ICONWARNING);
    }
}

// ==SỰ KIỆN: LƯU HÓA ĐƠN & GHI NHẬT KÝ=====
if (id == BTN_BH_LUU_KE_TIEP) {
    int itemCount = ListView_GetItemCount(hListViewBH);

    // Rule 1: ListView rỗng -> Cảnh báo, không cho lưu
    if (itemCount == 0) {
        MessageBoxW(hWnd, L"LỖI: Hóa đơn trống! Vui lòng nhập hàng hóa trước khi lưu.",
L"Lỗi", MB_OK | MB_ICONERROR);
        return 0;
    }

    // Rule 2: Ép dữ liệu tạm từ GUI xuống Controller (InvoiceObj)
    for (int i = 0; i < itemCount; i++) {
        // Chú ý: Tự tay gõ ngoặc vuông và số 256 nếu web bị nuốt mất nhé!
        wchar_t wSku[256] = { 0 }, wTen[256] = { 0 }, wSl[256] = { 0 }, wGia[256] = { 0 },
wKt[256] = { 0 }, wHsd[256] = { 0 }, wTong[256] = { 0 };

        ListView_GetItemText(hListViewBH, i, 1, wSku, 256);
        ListView_GetItemText(hListViewBH, i, 2, wTen, 256);
        ListView_GetItemText(hListViewBH, i, 3, wSl, 256);
        ListView_GetItemText(hListViewBH, i, 4, wGia, 256);
        ListView_GetItemText(hListViewBH, i, 5, wKt, 256);
        ListView_GetItemText(hListViewBH, i, 6, wHsd, 256);
        ListView_GetItemText(hListViewBH, i, 7, wTong, 256);

        // Chốt chuỗi vào 1 biến cố định trước
        wstring wsSku(wSku);
        wstring wsTen(wTen);
        wstring wsHsd(wHsd);

        Item itm;
        itm.sku = string(wsSku.begin(), wsSku.end());
        itm.ten = string(wsTen.begin(), wsTen.end());
        itm.sl = _wtoi(wSl);
        itm.gia = _wtof(wGia);
        itm.khauTru = _wtof(wKt);
    }
}

```



```

itm.hsd = string(wsHsd.begin(), wsHsd.end());
itm.tong = _wtof(wTong);

// Ném Item vào mảng 'list' của Transaction
appCtrl.invoiceObj.addTempItem(itm);
}

// Kích hoạt đa hình lưu file tự động thông qua class cha
appCtrl.switchTab(BTN_BAN_HANG);
if (appCtrl.processSaveTransaction()) {
    MessageBoxW(hWnd, L"Tạo hóa đơn thành công! Đã ghi file hoadon.csv và nhatky.csv",
L"Thành công", MB_OK | MB_ICONINFORMATION);

    // Xóa trắng giao diện, reset bảng, đưa tổng tiền về 0
    ListView_DeleteAllItems(hListViewBH);
    SetWindowTextW(hDoanhThuLabel, L"DOANH THU: 0");
    for (int i = 1; i <= 11; i += 2) SetWindowTextW(hBHInputs.at(i), L"");
}
else {
    MessageBoxW(hWnd, L"Lỗi hệ thống khi lưu file! Vui lòng thử lại.", L"Lỗi", MB_OK |
MB_ICONERROR);
}
}

// =====SỰ KIỆN: NHẬP & THÊM VÀO LISTVIEW NHẬP KHO=====
if (id == 1029 && IsWindowVisible(hListViewNH)) { // Chỉ chạy khi đang ở tab Nhập Kho
    // Lấy dữ liệu từ các ô trắng (Theo quy luật số lẻ: 1, 3, 5, 7, 9, 11)
    string sku = GetStrFromHWND(hNHInputs.at(1));
    string ten = GetStrFromHWND(hNHInputs.at(3));
    string slStr = GetStrFromHWND(hNHInputs.at(5));
    string giaStr = GetStrFromHWND(hNHInputs.at(7));
    string ncc = GetStrFromHWND(hNHInputs.at(9));
    string hsd = GetStrFromHWND(hNHInputs.at(11));

    // 1. Validate: Format không rỗng
    if (sku.empty() || ten.empty() || slStr.empty() || giaStr.empty() || ncc.empty() || hsd.empty()) {
        MessageBoxW(hWnd, L"LỖI: Vui lòng nhập đầy đủ thông tin Nhập kho!", L"Lỗi
Validate", MB_OK | MB_ICONERROR);
        return 0; // Chặn lại, KHÔNG thêm vào List
    }

    // 2. Validate: Đúng kiểu số và giá trị > 0
    int sl = 0;

```

```

double gia = 0.0;
try {
    sl = stoi(slStr);
    gia = stod(giaStr);
    if (sl <= 0 || gia <= 0) {
        MessageBoxW(hWnd, L"LỖI: Số lượng và Giá nhập phải lớn hơn 0!", L"Lỗi Validate",
MB_OK | MB_ICONERROR);
        return 0;
    }
}
catch (...) {
    MessageBoxW(hWnd, L"LỖI: Số lượng và Giá nhập phải là chữ số!", L"Lỗi Validate",
MB_OK | MB_ICONERROR);
    return 0;
}

double thanhTien = sl * gia;
double khaiTru = 0.0; // Nhập kho thường không có khấu trừ trực tiếp trên từng món

// 3. Kiểm tra SKU đã tồn tại trong ListView chưa?
int itemCount = ListView_GetItemCount(hListViewNH);
bool isExist = false;

for (int i = 0; i < itemCount; i++) {
    wchar_t itemSKU[256] = { 0 };
    ListView_GetItemText(hListViewNH, i, 1, itemSKU, 256);
    wstring wsSKU(itemSKU);
    string currentSKU(wsSKU.begin(), wsSKU.end());

    if (currentSKU == sku) {
        // ĐÃ TỒN TẠI -> Cộng dồn Số lượng & Thành tiền
        wchar_t itemSL[256] = { 0 };
        ListView_GetItemText(hListViewNH, i, 3, itemSL, 256);
        int oldSL = _wtoi(itemSL);
        int newSL = oldSL + sl;
        double newTong = newSL * gia;

        // Cập nhật lại giao diện
        wstring wsNewSL = to_wstring(newSL);
        wstring wsNewTong = to_wstring(newTong);
        ListView_SetItemText(hListViewNH, i, 3, (LPWSTR)wsNewSL.c_str());
        ListView_SetItemText(hListViewNH, i, 7, (LPWSTR)wsNewTong.c_str());

        isExist = true;
    }
}

```

```

        break;
    }
}

// 4. CHƯA TỒN TẠI -> Tạo dòng mới với STT tính tiền
if (!isExist) {
    int newRow = itemCount;
    wstring wsSTT = to_wstring(newRow + 1);
    wstring wSku(sku.begin(), sku.end());
    wstring wTen(ten.begin(), ten.end());
    wstring wSl = to_wstring(sl);
    wstring wGia = to_wstring(gia);
    wstring wKt = to_wstring(khauTru);
    wstring wHsd(hsd.begin(), hsd.end());
    wstring wTong = to_wstring(thanhTien);

    LVITEMW lvi = { 0 };
    lvi.mask = LVIF_TEXT;
    lvi.iItem = newRow;
    lvi.iSubItem = 0;
    lvi.pszText = (LPWSTR)wsSTT.c_str();
    ListView_InsertItem(hListViewNH, &lvi);

    ListView_SetItemText(hListViewNH, newRow, 1, (LPWSTR)wSku.c_str());
    ListView_SetItemText(hListViewNH, newRow, 2, (LPWSTR)wTen.c_str());
    ListView_SetItemText(hListViewNH, newRow, 3, (LPWSTR)wSl.c_str());
    ListView_SetItemText(hListViewNH, newRow, 4, (LPWSTR)wGia.c_str());
    ListView_SetItemText(hListViewNH, newRow, 5, (LPWSTR)wKt.c_str());
    ListView_SetItemText(hListViewNH, newRow, 6, (LPWSTR)wHsd.c_str());
    ListView_SetItemText(hListViewNH, newRow, 7, (LPWSTR)wTong.c_str());
}

// Xóa rỗng các ô nhập để chuẩn bị quét mã tiếp theo (Giữ lại Nhà CC)
SetWindowTextW(hNHInputs.at(1), L "");
SetWindowTextW(hNHInputs.at(3), L "");
SetWindowTextW(hNHInputs.at(5), L "");
SetWindowTextW(hNHInputs.at(7), L "");
SetWindowTextW(hNHInputs.at(11), L "");
}

// =====SỰ KIỆN: LƯU NHẬP KHO & GHI NHẬT KÝ=====
if (id == BTN_NH_LUU) {
    int itemCount = ListView_GetItemCount(hListViewNH);

```

```

// Rule 1: ListView rỗng -> Cảnh báo, không cho lưu
if (itemCount == 0) {
    MessageBoxW(hWnd, L"LỖI: Danh sách trống! Vui lòng nhập hàng trước khi lưu.",
L"Lỗi", MB_OK | MB_ICONERROR);
    return 0;
}

string ncc = GetStrFromHWND(hNHInputs.at(9));

// Tự động tạo Ngày Nhập bằng thời gian thực của máy tính
time_t now = time(0);
tm ltm;
localtime_s(&ltm, &now);
char dateBuffer[80]; // (Tự gõ ngoặc vuông nếu web nuốt ký tự)
strftime(dateBuffer, sizeof(dateBuffer), "%Y-%m-%d", &ltm);
string ngayNhap = string(dateBuffer);

// Rule 2: Đầy chi tiết phiếu nhập (NCC, Ngày) vào Controller trước [1]
appCtrl.importObj.setImportDetails(ncc, ngayNhap);

// Rule 3: Ép dữ liệu tạm từ ListView xuống biến 'list' trong RAM
for (int i = 0; i < itemCount; i++) {
    // (Tự gõ ngoặc vuông và số 256 nếu web nuốt ký tự)
    wchar_t wSku[256] = { 0 }, wTen[256] = { 0 }, wSl[256] = { 0 }, wGia[256] = { 0 },
wKt[256] = { 0 }, wHsd[256] = { 0 }, wTong[256] = { 0 };

    ListView_GetItemText(hListViewNH, i, 1, wSku, 256);
    ListView_GetItemText(hListViewNH, i, 2, wTen, 256);
    ListView_GetItemText(hListViewNH, i, 3, wSl, 256);
    ListView_GetItemText(hListViewNH, i, 4, wGia, 256);
    ListView_GetItemText(hListViewNH, i, 5, wKt, 256);
    ListView_GetItemText(hListViewNH, i, 6, wHsd, 256);
    ListView_GetItemText(hListViewNH, i, 7, wTong, 256);

    // Chốt chuỗi an toàn (Chống lỗi Different Containers)
    wstring wsSku(wSku);
    wstring wsTen(wTen);
    wstring wsHsd(wHsd);

    Item itm;
    itm.sku = string(wsSku.begin(), wsSku.end());
    itm.ten = string(wsTen.begin(), wsTen.end());
    itm.sl = _wtoi(wSl);
    itm.gia = _wtof(wGia);

```

```

itm.khauTru = _wtof(wKt);
itm.hsd = string(wsHsd.begin(), wsHsd.end());
itm.tong = _wtof(wTong);

// Ném Item vào mảng 'list' của Transaction cha
appCtrl.importObj.addTempItem(itm);
}

// Rule 4: Kích hoạt đa hình lưu file tự động thông qua class cha
appCtrl.switchTab(BTN_NHAP_KHO); // [2, 3]
if (appCtrl.processSaveTransaction()) { // Gọi logic Validate -> Ghi File -> Ghi Nhật Ký
    MessageBoxW(hWnd, L"Lưu phiếu nhập kho thành công! Đã ghi file nhapkho.csv và
nhatky.csv", L"Thành công", MB_OK | MB_ICONINFORMATION);

    // Rule 5: Reset giao diện
    ListView_DeleteAllItems(hListViewNH);
    for (int i = 1; i <= 11; i += 2) SetWindowTextW(hNHInputs.at(i), L "");
}
else {
    MessageBoxW(hWnd, L"Lỗi hệ thống khi lưu file! Vui lòng thử lại.", L"Lỗi", MB_OK |
MB_ICONERROR);
}
}

// =====SỰ KIỆN: NHẬP & THÊM VÀO LISTVIEW XUẤT KHO=====
if (id == 1029 && IsWindowVisible(hListViewXH)) { // Chỉ chạy khi ở tab Xuất Kho
    // Lấy dữ liệu từ các ô trống (Theo quy luật số lẻ)
    string sku = GetStrFromHWND(hXHInputs.at(1));
    string slStr = GetStrFromHWND(hXHInputs.at(5));

    // 1. Validate: Format không rỗng
    if (sku.empty() || slStr.empty()) {
        MessageBoxW(hWnd, L"LỖI: Vui lòng nhập SKU và Số lượng!", L"Lỗi Validate",
MB_OK | MB_ICONERROR);
        return 0; // KHÔNG thêm vào List
    }

    // 2. Validate: SKU hợp lệ
    if (!appCtrl.db.isValidSKU(sku)) {
        MessageBoxW(hWnd, L"LỖI: SKU không tồn tại trong kho!", L"Lỗi Validate", MB_OK |
MB_ICONERROR);
        return 0;
    }
}

```

```

// 3. Validate: Số lượng > 0
int sl = 0;
try {
    sl = stoi(slStr);
    if (sl <= 0) {
        MessageBoxW(hWnd, L"LỖI: Số lượng xuất phải lớn hơn 0!", L"Lỗi Validate",
MB_OK | MB_ICONERROR);
        return 0;
    }
}
catch (...) {
    MessageBoxW(hWnd, L"LỖI: Số lượng phải là chữ số!", L"Lỗi Validate", MB_OK |
MB_ICONERROR);
    return 0;
}

Product p = appCtrl.db.getProductInfo(sku);

// 4. Validate: Số lượng xuất <= Số lượng Tồn kho
if (sl > p.tonKho) {
    wstring errMsg = L"LỖI: Số lượng xuất (" + to_wstring(sl) + L") vượt quá Tồn kho hiện tại
(" + to_wstring(p.tonKho) + L)!";
    MessageBoxW(hWnd, errMsg.c_str(), L"Lỗi Validate", MB_OK | MB_ICONERROR);
    return 0;
}

// Nếu hợp lệ -> Tự động điền Tên và Giá từ RAM lên giao diện cho đẹp
SetWindowTextA(hXHInputs.at(3), p.ten.c_str());
SetWindowTextA(hXHInputs.at(7), to_string(p.gia).c_str());

double thanhTien = sl * p.gia;
double khaiTru = 0.0;

// 5. Kiểm tra SKU đã tồn tại trong ListView chưa?
int itemCount = ListView_GetItemCount(hListViewXH);
bool isExist = false;

for (int i = 0; i < itemCount; i++) {
    wchar_t itemSKU[256] = { 0 }; // (Tự gỡ ngoặc vuông nếu web bị nuốt)
    ListView_GetItemText(hListViewXH, i, 1, itemSKU, 256);
    wstring wsSKU(itemSKU);
    string currentSKU(wsSKU.begin(), wsSKU.end());

    if (currentSKU == sku) {

```

```

// ĐÃ TỒN TẠI -> Cộng dồn
wchar_t itemSL[256] = { 0 };
ListView_GetItemText(hListViewXH, i, 3, itemSL, 256);
int oldSL = _wtoi(itemSL);
int newSL = oldSL + sl;

// Phải Validate lại tổng số lượng gộp xem có lỗi tồn kho không
if (newSL > p.tonKho) {
    MessageBoxW(hWnd, L"LỖI: Tổng số lượng cộng dồn vượt quá Tồn kho!", L"Lỗi Validate", MB_OK | MB_ICONERROR);
    return 0;
}

double newTong = newSL * p.gia;

// Cập nhật lại giao diện
wstring wsNewSL = to_wstring(newSL);
wstring wsNewTong = to_wstring(newTong);
ListView_SetItemText(hListViewXH, i, 3, (LPWSTR)wsNewSL.c_str());
ListView_SetItemText(hListViewXH, i, 7, (LPWSTR)wsNewTong.c_str());

isExist = true;
break;
}
}

// 6. CHƯA TỒN TẠI -> Thêm dòng mới
if (!isExist) {
    int newRow = itemCount;
    wstring wsSTT = to_wstring(newRow + 1);
    wstring wSku(sku.begin(), sku.end());
    wstring wTen(p.ten.begin(), p.ten.end());
    wstring wSl = to_wstring(sl);
    wstring wGia = to_wstring(p.gia);
    wstring wKt = to_wstring(khauTru);
    wstring wHsd(p.hsd.begin(), p.hsd.end());
    wstring wTong = to_wstring(thanhTien);

    LVITEMW lvi = { 0 };
    lvi.mask = LVIF_TEXT;
    lvi.iItem = newRow;
    lvi.iSubItem = 0;
    lvi.pszText = (LPWSTR)wsSTT.c_str();
    ListView_InsertItem(hListViewXH, &lvi);

```

```

        ListView_SetItemText(hListViewXH, newRow, 1, (LPWSTR)wSku.c_str());
        ListView_SetItemText(hListViewXH, newRow, 2, (LPWSTR)wTen.c_str());
        ListView_SetItemText(hListViewXH, newRow, 3, (LPWSTR)wSl.c_str());
        ListView_SetItemText(hListViewXH, newRow, 4, (LPWSTR)wGia.c_str());
        ListView_SetItemText(hListViewXH, newRow, 5, (LPWSTR)wKt.c_str());
        ListView_SetItemText(hListViewXH, newRow, 6, (LPWSTR)wHsd.c_str());
        ListView_SetItemText(hListViewXH, newRow, 7, (LPWSTR)wTong.c_str());
    }

    // Xóa rỗng các ô nhập để chuẩn bị quét mã tiếp theo
    SetWindowTextW(hXHInputs.at(1), L"");
    SetWindowTextW(hXHInputs.at(5), L"");
}

// =====SỰ KIỆN: LƯU XUẤT KHO & GHI NHẬT KÝ=====
if (id == BTN_XH_LUU) {
    int itemCount = ListView_GetItemCount(hListViewXH);

    if (itemCount == 0) {
        MessageBoxW(hWnd, L"LỖI: Danh sách trống! Vui lòng nhập hàng trước khi xuất.",
            L"Lỗi", MB_OK | MB_ICONERROR);
        return 0;
    }
    string lyDo = GetStrFromHWND(hXHInputs.at(9));
    if (lyDo.empty() ||
        (lyDo != "Muon hang" && lyDo != "MUON HANG" &&
         lyDo != "Tieu huy" && lyDo != "TIEU HUY" &&
         lyDo != "Thien nguyen" && lyDo != "THIEN NGUYEN")) {
        MessageBoxW(hWnd, L"LỖI: Ly do phai nhap dung 1 trong 3 loai:\nMUON
HANG\nTIEU HUY\nTHIEN NGUYEN", L"Lỗi Validate", MB_OK | MB_ICONERROR);
        return 0;
    }

    // Tự động tạo Ngày Xuất bằng thời gian thực
    time_t now = time(0);
    tm ltm;
    localtime_s(&ltm, &now);
    char dateBuffer[80];
    strftime(dateBuffer, sizeof(dateBuffer), "%Y-%m-%d", &ltm);
    string ngayXuat = string(dateBuffer);

    // Truyền Lý Do và Ngày Xuất xuống class Export
    appCtrl.exportObj.setExportDetails(lyDo, ngayXuat);
}

```



```

// Ép dữ liệu tạm từ ListView xuống RAM
for (int i = 0; i < itemCount; i++) {
    wchar_t wSku[256] = { 0 }, wTen[256] = { 0 }, wSl[256] = { 0 }, wGia[256] = { 0 },
wKt[256] = { 0 }, wHsd[256] = { 0 }, wTong[256] = { 0 };

    ListView_GetItemText(hListViewXH, i, 1, wSku, 256);
    ListView_GetItemText(hListViewXH, i, 2, wTen, 256);
    ListView_GetItemText(hListViewXH, i, 3, wSl, 256);
    ListView_GetItemText(hListViewXH, i, 4, wGia, 256);
    ListView_GetItemText(hListViewXH, i, 5, wKt, 256);
    ListView_GetItemText(hListViewXH, i, 6, wHsd, 256);
    ListView_GetItemText(hListViewXH, i, 7, wTong, 256);

    wstring wsSku(wSku);
    wstring wsTen(wTen);
    wstring wsHsd(wHsd);

    Item itm;
    itm.sku = string(wsSku.begin(), wsSku.end());
    itm.ten = string(wsTen.begin(), wsTen.end());
    itm.sl = _wtoi(wSl);
    itm.gia = _wtof(wGia);
    itm.khauTru = _wtof(wKt);
    itm.hsd = string(wsHsd.begin(), wsHsd.end());
    itm.tong = _wtof(wTong);

    appCtrl.exportObj.addTempItem(itm);
}

// Kích hoạt Đa hình để ghi file
appCtrl.switchTab(BTN_XUAT_KHO);
if (appCtrl.processSaveTransaction()) {
    MessageBoxW(hWnd, L"Lưu phiếu xuất kho thành công! Đã ghi file xuấtkho.csv và
nhatky.csv", L"Thành công", MB_OK | MB_ICONINFORMATION);

    ListView_DeleteAllItems(hListViewXH);
    for (int i = 1; i <= 11; i += 2) SetWindowTextW(hXHInputs.at(i), L"");
}
else {
    MessageBoxW(hWnd, L"Lỗi hệ thống khi lưu file! Vui lòng thử lại.", L"Lỗi", MB_OK |
MB_ICONERROR);
}
}

```

```

// =====SỰ KIỆN: LƯU SỔ GHI CHÚ=====
if (id == BTN_GC_LUU) {
    // 1. Lấy dữ liệu từ các ô nhập liệu (Quy luật vị trí số lẻ)
    string caLam = GetStrFromHWND(hGCInputs.at(1));
    string nguoiLap = GetStrFromHWND(hGCInputs.at(3));
    string noiDung = GetStrFromHWND(hGCInputs.at(5));
    string phanLoai = GetStrFromHWND(hGCInputs.at(7));
    string thoiGian = GetStrFromHWND(hGCInputs.at(9));
    string maLK = GetStrFromHWND(hGCInputs.at(11));

    // 2. Validate: Nội dung, Ca làm, Người lập không được rỗng [1, 2]
    if (noiDung.empty() || caLam.empty() || nguoiLap.empty() || phanLoai.empty()) {
        MessageBoxW(hWnd, L"LỖI: Vui lòng nhập đầy đủ Ca làm, Người lập, Nội dung và Phân loại!", L"Lỗi Validate", MB_OK | MB_ICONERROR);
        return 0; // KHÔNG lưu, KHÔNG hiển thị ListView [2, 3]
    }

    // 3. Validate: Người lập hợp lệ (Có tồn tại trong kho RAM) [1]
    if (!appCtrl.db.isValidUser(nguoiLap)) {
        MessageBoxW(hWnd, L"LỖI: Mã Người Lập (QL/NV) không tồn tại trong hệ thống!", L"Lỗi Validate", MB_OK | MB_ICONERROR);
        return 0;
    }

    // 4. Validate: Phân loại bắt buộc thuộc danh sách (Chấp nhận cả in hoa từ bàn phím POS)
    if (phanLoai != "Thu tuc" && phanLoai != "THU TUC" &&
        phanLoai != "Giay to" && phanLoai != "GIAY TO" &&
        phanLoai != "Y te" && phanLoai != "Y TE" &&
        phanLoai != "Chay no" && phanLoai != "CHAY NO" &&
        phanLoai != "Thue" && phanLoai != "THUE") {
        MessageBoxW(hWnd, L"LỖI: Phân loại phải là 1 trong 5 loại:\nTHU TUC, GIAY TO, Y TE, CHAY NO, THUE", L"Lỗi Validate", MB_OK | MB_ICONERROR);
        return 0;
    }

    // 5. Tự động khởi tạo Thời gian thực & Mã Ghi Chú (GC + Timestamp)
    time_t now = time(0);
    tm ltm;
    localtime_s(&ltm, &now);

    char tBuf[80]; // (Tự gỡ ngoặc vuông nếu web nuốt ký tự)
    strftime(tBuf, sizeof(tBuf), "%Y-%m-%d %H:%M:%S", &ltm);
    string curTime = string(tBuf);

```

```

char mBuf[80];
strftime(mBuf, sizeof(mBuf), "GC%Y%m%d%H%M%S", &tm);
string maGC = string(mBuf);

// Nếu ô Thời gian người dùng không gõ gì, lấy thời gian thực tự động cho an toàn
if (thoiGian.empty()) thoiGian = curTime;

// 6. GHI VÀO FILE SỔ GHI CHÚ CHI TIẾT (soghichu.csv) [2]
ofstream fileSo("soghichu.csv", ios::app);
if (fileSo.is_open()) {
    fileSo << curTime << ",GHI_CHU," << caLam << "," << nguoiLap << ","
        << phanLoai << "," << maLK << "," << noiDung << "\n";
    fileSo.close();
}
else {
    MessageBoxW(hWnd, L"LỖI: Không thể mở file soghichu.csv để ghi!", L"Lỗi Hệ Thống",
MB_OK | MB_ICONERROR);
    return 0;
}

// 7. GHI VÀO FILE NHẬT KÝ TỔNG (nhatky.csv) [2, 3]
ofstream fileNk("nhatky.csv", ios::app);
if (fileNk.is_open()) {
    fileNk << curTime << ",GHI_CHU," << maGC << "\n";
    fileNk.close();
}

// 8. CẬP NHẬT HIỂN THỊ LÊN LISTVIEW (Real-time) [3]
int newRow = ListView_GetItemCount(hListViewGC);
wstring wsSTT = to_wstring(newRow + 1);
wstring wCa(caLam.begin(), caLam.end());
wstring wNguoi(nguoiLap.begin(), nguoiLap.end());
wstring wLoai(phanLoai.begin(), phanLoai.end());
wstring wTime(curTime.begin(), curTime.end());
wstring wND(noiDung.begin(), noiDung.end());
wstring wMaLK(maLK.begin(), maLK.end());

LVITEMW lvi = { 0 };
lvi.mask = LVIF_TEXT;
lvi.iItem = newRow;
lvi.iSubItem = 0;
lvi.pszText = (LPWSTR)wsSTT.c_str();
ListView_InsertItem(hListViewGC, &lvi);

```

```

// Map cột đúng theo format: | STT | Ca | Người lập | Phân loại | Time | Nội dung | Mã liên
quan | [2]
ListView_SetItemText(hListViewGC, newRow, 1, (LPWSTR)wCa.c_str());
ListView_SetItemText(hListViewGC, newRow, 2, (LPWSTR)wNguoi.c_str());
ListView_SetItemText(hListViewGC, newRow, 3, (LPWSTR)wLoai.c_str());
ListView_SetItemText(hListViewGC, newRow, 4, (LPWSTR)wTime.c_str());
ListView_SetItemText(hListViewGC, newRow, 5, (LPWSTR)wND.c_str());
ListView_SetItemText(hListViewGC, newRow, 6, (LPWSTR)wMaLK.c_str());

// 9. Reset trắng giao diện để chuẩn bị nhập ghi chú tiếp theo
for (int i = 1; i <= 11; i += 2) {
    // Có thể bỏ qua không xóa Ca làm và Người lập để User đỡ phải gõ lại nhiều lần
    if (i != 1 && i != 3) SetWindowTextW(hGCInputs.at(i), L "");
}
MessageBoxW(hWnd, L"Đã lưu Ghi chú thành công và ghi vào sổ nhật ký!", L"Thành công",
MB_OK | MB_ICONINFORMATION);
}

if (HIWORD(wp) == EN_SETFOCUS) hActiveEdit = (HWND)lp;
if (id >= BTN_KEY_BASE && id < BTN_KEY_BASE + 42) {
    const wchar_t* v = layout[id - BTN_KEY_BASE];
    if (wcscmp(v, L"<-") == 0) SendMessage(hActiveEdit, WM_CHAR, 0x08, 0);
    else if (wcscmp(v, L"Enter") == 0) {}
    else SendMessage(hActiveEdit, EM_REPLACESEL, TRUE, (LPARAM)v);
}
else if (id == BTN_KEY_BASE + 99) SendMessage(hActiveEdit, EM_REPLACESEL, TRUE,
(LPARAM)L" ");
break;
}
case WM_PAINT: {
    PAINTSTRUCT ps;
    HDC hdc = BeginPaint(hWnd, &ps);
    if (sysManager.getStatus()) {
        Graphics g(hdc);
        DrawLargeLogo(g, 600, 250, 50);
    }
    EndPaint(hWnd, &ps);
    break;
}
case WM_DESTROY: PostQuitMessage(0); break;
default: return DefWindowProc(hWnd, msg, wp, lp);
} return 0;
}

```

```

int WINAPI WinMain(HINSTANCE h, HINSTANCE hP, LPSTR lp, int n) {
    GdiplusStartupInput gsi;
    ULONG_PTR token;
    GdiplusStartup(&token, &gsi, NULL);

    WNDCLASS wc = { 0 };
    wc.lpfnWndProc = WndProc;
    wc.hInstance = h;
    wc.hbrBackground = (HBRUSH)(COLOR_BTNFACE + 1);
    wc.lpszClassName = L"GO_POS";
    RegisterClass(&wc);

    CreateWindow(L"GO_POS", L"GO POS System Pro", WS_OVERLAPPEDWINDOW |
WS_VISIBLE, 100, 100, 1100, 800, NULL, NULL, h, NULL);

    MSG msg;
    while (GetMessage(&msg, NULL, 0, 0)) {
        TranslateMessage(&msg);
        DispatchMessage(&msg);
    }
    GdiplusShutdown(token);
    return 0;
}

```

Bảng 3.2. Chương trình DataManager.h

```

#pragma once
#include <iostream>
#include <vector>
#include <string>

using namespace std;

// =====ĐỊNH NGHĨA CẤU TRÚC DỮ LIỆU=====
struct User {
    string ma;
    string role;
};

struct Product {
    string sku;
    string ten;
}

```

```

double gia = 0.0;
double khauTru = 0.0;
int tonKho = 0;
string hsd;
};

// LỚP QUẢN LÝ DỮ LIỆU TỪ FILE CSV/TXT (RAM CỦA HỆ THỐNG)=====
class DataManager {
private:
    vector<Product> productList;
    vector<User> userList;

    // [CODE MỚI THÊM]: Biến lưu trữ tổng doanh thu bán hàng trong ca
    double tongDoanhThu = 0.0;

public:
    // =====XỬ LÝ DỮ LIỆU HÀNG HÓA=====
    void addProduct(const Product& p) {
        productList.push_back(p);
    }

    bool isValidSKU(string sku) {
        for (const auto& p : productList) {
            if (p.sku == sku) return true;
        }
        return false;
    }

    int getTonKho(string sku) {
        for (const auto& p : productList) {
            if (p.sku == sku) return p.tonKho;
        }
        return 0;
    }

    Product getProductInfo(string sku) {
        for (const auto& p : productList) {
            if (p.sku == sku) return p;
        }
        return Product();
    }

    void updateStock(string sku, int qtyChange) {

```

```

    for (auto& p : productList) {
        if (p.sku == sku) {
            p.tonKho += qtyChange;
            return;
        }
    }
}

// =====XỬ LÝ DỮ LIỆU NHÂN SỰ=====
void addUser(const User& u) {
    userList.push_back(u);
}

bool isValidUser(string ma) {
    for (const auto& u : userList) {
        if (u.ma == ma) return true;
    }
    return false;
}

// =====QUẢN LÝ DOANH THU (CẬP NHẬT CHO GIAO DIỆN)=====
// Hàm dùng để Controller gọi khi lưu hóa đơn thành công
void addDoanhThu(double tien) {
    tongDoanhThu += tien;
}

// Hàm dùng để View (GUI) lấy số tiền hiển thị lên nhãn
double getDoanhThu() {
    return tongDoanhThu;
}
};

```

Bảng 3.3. Chương trình UpoadManager.h

```

#pragma once
#include <iostream>
#include <fstream> // Đọc file
#include <sstream> // Xử lý chuỗi theo luồng (để cắt chuỗi)
#include <string>
#include <vector>
#include <windows.h> // Sử dụng để hiển thị MessageBox thông báo lỗi

#include "11b_DataManager.h"

```

```

using namespace std;

class UploadManager {
private:
    // Con trỏ kết nối đến kho dữ liệu trung tâm để lưu sau khi nạp thành công
    DataManager* db = nullptr;

public:
    // Hàm thiết lập kết nối với DataManager
    void setDataManager(DataManager* manager) { this->db = manager; }

    /**
     * Hàm nạp dữ liệu SẢN PHẨM (8 cột)
     * Đọc file từ đường dẫn filePath và đẩy vào danh sách Product của DataManager
     */
    bool uploadProductData(wstring filePath) {
        if (db == nullptr) return false;

        ifstream file(filePath); // Mở file để đọc
        if (!file.is_open()) return false;

        string line;
        int count = 0; // Đếm số sản phẩm nạp thành công
        while (getline(file, line)) {
            if (line.empty()) continue; // Bỏ qua dòng trống

            // Chuẩn hóa: Nếu file dùng dấu ';' để phân cách thì đổi hết thành dấu ','
            for (char& c : line) if (c == ';') c = ',';

            stringstream ss(line);
            string item;
            vector<string> tokens; // Mảng chứa các từ đã cắt từ dòng

            // Cắt dòng dựa trên dấu phẩy (CSV)
            while (getline(ss, item, ',')) {
                // Xử lý Trim: Xóa khoảng trắng thừa ở đầu và cuối mỗi từ
                size_t start = item.find_first_not_of(" \t\r\n");
                if (start == string::npos) tokens.push_back("");
                else {
                    size_t end = item.find_last_not_of(" \t\r\n");
                    tokens.push_back(item.substr(start, end - start + 1));
                }
            }
        }
    }
}

```



```

// [NGHIỆP VỤ]: Kiểm tra dòng phải có ít nhất 8 cột dữ liệu
if (tokens.size() >= 8) {
    Product p;
    p.sku = tokens.at(1);    // Cột 2: Mã SKU
    p.ten = tokens.at(2);    // Cột 3: Tên sản phẩm
    p.khauTru = 0;          // Mặc định nạp vào chưa có khấu trừ
    p.hsd = tokens.at(6);    // Cột 7: Hạn sử dụng

    try {
        // Chuyển đổi chuỗi sang số (stod cho double, stoi cho int)
        p.gia = stod(tokens.at(3)); // Cột 4: Giá bán
        p.tonKho = stoi(tokens.at(4)); // Cột 5: Số lượng tồn kho

        db->addProduct(p); // Đưa sản phẩm vào RAM (DataManager)
        count++;
    }
    catch (...) {
        // Nếu dữ liệu cột giá/số lượng không phải là số -> Bỏ qua dòng này
        continue;
    }
}

file.close();

// Nếu không nạp được sản phẩm nào (file sai định dạng) -> Báo lỗi
if (count == 0) {
    MessageBoxW(NULL, L"LỖI SỐ 3: Mở file thành công nhưng không nạp được dữ liệu!",
L"Máy Quét Lỗi", MB_OK | MB_ICONERROR);
    return false;
}

return true;
}

/**
 * Hàm nạp dữ liệu NHÂN SỰ (2 cột: Mã, Chức vụ)
 * Thường dùng để đọc file NhanSu.txt
 */
bool uploadUserData(wstring filePath) {
    if (db == nullptr) return false;

    ifstream file(filePath);

```

```

if (!file.is_open()) return false;

string line;
int count = 0;
while (getline(file, line)) {
    if (line.empty()) continue;

    for (char& c : line) if (c == ';') c = ',';

    stringstream ss(line);
    string item;
    vector<string> tokens;

    while (getline(ss, item, ',')) {
        size_t start = item.find_first_not_of(" \t\r\n");
        if (start == string::npos) tokens.push_back("");
        else {
            size_t end = item.find_last_not_of(" \t\r\n");
            tokens.push_back(item.substr(start, end - start + 1));
        }
    }

    // File Nhân sự yêu cầu tối thiểu 2 cột: [Mã, Vai trò]
    if (tokens.size() >= 2) {
        User u;
        u.ma = tokens.at(0);

        // --- XỬ LÝ BOM (Byte Order Mark) ---
        // Một số file UTF-8 có 3 ký tự lạ ở đầu file, đoạn code này giúp xóa bỏ chúng
        // để so sánh mã nhân viên (như ADMIN) chính xác hơn.
        if (u.ma.size() >= 3 &&
            (unsigned char)u.ma.at(0) == 0xEF &&
            (unsigned char)u.ma.at(1) == 0xBB &&
            (unsigned char)u.ma.at(2) == 0xBF) {
            u.ma = u.ma.substr(3);
        }

        u.role = tokens.at(1);
        db->addUser(u); // Đưa nhân sự vào RAM
        count++;
    }
}
file.close();
return count > 0;

```

```
}
};
```

Bảng 3.4. Chương trình Controller.h

```
#pragma once
#include <iostream>
#include <vector>
#include <string>

// Nhung tất cả các Header cần thiết để Controller có thể quản lý
#include "11b_DataManager.h" // Kho dữ liệu
#include "0b_UploadManager.h" // Xử lý file đầu vào
#include "1b_Transaction.h" // Lớp cha của các giao dịch
#include "2b_Invoice.h" // Lớp Bán hàng
#include "3b_Import.h" // Lớp Nhập kho
#include "4b_Export.h" // Lớp Xuất kho
#include "6b_NoteManager.h" // Lớp Ghi chú
#include "7b_ShiftManager.h" // Lớp Quản lý ca
#include "8a_DataExporter.h" // Lớp Xuất báo cáo

using namespace std;

/**
 * Lớp Controller: Trung tâm điều phối của ứng dụng
 */
class Controller {
public:
    // 1. KHO DỮ LIỆU TRUNG TÂM (Lưu trên RAM)
    // Mọi module khác đều dùng chung con trỏ từ đối tượng db này
    DataManager db;

    // 2. CÁC MODULE NGHIỆP VỤ (Khởi tạo tĩnh)
    Invoice invoiceObj; // Xử lý Bán hàng
    Import importObj; // Xử lý Nhập kho
    Export exportObj; // Xử lý Xuất kho
    NoteManager note; // Xử lý Ghi chú
    ShiftManager shift; // Xử lý Mở/Kết ca
    DataExporter exporter; // Chức năng xuất báo cáo
    UploadManager uploadObj; // Xử lý đọc file danh mục hàng hóa/nhân viên

    // 3. CON TRỎ ĐA HÌNH (Polymorphism)
```

```
// Đây là "Cánh tay robot" linh hoạt: Lúc thì trở vào Bán hàng, lúc trở vào Nhập kho
// Giúp GUI không phải viết quá nhiều hàm saveBanHang(), saveNhapKho()...
Transaction* activeTransaction = nullptr;
```

```
/**
```

```
* 4. CONSTRUCTOR (Hàm khởi tạo)
```

```
* Thực hiện "Cắm dây" (Dependency Injection): Truyền địa chỉ của 'db'
```

```
* vào các module để tất cả dùng chung một nguồn dữ liệu duy nhất.
```

```
*/
```

```
Controller() {
```

```
    uploadObj.setDataManager(&db);
```

```
    invoiceObj.setDataManager(&db);
```

```
    importObj.setDataManager(&db);
```

```
    exportObj.setDataManager(&db);
```

```
    shift.setDataManager(&db);
```

```
    note.setDataManager(&db);
```

```
// Tạo sẵn một User Admin để có thể đăng nhập/kiểm tra hệ thống ngay lập tức
```

```
User testUser;
```

```
testUser.ma = "ADMIN";
```

```
testUser.role = "QuanLy";
```

```
db.addUser(testUser);
```

```
}
```

```
/**
```

```
* 5. HÀM CHUYỂN TAB
```

```
* Khi người dùng bấm nút trên GUI, hàm này sẽ điều hướng con trỏ activeTransaction
```

```
*/
```

```
void switchTab(int tabID) {
```

```
    switch (tabID) {
```

```
        case 54: // ID nút BÁN HÀNG
```

```
            activeTransaction = &invoiceObj;
```

```
            break;
```

```
        case 55: // ID nút NHẬP KHO
```

```
            activeTransaction = &importObj;
```

```
            break;
```

```
        case 56: // ID nút XUẤT KHO
```

```
            activeTransaction = &exportObj;
```

```
            break;
```

```
        default:
```

```
            // Các tab như Ghi chú, Mở ca không kế thừa Transaction thì set Null
```

```
            activeTransaction = nullptr;
```

```
            break;
```

```
    }
```

```

}

/**
 * 6. HÀM XỬ LÝ LƯU ĐA HÌNH (Cực kỳ quan trọng)
 * GUI chỉ cần gọi hàm này khi người dùng bấm nút "LƯU" hoặc "THANH TOÁN"
 */
bool processSaveTransaction() {
    // Kiểm tra xem có đang ở Tab giao dịch (Bán/Nhập/Xuất) không
    if (activeTransaction == nullptr) return false;

    // BƯỚC 1: Kiểm tra danh sách hàng hóa có trống không (Validate)
    if (!activeTransaction->validateBeforeSave()) {
        return false; // Trả về false để GUI báo lỗi "Danh sách trống"
    }

    // BƯỚC 2: Gọi các hàm thực thi (Đã được định nghĩa đa hình ở các lớp con)
    activeTransaction->save(); // Lưu file chi tiết phiếu (CSV/TXT)
    activeTransaction->writeLog(); // Ghi lại lịch sử vào nhatky.csv
    activeTransaction->reset(); // Dọn dẹp bộ nhớ tạm để chuẩn bị cho đơn hàng tiếp theo

    return true; // Thành công -> GUI xóa trắng bảng hiển thị và báo thành công
}
};

```

Bảng 3.5. Chương trình SystemManager.h

```

#pragma once
#include <iostream>

using namespace std;

// 1. Forward Declaration (Khai báo tiền thân):
// Thông báo cho trình biên dịch rằng "Controller" là một class sẽ xuất hiện sau.
// Điều này giúp tránh lỗi vòng lặp include khi SystemManager cần dùng Controller
// và ngược lại.
class Controller;

class SystemManager {
private:
    bool isPowerOn = false; // Biến lưu trữ trạng thái nguồn (mặc định là Tắt)

public:

```

```
// Hàm kiểm tra trạng thái nguồn hiện tại (Đang bật hay Đang tắt)
bool getStatus() const { return isPowerOn; }
```

```
/**
```

```
 * Khai báo hàm chuyển đổi trạng thái nguồn.
```

```
 * Lưu ý: Chỉ khai báo tên hàm ở đây, không viết nội dung (body) vì tại thời điểm này
```

```
 * trình biên dịch vẫn chưa biết cấu trúc chi tiết bên trong của Controller.
```

```
 */
```

```
bool togglePower(Controller* appCtrl);
```

```
};
```

```
// 2. Nạp cấu trúc chi tiết của Controller:
```

```
// Sau khi đã định nghĩa xong lớp SystemManager, ta mới include Controller.
```

```
#include "10b_Controller.h"
```

```
/**
```

```
 * Hàm Toggle Power (Bật/Tắt nguồn):
```

```
 * Sử dụng từ khóa 'inline' để tối ưu hóa hiệu suất vì hàm này thường được gọi từ GUI.
```

```
 */
```

```
inline bool SystemManager::togglePower(Controller* appCtrl) {
```

```
    if (!isPowerOn) {
```

```
        // --- TRƯỜNG HỢP 1: HỆ THỐNG ĐANG TẮT -> BẬT LÊN ---
```

```
        isPowerOn = true;
```

```
        // Trả về true để GUI biết và chuyển sang giao diện chính/màn hình chờ
```

```
        return true;
```

```
    }
```

```
    else {
```

```
        // --- TRƯỜNG HỢP 2: HỆ THỐNG ĐANG BẬT -> TẮT ĐI ---
```

```
        // KIỂM TRA AN TOÀN DỮ LIỆU (Nghệ vụ quan trọng):
```

```
        // Nếu Controller đang có một giao dịch dở dang (Bán hàng/Nhập/Xuất chưa bấm Lưu)
```

```
        if (appCtrl != nullptr && appCtrl->activeTransaction != nullptr) {
```

```
            // Nếu danh sách hàng hóa trong giao dịch tạm thời không rỗng
```

```
            if (!appCtrl->activeTransaction->isEmpty()) {
```

```
                // Tự động thực hiện lệnh lưu (Save) trước khi ngắt nguồn
```

```
                // Điều này đảm bảo nhân viên quên bấm Lưu thì dữ liệu vẫn vào file CSV
```

```
                appCtrl->processSaveTransaction();
```

```
            }
```

```
        }
```

```
        isPowerOn = false; // Chuyển trạng thái sang Tắt
```

```
        // Trả về false để GUI hiển thị thông báo "Đã lưu & Đang tắt..."
```

```
        return false;
```

```
}
}
```

Bảng 3.6. Chương trình Transaction.h

```
#pragma once
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>

// Nhúng DataManager để lớp này có thể truy cập các hàm kiểm tra kho hàng, nhân viên
#include "11b_DataManager.h"

using namespace std;

/**
 * Cấu trúc Item: Đại diện cho một dòng hàng hóa trong giao dịch
 * Ví dụ: Một dòng trong hóa đơn gồm mã, tên, số lượng, giá...
 */
struct Item {
    string sku;        // Mã định danh sản phẩm (Stock Keeping Unit)
    string ten;        // Tên gọi sản phẩm
    int sl = 0;        // Số lượng (mặc định bằng 0 để tránh lỗi giá trị rác)
    double gia = 0.0;  // Đơn giá
    double khauTru = 0.0; // Số tiền được giảm trừ/chiết khấu
    string hsd;        // Hạn sử dụng (lưu dạng chuỗi YYYY-MM-DD)
    double tong = 0.0; // Thành tiền của riêng dòng này (thường là sl * gia - khauTru)
};

/**
 * Lớp Transaction: Lớp cha định nghĩa các hành vi chung cho mọi loại phiếu
 * (Hóa đơn, Phiếu nhập, Phiếu xuất)
 */
class Transaction {
protected:
    // Mảng lưu trữ các mặt hàng đang được chọn (Lưu tạm trên RAM)
    // Dữ liệu này sẽ được hiển thị lên ListView trên giao diện
    vector<Item> list;

    string id; // Mã số phiếu (Ví dụ: HD001, NK002)
    string time; // Dấu mốc thời gian thực hiện giao dịch (Timestamp)
};
```

```
// Con trỏ kết nối đến DataManager để thực hiện các nghiệp vụ như:
// Kiểm tra SKU tồn tại, lấy giá hàng, trừ tồn kho...
DataManager* db = nullptr;
```

public:

```
// Hàm hủy ảo (Virtual Destructor) để đảm bảo các lớp con được giải phóng bộ nhớ an toàn
virtual ~Transaction() = default;
```

```
// Thêm trực tiếp một Item vào danh sách tạm
void addTempItem(const Item& itm) {
    list.push_back(itm);
}
```

```
// "Cắm dây" kết nối với DataManager
void setDataManager(DataManager* manager) {
    this->db = manager;
}
```

```
// Kiểm tra xem danh sách hiện tại có trống không
bool isEmpty() const { return list.empty(); }
```

```
// Trả về danh sách các mặt hàng (dùng để GUI vẽ lên bảng)
const vector<Item>& getList() const { return list; }
```

```
/**
 * Tính tổng số tiền của toàn bộ giao dịch
 * Cộng dồn tất cả các giá trị 'tong' của từng Item trong vector
 */
```

```
double getTotalAmount() const {
    double total = 0;
    for (const auto& item : list) {
        total += item.tong;
    }
    return total;
}
```

```
// --- CÁC HÀM THUẦN ẢO (PURE VIRTUAL FUNCTIONS) ---
// Các lớp con (Invoice, Import, Export) BẮT BUỘC phải cài đặt cụ thể các hàm này
```

```
// 1. Kiểm tra tính hợp lệ của sản phẩm (Ví dụ: Bán hàng thì phải kiểm tra tồn kho)
virtual bool validateItem(const Item& in) = 0;
```

```
// 2. Thêm sản phẩm vào list (Xử lý cộng dồn nếu trùng mã SKU)
```


	<pre> virtual void addItem(const Item& in) = 0; // 3. Xóa một dòng hàng hóa khỏi danh sách dựa trên vị trí (index) virtual void removeItem(int index) { if (index >= 0 && index < (int)list.size()) { list.erase(list.begin() + index); } } // 4. Kiểm tra tổng thể trước khi bấm nút Lưu virtual bool validateBeforeSave() { if (list.empty()) { cout << "Loi: Danh sach trong, khong the luu!" << endl; return false; } return true; } // 5. Ghi dữ liệu ra file vật lý (CSV/TXT) virtual void save() = 0; // 6. Ghi lại lịch sử hoạt động vào file nhật ký hệ thống virtual void writeLog() = 0; /** * 7. Làm sạch trạng thái: Xóa hết hàng trong list, reset mã và thời gian * Thường dùng sau khi đã lưu file thành công để chuẩn bị cho giao dịch mới. */ void reset() { list.clear(); id = ""; time = ""; } }; </pre>
--	--

	Bảng 3.7. Chương trình Invoice.h
	<pre> #pragma once #include <iostream> #include <vector> #include <string> #include <fstream> </pre>

```

#include <algorithm>
#include <ctime>
#include "lb_Transaction.h" // Kế thừa lớp cơ sở Transaction

using namespace std;

/**
 * Lớp Invoice: Quản lý nghiệp vụ bán hàng và xuất hóa đơn
 */
class Invoice : public Transaction {
public:
    // --- 1. KIỂM TRA TÍNH HỢP LỆ (Validate) ---
    // Đảm bảo hàng bán ra phải có trong danh mục và đủ số lượng tồn kho
    bool validateItem(const Item& in) override {
        if (db == nullptr) return false;

        // Kiểm tra SKU có tồn tại trong hệ thống (file danh mục hàng) hay không
        if (!db->isValidSKU(in.sku)) return false;

        // Kiểm tra số lượng nhập vào phải lớn hơn 0
        // và không được vượt quá số lượng tồn kho thực tế của mã hàng đó
        if (in.sl <= 0 || in.sl > db->getTonKho(in.sku)) return false;

        return true;
    }

    // --- 2. THÊM MẶT HÀNG VÀO GIỎ HÀNG TẠM ---
    void addItem(const Item& in) override {
        // Bước 1: Validate trước, nếu sai (ví dụ: quá tồn kho) thì thoát ngay
        if (!validateItem(in)) {
            return;
        }

        // Bước 2: Tự động lấy thông tin chi tiết (Tên, Giá, Khấu trừ) từ Database
        Product p = db->getProductInfo(in.sku);

        bool found = false;
        // Bước 3: Kiểm tra sản phẩm đã có trong giỏ hàng hiện tại chưa
        for (auto& item : list) {
            if (item.sku == in.sku) {
                // Nếu đã có: Cộng dồn số lượng
                item.sl += in.sl;
                // Tính lại thành tiền: (Số lượng * Giá) - Khấu trừ
                item.tong = (item.sl * p.gia) - p.khauTru;
            }
        }
    }
};

```

```

        found = true;
        break;
    }
}

// Bước 4: Nếu là sản phẩm mới thêm lần đầu vào hóa đơn
if (!found) {
    Item newItem;
    newItem.sku = in.sku;
    newItem.ten = p.ten;
    newItem.sl = in.sl;
    newItem.gia = p.gia;
    newItem.khauTru = p.khauTru;
    newItem.hsd = p.hsd;
    newItem.tong = (in.sl * p.gia) - p.khauTru;

    list.push_back(newItem);
}
}

// --- 3. LƯU VÀ XUẤT HÓA ĐƠN ---
void save() override {
    // Kiểm tra xem giỏ hàng có trống không trước khi lưu
    if (!validateBeforeSave()) return;

    // 1. Tạo mã hóa đơn dựa trên thời gian thực (Unix Timestamp)
    time_t now = ::time(0); // Sử dụng :: để gọi hàm toàn cục của C
    this->time = to_string(now);
    this->id = "HD_" + this->time;

    // 2. Định dạng tên file cho 2 loại báo cáo (CSV để quản lý, TXT để in bill)
    string csvFileName = "hoadon_" + this->time + ".csv";
    string txtFileName = "hoadon_" + this->time + ".txt";

    // --- GHI FILE CSV (Dùng để nhập liệu/Excel) ---
    ofstream fCsv(csvFileName);
    if (fCsv.is_open()) {
        fCsv << "STT,SKU,Ten Hang,SL,Gia,Khau Tru,HSD,Thanh Tien\n";
        int stt = 1;
        for (const auto& item : list) {
            fCsv << stt++ << "," << item.sku << "," << item.ten << ","
                << item.sl << "," << item.gia << "," << item.khauTru << ","
                << item.hsd << "," << item.tong << "\n";
        }
    }
}

```

	<pre> // NGHIỆP VỤ QUAN TRỌNG: Trừ số lượng tồn kho trong hệ thống db->updateStock(item.sku, -item.sl); } fCsv << "Tong Tien,,,,,,,,," << getTotalAmount() << "\n"; fCsv.close(); } // --- GHI FILE TXT (Dạng hóa đơn in ấn cho khách hàng) --- ofstream fTxt(txtFileName); if (fTxt.is_open()) { fTxt << "=== HOA DON BAN HANG ===\n"; fTxt << "Ma Hoa Don: " << this->id << "\n"; fTxt << "Thoi gian: " << this->time << "\n"; fTxt << "-----\n"; int stt = 1; for (const auto& item : list) { fTxt << stt++ << ". " << item.ten << " (SKU: " << item.sku << ")\n"; fTxt << " SL: " << item.sl << " x Gia: " << item.gia << " = " << item.tong << "\n"; } fTxt << "-----\n"; fTxt << "TONG CONG: " << getTotalAmount() << " VND\n"; fTxt.close(); } } // --- 4. GHI NHẬT KÝ HOẠT ĐỘNG (Logging) --- void writeLog() override { // Lưu lại lịch sử bán hàng vào nhật ký chung của hệ thống ofstream fLog("nhatky.csv", ios::app); // Chế độ ios::app để ghi nối tiếp vào cuối file if (fLog.is_open()) { // Định dạng: [Thời gian, Hành động, Mã hóa đơn, Tổng tiền] fLog << this->time << ",BAN_HANG," << this->id << "," << getTotalAmount() << "\n"; fLog.close(); } } }; </pre>
--	--

	Bảng 3.8. Chương trình Import.h
	<pre> #pragma once #include <iostream> #include <vector> </pre>

```

#include <string>
#include <fstream>
#include <ctime>
#include "1b_Transaction.h" // Kế thừa từ lớp cơ sở Transaction

using namespace std;

/**
 * Lớp Import: Quản lý nghiệp vụ nhập hàng hóa vào kho
 */
class Import : public Transaction {
private:
    string nhaCC;    // Lưu trữ thông tin Nhà cung cấp cho đơn nhập hiện tại
    string ngayNhap; // Lưu trữ ngày nhập hàng

public:
    // Nhận thông tin chi tiết về đơn nhập từ giao diện (GUI)
    void setImportDetails(string nhaCungCap, string date) {
        this->nhaCC = nhaCungCap;
        this->ngayNhap = date;
    }

    // --- 1. KIỂM TRA TÍNH HỢP LỆ CỦA MẶT HÀNG ---
    bool validateItem(const Item& in) override {
        // [QUY TẮC]: Mã SKU, Tên sản phẩm và Hạn sử dụng không được để trống
        if (in.sku.empty() || in.ten.empty() || in.hsd.empty()) return false;

        // [QUY TẮC]: Số lượng nhập và Giá nhập phải lớn hơn 0
        if (in.sl <= 0 || in.gia <= 0) return false;

        return true;
    }

    // --- 2. THÊM MẶT HÀNG VÀO DANH SÁCH CHỜ ---
    void addItem(const Item& in) override {
        // Kiểm tra tính hợp lệ trước khi cho phép thêm vào danh sách
        if (!validateItem(in)) {
            return; // Nếu không hợp lệ, thoát ra để giao diện hiển thị thông báo lỗi
        }

        bool found = false;
        // Kiểm tra xem sản phẩm (SKU) này đã có trong danh sách nhập hiện tại chưa
        for (auto& item : list) {
            if (item.sku == in.sku) {

```

```

        // Nếu đã tồn tại: Cộng dồn số lượng và cập nhật thông tin mới nhất (Giá, HSD)
        item.sl += in.sl;
        item.gia = in.gia;
        item.hsd = in.hsd;
        item.tong = item.sl * item.gia; // Tính lại thành tiền
        found = true;
        break;
    }
}

// Nếu là SKU mới trong đơn nhập này
if (!found) {
    Item newItem = in;
    newItem.khauTru = 0; // Hàng nhập kho mới luôn có khấu trừ bằng 0
    newItem.tong = in.sl * in.gia;
    list.push_back(newItem); // Thêm vào mảng tạm (vector)
}
}

// --- 3. KIỂM TRA ĐIỀU KIỆN TRƯỚC KHI XUẤT FILE ---
bool validateBeforeSave() override {
    // Danh sách hàng phải có ít nhất 1 món và phải có ngày nhập mới cho lưu
    if (list.empty() || this->ngayNhap.empty()) return false;
    return true;
}

// --- 4. LƯU CHI TIẾT ĐƠN NHẬP VÀ CẬP NHẬT KHO ---
void save() override {
    if (!validateBeforeSave()) return;

    // 1. Tạo mã nhập kho (ID) duy nhất dựa trên thời gian thực
    // Sử dụng ::time(0) để chỉ định rõ hàm time của hệ thống C
    time_t now = ::time(0);
    this->time = to_string(now);
    this->id = "NK_" + this->time;

    // 2. Tạo và ghi file CSV chi tiết đơn nhập
    string fileName = "nhapkho_" + this->time + ".csv";
    ofstream fCsv(fileName);
    if (fCsv.is_open()) {
        // Ghi thông tin chung của đơn hàng
        fCsv << "Ma NK: " << this->id << ", Ngay Nhap: " << this->ngayNhap << ", Nha CC: " <<
this->nhaCC << "\n";
        fCsv << "STT,SKU,Ten SP,SL,Gia Nhap,HSD,Thanh Tien\n";
    }
}

```

```

int stt = 1;
for (const auto& item : list) {
    // Ghi chi tiết từng sản phẩm
    fCsv << stt++ << "," << item.sku << "," << item.ten << ","
        << item.sl << "," << item.gia << "," << item.hsd << ","
        << item.tong << "\n";

    // --- NGHIỆP VỤ THỰC TẾ ---
    // Gọi sang DataManager để cộng thêm số lượng vào tổng tồn kho của hệ thống
    if (db != nullptr) {
        db->updateStock(item.sku, item.sl);
    }
}
// Ghi tổng tiền của toàn bộ phiếu nhập
fCsv << "Tong Tien,,,,," << getTotalAmount() << "\n";
fCsv.close();
}
}

// --- 5. GHI NHẬT KÝ HOẠT ĐỘNG ---
void writeLog() override {
    // Ghi lại lịch sử nhập kho vào file nhatkys.csv để theo dõi dòng thời gian
    ofstream fLog("nhatkys.csv", ios::app); // Mở ở chế độ append (ghi tiếp vào cuối file)
    if (fLog.is_open()) {
        // Định dạng lưu: [Thời gian Unix, Hành động, Mã phiếu nhập]
        fLog << this->time << ",NHAP_KHO," << this->id << "\n";
        fLog.close();
    }
}
};

```

Bảng 3.9. Chương trình Export.h

```

#pragma once
#include <iostream>
#include <vector>
#include <string>
#include <fstream>
#include <ctime>
#include "lb_Transaction.h" // Kế thừa lớp cơ sở Transaction để dùng chung cấu trúc

```

```

using namespace std;

/**
 * Lớp Export: Xử lý nghiệp vụ xuất kho hàng hóa
 */
class Export : public Transaction {
private:
    string lyDo;    // Lý do xuất (Mượn hàng, Tiêu hủy, Thiên nguyện...)
    string ngayXuat; // Ngày thực hiện xuất kho (do người dùng nhập từ GUI)

public:
    // Cập nhật thông tin chi tiết về đơn xuất từ giao diện người dùng
    void setExportDetails(string lyDo, string ngayXuat) {
        this->lyDo = lyDo;
        this->ngayXuat = ngayXuat;
    }

    // --- 1. KIỂM TRA TÍNH HỢP LỆ CỦA MẶT HÀNG (Validate) ---
    bool validateItem(const Item& in) override {
        // Kiểm tra SKU và tên không được để trống
        if (in.sku.empty() || in.ten.empty()) return false;

        // Số lượng xuất phải lớn hơn 0
        if (in.sl <= 0) return false;

        // Phải có kết nối với DataManager (db) để kiểm tra dữ liệu hệ thống
        if (db == nullptr) return false;

        // Kiểm tra mã SKU này có tồn tại trong danh sách sản phẩm không
        if (!db->isValidSKU(in.sku)) return false;

        // QUAN TRỌNG: Kiểm tra số lượng xuất không được vượt quá số lượng tồn kho hiện tại
        if (in.sl > db->getTonKho(in.sku)) return false;

        return true;
    }

    // --- 2. THÊM MẶT HÀNG VÀO DANH SÁCH TẠM ---
    void addItem(const Item& in) override {
        // Nếu mặt hàng không hợp lệ (sai SKU hoặc thiếu hàng) thì không thêm
        if (!validateItem(in)) {
            return;
        }
    }

```



```

bool found = false;
// Kiểm tra xem SKU này đã có trong danh sách chuẩn bị xuất chưa (xử lý trùng lặp)
for (auto& item : list) {
    if (item.sku == in.sku) {
        // Nếu đã có: Cộng dồn số lượng và cập nhật tổng tiền mặt hàng đó
        item.sl += in.sl;
        item.tong = item.sl * item.gia;
        found = true;
        break;
    }
}

// Nếu là mặt hàng mới trong đơn xuất này
if (!found) {
    // Lấy thêm thông tin bổ sung (Khấu trừ, HSD) từ Database hệ thống
    Product p = db->getProductInfo(in.sku);

    Item newItem = in;
    newItem.khauTru = p.khauTru;
    newItem.hsd = p.hsd;
    newItem.tong = in.sl * in.gia;

    list.push_back(newItem); // Thêm vào mảng tạm
}

// --- 3. KIỂM TRA ĐIỀU KIỆN TRƯỚC KHI LƯU FILE ---
bool validateBeforeSave() override {
    // Kiểm tra danh sách xuất không rỗng và ngày xuất đã được điền
    if (list.empty() || this->ngayXuat.empty()) {
        return false;
    }
    return true;
}

// --- 4. LƯU CHI TIẾT PHIẾU XUẤT RA FILE CSV ---
void save() override {
    if (!validateBeforeSave()) return;

    // Tạo mã định danh duy nhất cho phiếu xuất dựa trên thời gian (timestamp)
    time_t now = ::time(0);
    this->time = to_string(now);
    this->id = "XK_" + this->time; // Ví dụ: XK_1714812345
}

```

```

// Tạo file CSV riêng cho mỗi lần xuất kho
string fileName = "xuatkho_" + this->time + ".csv";
ofstream fCsv(fileName);

if (fCsv.is_open()) {
    // Ghi dòng tiêu đề phiếu xuất
    fCsv << "Ma XK: " << this->id << ", Ngay Xuat: " << this->ngayXuat << ", Ly Do: " <<
this->lyDo << "\n";
    fCsv << "STT,SKU,Ten SP,SL,Gia Tri,KT,HSD,Tong\n";

    int stt = 1;
    for (const auto& item : list) {
        // Ghi từng mặt hàng
        fCsv << stt++ << ", " << item.sku << ", " << item.ten << ", "
        << item.sl << ", " << item.gia << ", " << item.khauTru << ", "
        << item.hsd << ", " << item.tong << "\n";

        // --- NGHIỆP VỤ QUAN TRỌNG ---
        // Cập nhật lại kho tổng: Trừ đi số lượng vừa xuất (dùng dấu âm -item.sl)
        if (db != nullptr) {
            db->updateStock(item.sku, -item.sl);
        }
    }
    // Ghi tổng giá trị của toàn bộ phiếu xuất
    fCsv << "Tong Tien,,,,,,,, " << getTotalAmount() << "\n";
    fCsv.close();
}

// --- 5. GHI NHẬT KÝ HOẠT ĐỘNG ---
void writeLog() override {
    // Ghi lại dấu vết hoạt động vào file nhật ký dùng chung của hệ thống
    ofstream fLog("nhatky.csv", ios::app); // Mở ở chế độ append (ghi tiếp vào cuối file)
    if (fLog.is_open()) {
        // Định dạng: [Thời gian, Loại hành động, Mã phiếu xuất]
        fLog << this->time << ",XUAT_KHO," << this->id << "\n";
        fLog.close();
    }
}
};

```

Bảng 3.10. Chương trình NoteManager.h

```
#pragma once // Đảm bảo file header chỉ được nạp một lần duy nhất khi biên dịch
#include <iostream>
#include <vector>
#include <string>
#include <fstream> // Thư viện hỗ trợ ghi file (soghichu.csv, nhatky.csv)
#include <ctime> // Thư viện hỗ trợ lấy thời gian hệ thống
#include "lib_DataManager.h" // Nhúng lớp quản lý dữ liệu để kiểm tra User hợp lệ

using namespace std;

/**
 * Cấu trúc (Struct) lưu trữ một dòng ghi chú
 * Được thiết kế để tương thích với việc hiển thị dữ liệu lên ListView của GUI
 */
struct NoteItem {
    string caLam; // Tên ca làm việc
    string nguoiLap; // Tên/Mã người lập ghi chú
    string noiDung; // Nội dung chi tiết của ghi chú
    string phanLoai; // Loại ghi chú (Y tế, Cháy nổ, Thuế...)
    string thoiGian; // Thời gian lập ghi chú
    string quanTrong; // Mức độ ưu tiên hoặc mã liên quan
};

class NoteManager {
private:
    DataManager* db = nullptr; // Con trỏ kết nối đến lớp quản lý dữ liệu gốc
    vector<NoteItem> list; // Danh sách các ghi chú đã thêm (dùng để hiển thị real-time)

public:
    // Kết nối đối tượng DataManager vào lớp NoteManager (Dependency Injection)
    void setDataManager(DataManager* manager) {
        this->db = manager;
    }

    // Trả về danh sách ghi chú (dạng tham chiếu hằng) để GUI lấy dữ liệu hiển thị
    const vector<NoteItem>& getList() const { return list; }

    /**
     * 1. HÀM KIỂM TRA TÍNH HỢP LỆ (VALIDATION)
     * Đảm bảo các dữ liệu đầu vào không bị rỗng và người lập có tồn tại trong hệ thống
     */
    bool validateNote(string ca, string nguoi, string noiDung, string loai) {
        // Kiểm tra cơ bản: Không được để trống các trường quan trọng
    }
};
```

```

if (noiDung.empty() || ca.empty() || nguoi.empty() || loai.empty()) return false;

// Kiểm tra phân loại (Fix lỗi C2678 liên quan đến kiểu chuỗi Unicode/L"... " nếu có)
if (loai != "Thu tuc" && loai != "Giay to" && loai != "Y te" && loai != "Chay no" && loai !=
"Thue") {
    // Có thể bổ sung logic cảnh báo ở đây nếu cần thiết
}

// Gọi sang DataManager để kiểm tra xem "nguoi" (mã NV/QL) có tồn tại trong file CSV gốc
không
if (db == nullptr || !db->isValidUser(nguoi)) {
    return false;
}

return true;
}

/**
 * 2. XỬ LÝ LƯU GHI CHÚ VÀ CẬP NHẬT GIAO DIỆN
 * Lưu vào file vật lý và đẩy vào mảng động để hiển thị ngay lập tức
 */
bool addAndSaveNote(string ca, string nguoi, string noiDung, string loai, string thoiGian, string
mucDo) {
    // Bước 1: Validate dữ liệu trước khi làm bất cứ việc gì khác
    if (!validateNote(ca, nguoi, noiDung, loai)) {
        return false;
    }

    // Bước 2: Tạo định danh duy nhất (Unique ID) cho ghi chú bằng timestamp
    time_t now = ::time(0);
    string timeStr = to_string(now);
    string maGC = "GC_" + timeStr;

    // Bước 3: Ghi dữ liệu chi tiết vào file "soghichu.csv" (Chế độ Append - ghi tiếp vào cuối)
    ofstream fCsv("soghichu.csv", ios::app);
    if (fCsv.is_open()) {
        // Định dạng lưu: [thời gian, nhãn, ca, người lập, phân loại, mức độ, nội dung]
        fCsv << timeStr << ",GHI_CHU," << ca << "," << nguoi << ","
        << loai << "," << mucDo << "," << noiDung << "\n";
        fCsv.close();
    }

    // Bước 4: Ghi vết (Log) vào file "nhatky.csv" để hệ thống quản lý chung
    ofstream logFile("nhatky.csv", ios::app);

```

```

        if (logFile.is_open()) {
            logFile << timeStr << ",GHI_CHU," << maGC << "\n";
            logFile.close();
        }

        // Bước 5: Đưa vào danh sách trong bộ nhớ (vector)
        // Việc này giúp ListView trên giao diện cập nhật ngay mà không cần đọc lại file từ đầu
        NoteItem n = { ca, nguoi, noiDung, loai, thoiGian, mucDo };
        list.push_back(n);

        return true;
    }
};

```

Bảng 3.11. Chương trình ShiftManager.h

```

#pragma once // Ngăn chặn việc khai báo chồng chéo thư viện khi include nhiều nơi
#include <iostream>
#include <vector>
#include <string>
#include <fstream> // Thư viện để làm việc với file (ghi nhật ký)
#include <ctime>    // Thư viện xử lý thời gian hệ thống
#include <windows.h> // Hỗ trợ các hàm hệ thống Windows

// Những lớp quản lý dữ liệu để sử dụng các hàm kiểm tra mã nhân viên/quản lý
#include "11b_DataManager.h"

using namespace std;

class ShiftManager {
private:
    // Con trỏ db kết nối tới đối tượng DataManager để truy vấn dữ liệu từ file CSV
    DataManager* db = nullptr;

    /*
     * Hàm nội bộ: Lấy thời gian hiện tại của hệ thống
     * Trả về chuỗi định dạng: YYYY-MM-DD HH:MM:SS
     */
    string getCurrentTimeStr() {
        time_t now = time(0); // Lấy thời gian hiện tại dưới dạng số giây
        tm ltm;
    }

```

```

// Sử dụng localtime_s (an toàn hơn localtime) để chuyển đổi sang cấu trúc thời gian
localtime_s(&ltm, &now);

// Tạo bộ đệm lưu trữ chuỗi thời gian
string buffer;
buffer.resize(80);

// Định dạng thời gian thành chuỗi (Năm-Tháng-Ngày Giờ:Phút:Giây)
strftime(&buffer.front(), buffer.size(), "%Y-%m-%d %H:%M:%S", &ltm);
return string(buffer.c_str());
}

public:
// Phương thức để gán (tiêm) đối tượng DataManager vào lớp này
void setDataManager(DataManager* manager) {
    this->db = manager;
}

/*
* Hàm lưu dữ liệu MỞ CA
* maQL: Mã quản lý, maNV: Mã nhân viên, tienDauStr: Tiền đầu ca, tienKetStr: Tiền trong kết
*/
bool saveMoCa(string maQL, string maNV, string tienDauStr, string tienKetStr) {
    // 1. Kiểm tra đầu vào: Không được để trống
    if (maQL.empty() || maNV.empty() || tienDauStr.empty() || tienKetStr.empty()) return false;

    // Kiểm tra mã QL và mã NV có tồn tại trong cơ sở dữ liệu (file CSV) không
    if (db == nullptr || !db->isValidUser(maQL) || !db->isValidUser(maNV)) return false;

    // 2. Chuyển đổi kiểu dữ liệu từ chuỗi sang số thực (double)
    double tienDau = 0, tienKet = 0;
    try {
        tienDau = stod(tienDauStr);
        tienKet = stod(tienKetStr);
        // Tiền không được là số âm
        if (tienDau < 0 || tienKet < 0) return false;
    }
    catch (...) {
        return false; // Trả về false nếu chuỗi tiền chứa ký tự không phải số
    }

    // 3. Ghi dữ liệu vào file nhatty.csv (chế độ ios::app - ghi tiếp vào cuối file)
    ofstream file("nhatty.csv", ios::app);
    if (file.is_open()) {

```

```

        file << getCurrentTimeStr() << ",MO_CA,QL:" << maQL << ",NV:" << maNV
        << ",DauCa:" << tienDau << ",Ket:" << tienKet << "\n";
        file.close();
        return true;
    }
    return false;
}

/*
* Hàm lưu dữ liệu KẾT CA
* Thêm các thông số về doanh thu và chênh lệch tiền thực tế
*/
bool saveKetCa(string maQL, string maNV, string tienKetStr, string doanhThuStr, string
chenhLechStr, string ghiChu) {
    // 1. Kiểm tra rỗng cho các trường bắt buộc
    if (maQL.empty() || maNV.empty() || tienKetStr.empty() || doanhThuStr.empty() ||
chenhLechStr.empty()) return false;

    // 2. Kiểm tra tính hợp lệ của người dùng qua DataManager
    if (db == nullptr || !db->isValidUser(maQL) || !db->isValidUser(maNV)) return false;

    // 3. Kiểm tra và chuyển đổi định dạng tiền tệ
    double tienKet = 0, doanhThu = 0, chenhLech = 0;
    try {
        tienKet = stod(tienKetStr);
        doanhThu = stod(doanhThuStr);
        chenhLech = stod(chenhLechStr);
    }
    catch (...) {
        return false;
    }

    // 4. Mọi thứ hợp lệ -> Ghi dòng nhật ký kết ca vào file CSV
    ofstream file("nhatky.csv", ios::app);
    if (file.is_open()) {
        file << getCurrentTimeStr() << ",KET_CA,QL:" << maQL << ",NV:" << maNV
        << ",Ket:" << tienKet << ",DT:" << doanhThu << ",ChenhLech:" << chenhLech
        << ",Note:" << ghiChu << "\n";
        file.close();
        return true;
    }
    return false;
}
};

```

--	--

	Bảng 3.12. Chương trình DataExporter
	<pre> #pragma once // Đảm bảo file header này chỉ được bao gồm (include) một lần trong quá trình biên dịch #include <iostream> // Thư viện nhập xuất cơ bản #include <fstream> // Thư viện làm việc với file (đọc/ghi file) #include <string> // Thư viện xử lý chuỗi ký tự #include <vector> // Thư viện sử dụng mảng động (vector) using namespace std; class DataExporter { public: /** * Hàm xử lý xuất dữ liệu ra file * @param loiDuLieu: Loại dữ liệu muốn xuất (ví dụ: "Nhat ky", "Ghi chu") * @param dinhDang: Định dạng file đích ("TXT" hoặc "CSV") * @param thuMucLuu: Đường dẫn thư mục để lưu file sau khi xuất * @return: true nếu xuất thành công, false nếu có lỗi xảy ra */ bool exportFile(string loiDuLieu, string dinhDang, string thuMucLuu) { // --- BƯỚC 1: XÁC ĐỊNH FILE NGUỒN --- // Dựa vào loại dữ liệu người dùng yêu cầu để tìm file .csv tương ứng trong hệ thống string sourceFileName = ""; if (loiDuLieu == "Nhat ky" loiDuLieu == "Nhatky") { sourceFileName = "nhatky.csv"; } else if (loiDuLieu == "Ghi chu" loiDuLieu == "Ghichu") { sourceFileName = "soghichu.csv"; } else { // Nếu loại dữ liệu không nằm trong danh sách hỗ trợ -> Thoát và trả về thất bại return false; } // --- BƯỚC 2: KIỂM TRA ĐỊNH DẠNG ĐẦU RA --- // Chỉ chấp nhận hai loại định dạng là TXT hoặc CSV (không phân biệt hoa thường) if (dinhDang != "TXT" && dinhDang != "CSV" && dinhDang != "txt" && dinhDang != "csv") { </pre>


```

    return false;
}

// --- BƯỚC 3: MỞ FILE NGUỒN ĐỂ ĐỌC ---
ifstream sourceFile(sourceFileName); // Mở luồng đọc file
if (!sourceFile.is_open()) {
    return false; // Nếu không mở được (file không tồn tại), trả về false
}

// --- BƯỚC 4: TẠO FILE ĐÍCH (FILE XUẤT) ---
// Xác định đuôi file dựa trên định dạng người dùng chọn
string extension = (dinhDang == "TXT" || dinhDang == "txt") ? ".txt" : ".csv";

// Tạo đường dẫn đầy đủ: Thư mục lưu + dấu gạch chéo + tên file xuất + đuôi file
string destFileName = thuMucLuu + "\\\" + "Export_" + loaiDuLieu + extension;

ofstream destFile(destFileName); // Mở luồng ghi file
if (!destFile.is_open()) {
    sourceFile.close(); // Đóng file nguồn đã mở trước đó
    return false; // Trả về false nếu đường dẫn sai hoặc không có quyền ghi file
}

// --- BƯỚC 5: SAO CHÉP VÀ CHUYỂN ĐỔI DỮ LIỆU ---
string line;
// Đọc từng dòng từ file nguồn cho đến hết file
while (getline(sourceFile, line)) {

    // Nếu xuất ra định dạng TXT:
    // Thay thế tất cả dấu phẩy (,) từ file CSV gốc thành dấu Tab (\t) để trình bày đẹp hơn
    if (dinhDang == "TXT" || dinhDang == "txt") {
        for (char& c : line) {
            if (c == ',') c = '\t';
        }
    }

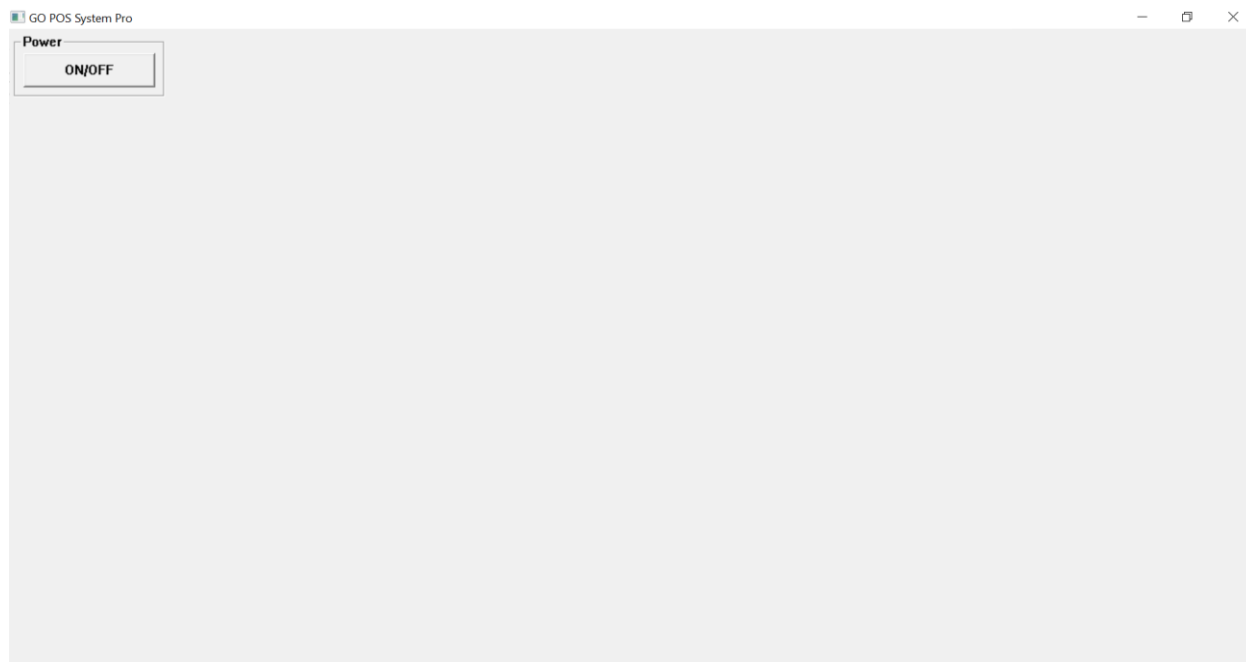
    // Ghi dòng dữ liệu (đã xử lý hoặc giữ nguyên) vào file đích
    destFile << line << "\n";
}

// --- BƯỚC 6: DỌN DẸP ---
sourceFile.close(); // Đóng kết nối file nguồn
destFile.close(); // Đóng kết nối file đích
return true; // Hoàn thành quá trình xuất dữ liệu
}

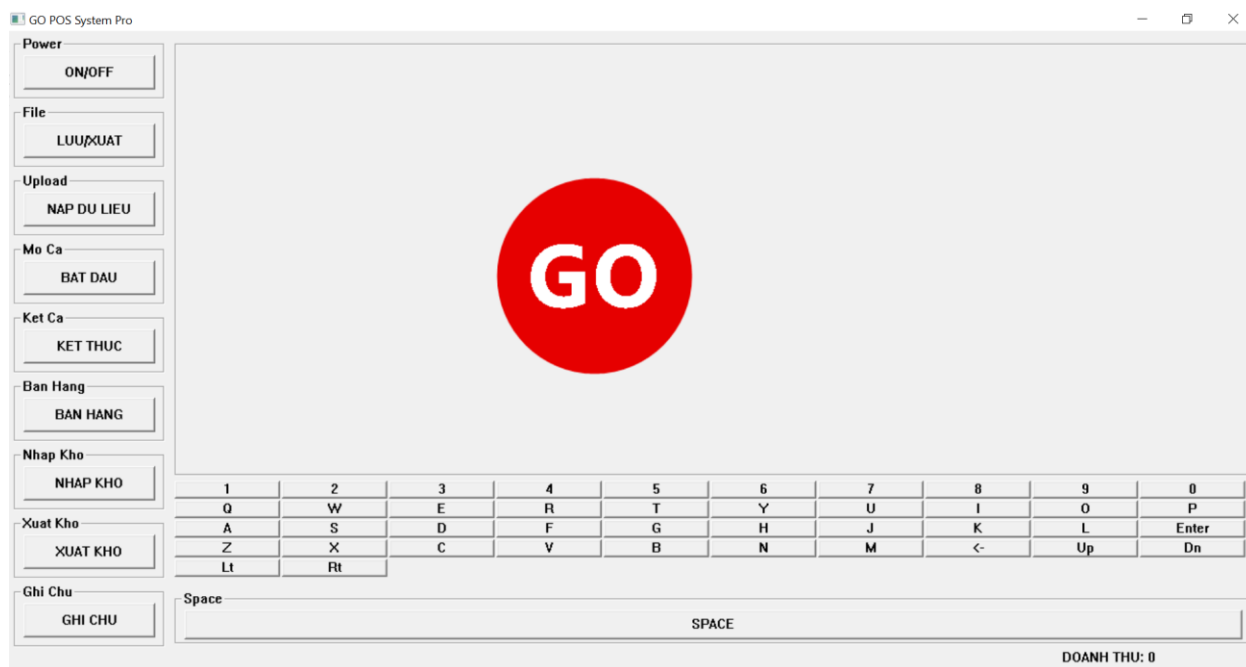
```

```
};
```

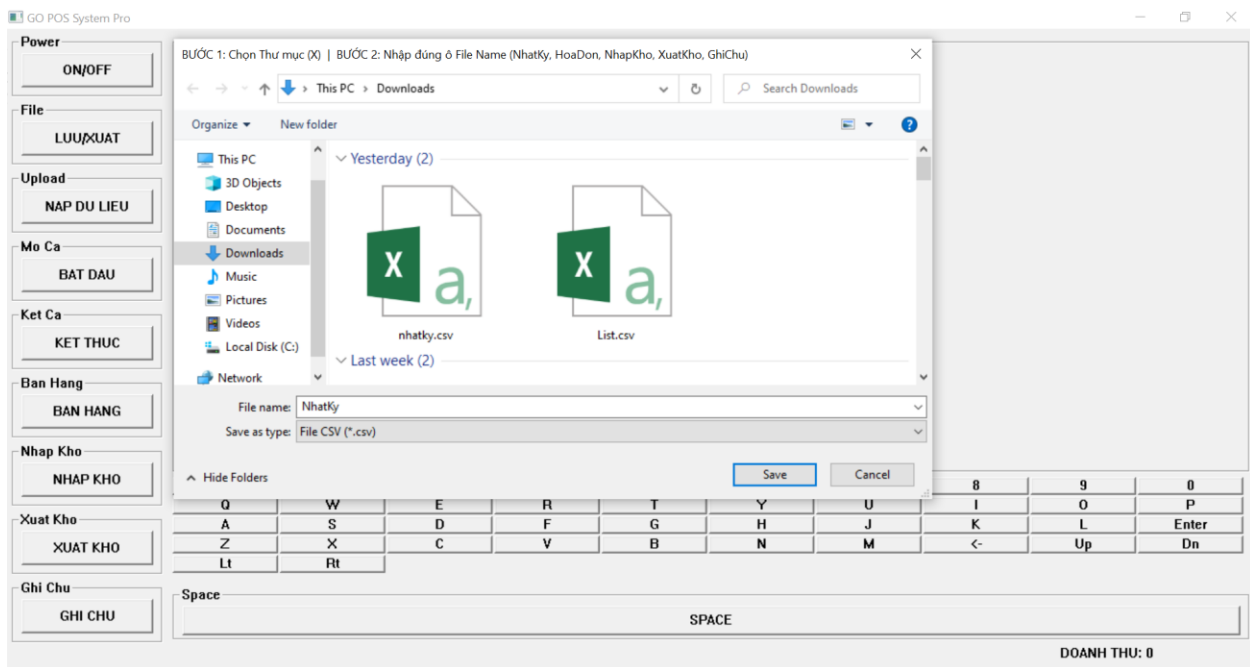
3.3. Kết quả giao diện



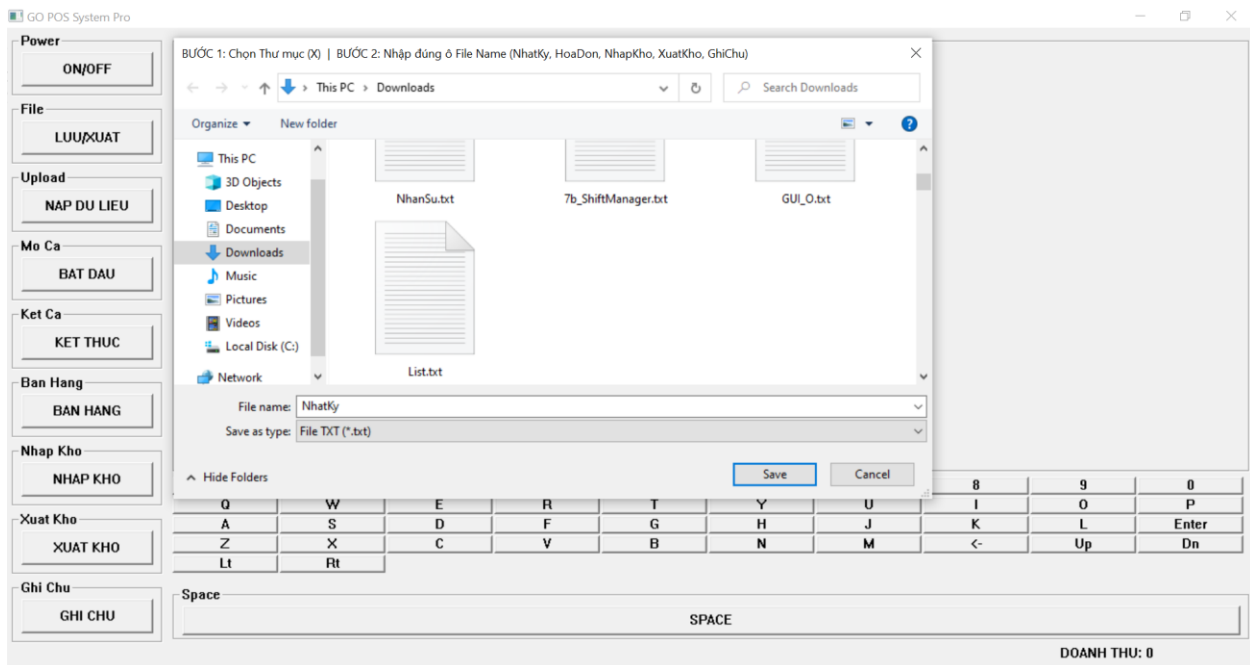
Hình 3.25. Giao diện khi tắt ứng dụng



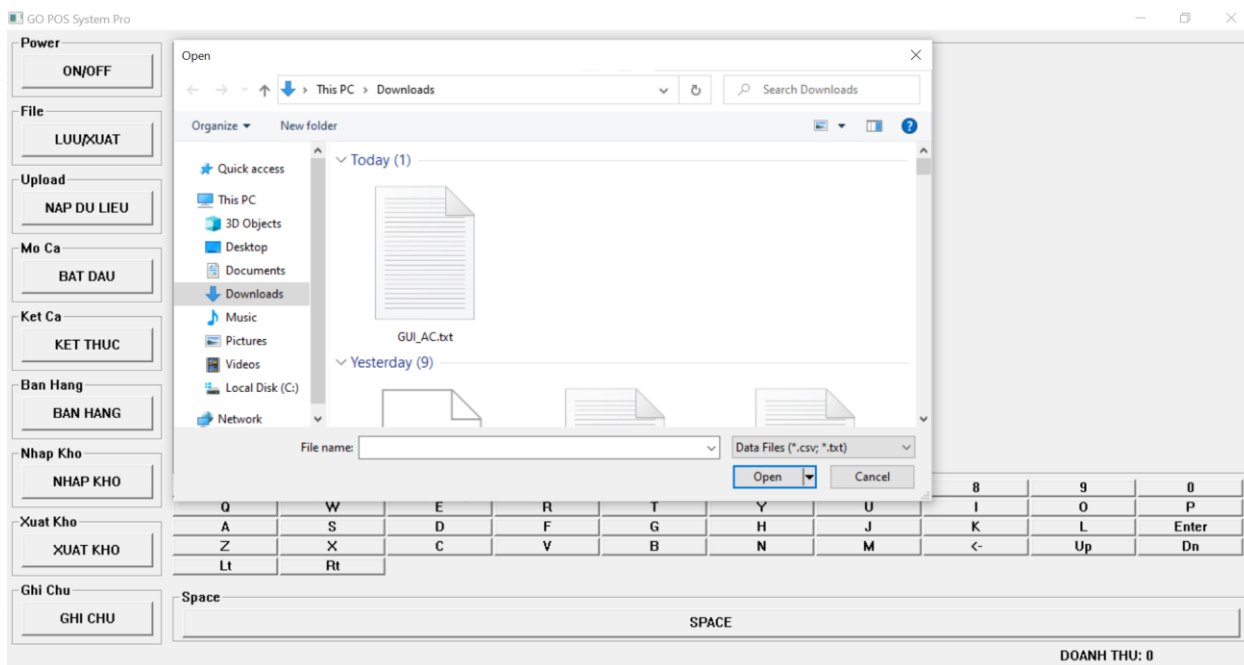
Hình 3.26. Giao diện khi bật ứng dụng



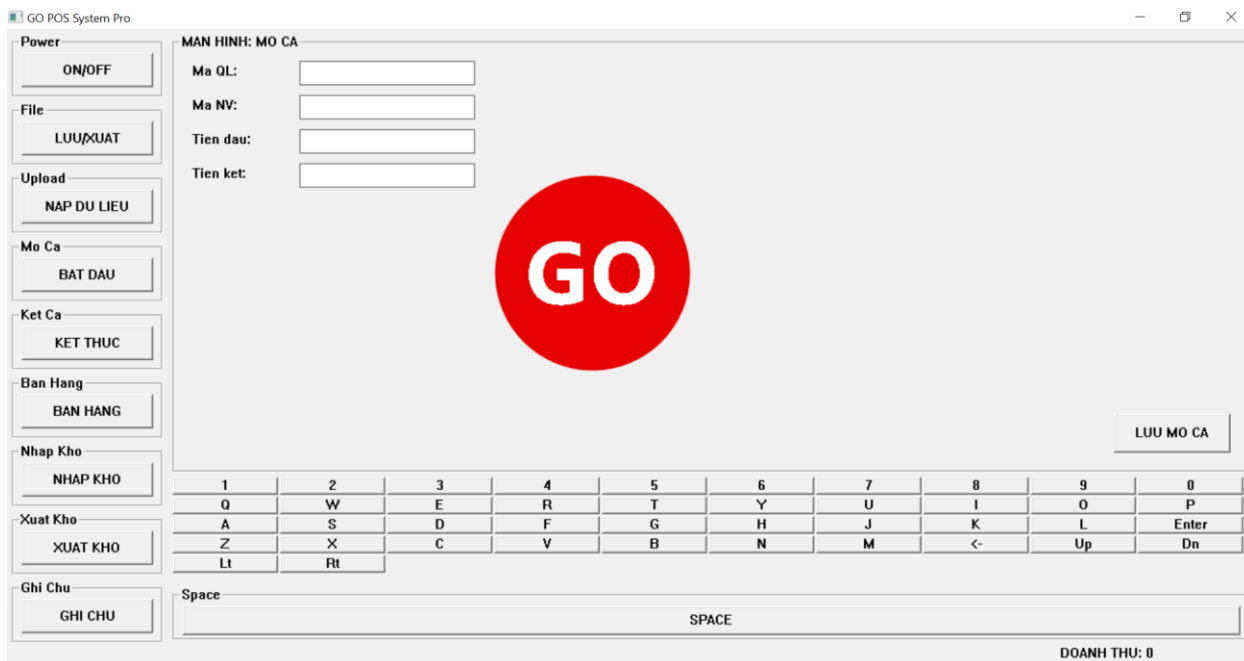
Hình 3.27. Giao diện LUU/XUAT (*.csv)



Hình 3.28. Giao diện LUU/XUAT (*.txt)



Hình 3.29. Giao diện NAP DU LIEU (*.csv, *.txt)



Hình 3.30. Giao diện BAT DAU

[illegible]

Hình 3.33. Giao diện NHẬP KHO

[illegible]

Hình 3.34. Giao diện XUAT KHO

27, SP027, Nuoc loc lon, 6000, 90, 2025-11-08, 2026-11-08, PepsiCo VN
 28, SP028, Tra dao, 14000, 50, 2025-09-25, 2026-03-25, PepsiCo VN
 29, SP029, Banh socola, 22000, 25, 2025-08-18, 2025-12-18, Mas an Consumer
 30, SP030, Mi cay, 6500, 110, 2025-09-19, 2026-03-19, AceCook VN
 31, SP031, Xuc xich duc, 15000, 40, 2025-12-01, 2026-03-01, Mas an Consumer
 32, SP032, Banh trung thu, 35000, 20, 2025-08-01, 2025-09-01, Mas an Consumer
 33, SP033, Nuoc ep tao, 18000, 30, 2025-11-15, 2026-02-15, TH True Milk
 34, SP034, Snack rong bien, 16000, 35, 2025-10-05, 2026-04-05, PepsiCo VN
 35, SP035, Mi han quoc, 12000, 60, 2025-09-30, 2026-03-30, AceCook VN
 36, SP036, Nuoc ep nho, 20000, 25, 2025-11-18, 2026-02-18, TH True Milk
 37, SP037, Sua dau nanh, 9000, 70, 2025-12-01, 2025-12-20, TH True Milk
 38, SP038, Banh mi kep, 17000, 45, 2025-10-10, 2025-10-20, Mas an Consumer
 39, SP039, Keo mut, 6000, 100, 2025-07-20, 2026-07-20, Mas an Consumer
 40, SP040, Tra tac, 10000, 80, 2025-11-01, 2026-05-01, PepsiCo VN
 41, SP041, Banh flan, 8000, 60, 2025-08-25, 2025-09-05, Mas an Consumer
 42, SP042, Nuoc ngot diet, 13000, 40, 2025-10-18, 2026-04-18, PepsiCo VN
 43, SP043, Mi udon, 15000, 35, 2025-09-22, 2026-03-22, AceCook VN
 44, SP044, Banh bao, 20000, 25, 2025-10-01, 2025-10-03, Mas an Consumer
 45, SP045, Sua bi do, 11000, 55, 2025-12-05, 2025-12-25, TH True Milk
 46, SP046, Nuoc ep oi, 16000, 30, 2025-11-10, 2026-02-10, TH True Milk
 47, SP047, Snack hanh, 9000, 70, 2025-10-08, 2026-04-08, PepsiCo VN
 48, SP048, Mi tom ly, 7500, 90, 2025-09-12, 2026-03-12, AceCook VN
 49, SP049, Banh quy bo, 18000, 35, 2025-08-28, 2026-02-28, Mas an Consumer
 50, SP050, Tra chanh, 9000, 80, 2025-11-02, 2026-05-02, PepsiCo VN
 51, SP051, Nuoc ep dua hau, 15000, 45, 2025-11-10, 2026-02-10, TH True Milk
 52, SP052, Mi cay han quoc, 13000, 60, 2025-09-01, 2026-03-01, AceCook VN
 53, SP053, Banh quy socola, 20000, 30, 2025-08-15, 2026-02-15, Mas an Consumer
 54, SP054, Keo cham, 5000, 150, 2025-07-01, 2026-07-01, Mas an Consumer
 55, SP055, Nuoc cam tuoi, 14000, 50, 2025-11-20, 2026-02-20, TH True Milk
 56, SP056, Sua chua uong, 8000, 70, 2025-12-01, 2025-12-20, TH True Milk
 57, SP057, Snack pho mai, 12000, 65, 2025-10-12, 2026-04-12, PepsiCo VN
 58, SP058, Mi tom chay, 6000, 100, 2025-09-10, 2026-03-10, AceCook VN
 59, SP059, Banh mi thit, 18000, 40, 2025-10-05, 2025-10-08, Mas an Consumer
 60, SP060, Tra sua matcha, 25000, 20, 2025-09-20, 2026-03-20, PepsiCo VN
 61, SP061, Nuoc ep cam dao, 17000, 35, 2025-11-11, 2026-02-11, TH True Milk
 62, SP062, Mi goi dac biet, 7000, 90, 2025-09-25, 2026-03-25, AceCook VN
 63, SP063, Banh quy vien, 16000, 50, 2025-08-18, 2026-02-18, Mas an Consumer
 64, SP064, Keo deo trai cay, 9000, 80, 2025-07-15, 2026-07-15, Mas an Consumer
 65, SP065, Nuoc chanh, 10000, 75, 2025-11-01, 2026-05-01, PepsiCo VN
 66, SP066, Sua tuoi khong duong, 17000, 45, 2025-12-10, 2025-12-25, TH True Milk
 67, SP067, Snack tom, 13000, 60, 2025-10-20, 2026-04-20, PepsiCo VN
 68, SP068, Mi xao bo, 8000, 85, 2025-09-18, 2026-03-18, AceCook VN
 69, SP069, Banh bong lan cuon, 20000, 30, 2025-08-10, 2025-08-25, Mas an Consumer
 70, SP070, Tra den, 12000, 65, 2025-11-05, 2026-05-05, PepsiCo VN
 71, SP071, Nuoc ep kiwi, 18000, 30, 2025-11-12, 2026-02-12, TH True Milk
 72, SP072, Mi ramen, 15000, 50, 2025-09-15, 2026-03-15, AceCook VN
 73, SP073, Banh quy bo sua, 19000, 40, 2025-08-22, 2026-02-22, Mas an Consumer
 74, SP074, Keo socola, 12000, 70, 2025-07-25, 2026-07-25, Mas an Consumer
 75, SP075, Nuoc ep dao, 16000, 45, 2025-11-18, 2026-02-18, TH True Milk
 76, SP076, Sua chua trai cay, 9000, 80, 2025-12-05, 2025-12-20, TH True Milk
 77, SP077, Snack khoai lang, 11000, 60, 2025-10-15, 2026-04-15, PepsiCo VN
 78, SP078, Mi tom cay dac biet, 7000, 95, 2025-09-20, 2026-03-20, AceCook VN
 79, SP079, Banh mi pate, 17000, 45, 2025-10-08, 2025-10-11, Mas an Consumer
 80, SP080, Tra sua truyen thong, 24000, 25, 2025-09-30, 2026-03-30, PepsiCo VN
 81, SP081, Nuoc ep xoai, 18000, 35, 2025-11-22, 2026-02-22, TH True Milk
 82, SP082, Mi tom dac biet, 6500, 100, 2025-09-12, 2026-03-12, AceCook VN
 83, SP083, Banh quy vani, 16000, 50, 2025-08-05, 2026-02-05, Mas an Consumer
 84, SP084, Keo deo la cay, 8000, 90, 2025-07-18, 2026-07-18, Mas an Consumer
 85, SP085, Nuoc cam ep, 14000, 60, 2025-11-15, 2026-02-15, TH True Milk
 86, SP086, Sua chua uong cam, 8500, 75, 2025-12-01, 2025-12-20, TH True Milk

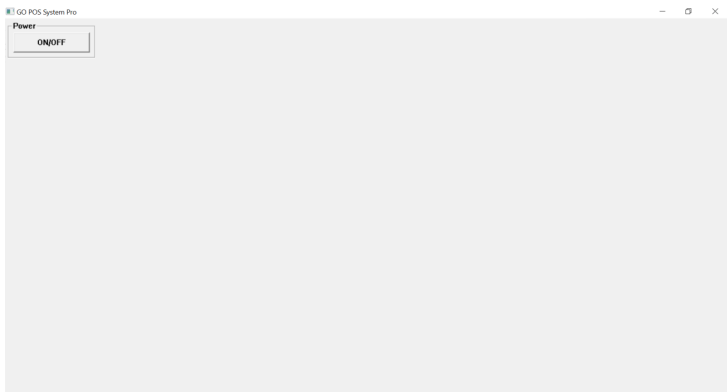
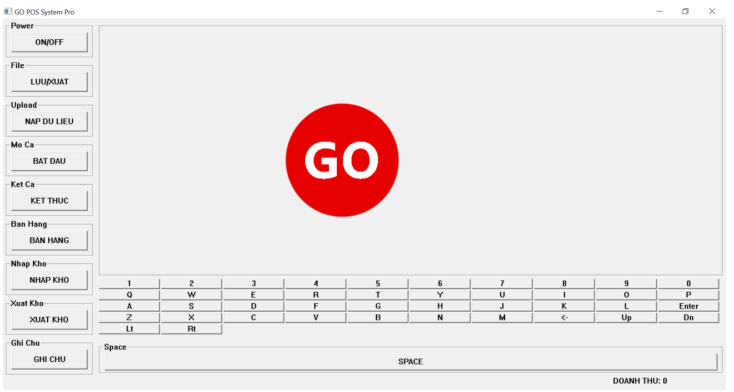
87,SP087,Snack cay,13000,55,2025-10-25,2026-04-25,PepsiCo VN
 88,SP088,Mi tom ly dac biet,7500,85,2025-09-10,2026-03-10,AceCook VN
 89,SP089,Banh mi chay,12000,50,2025-10-03,2025-10-06,Mas an Consumer
 90,SP090,Tra sua socola,26000,20,2025-09-28,2026-03-28,PepsiCo VN
 91,SP091,Nuoc ep dua,17000,40,2025-11-20,2026-02-20,TH True Milk
 92,SP092,Mi tom han quoc dac biet,8000,90,2025-09-18,2026-03-18,AceCook VN
 93,SP093,Banh quy cay,15000,60,2025-08-12,2026-02-12,Mas an Consumer
 94,SP094,Keo deo mini,4000,180,2025-07-05,2026-07-05,Mas an Consumer
 95,SP095,Nuoc cam lon,15000,55,2025-11-08,2026-02-08,TH True Milk
 96,SP096,Sua chua uong nho,9000,70,2025-12-10,2025-12-25,TH True Milk
 97,SP097,Snack pho mai cay,14000,60,2025-10-18,2026-04-18,PepsiCo VN
 98,SP098,Mi tom dac biet ly,8000,95,2025-09-22,2026-03-22,AceCook VN
 99,SP099,Banh mi bo,19000,35,2025-10-06,2025-10-09,Mas an Consumer
 100,SP100,Tra chanh dao,11000,80,2025-11-25,2026-05-25,PepsiCo VN

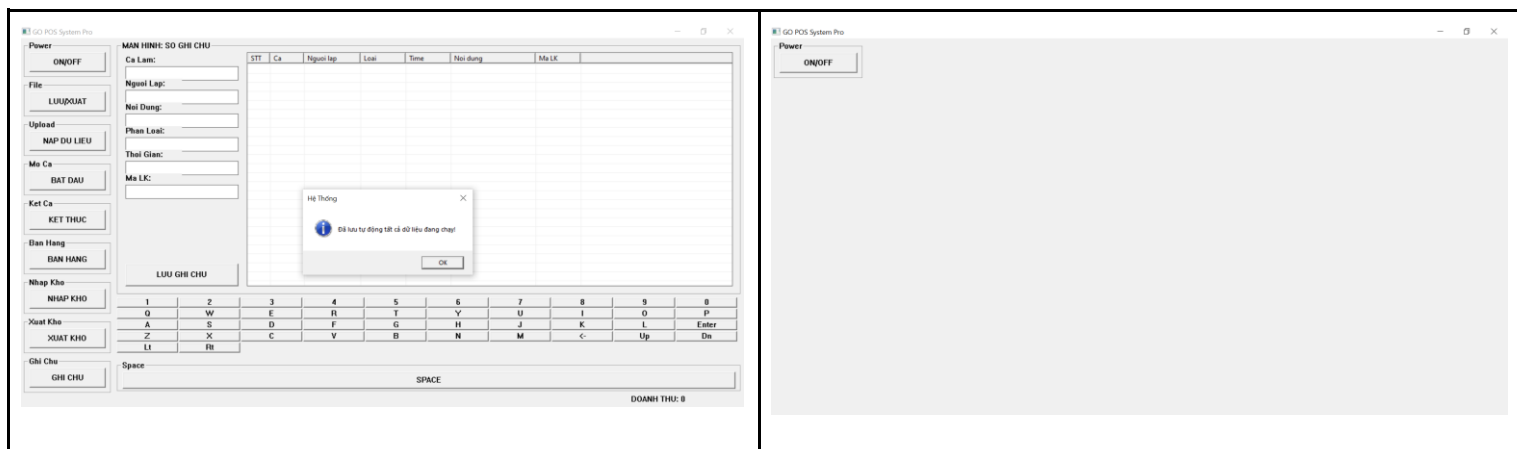
File NhanSu.txt: Ma, ChucVu

QL001,QuanLy
 QL002,QuanLy
 NV001,NhanVien
 NV002,NhanVien
 NV003,NhanVien
 NV004,NhanVien
 NV005,NhanVien

3.4.2. Các trường hợp kiểm thử

3.4.2.1. Test case 1: Kiểm thử tính năng Power

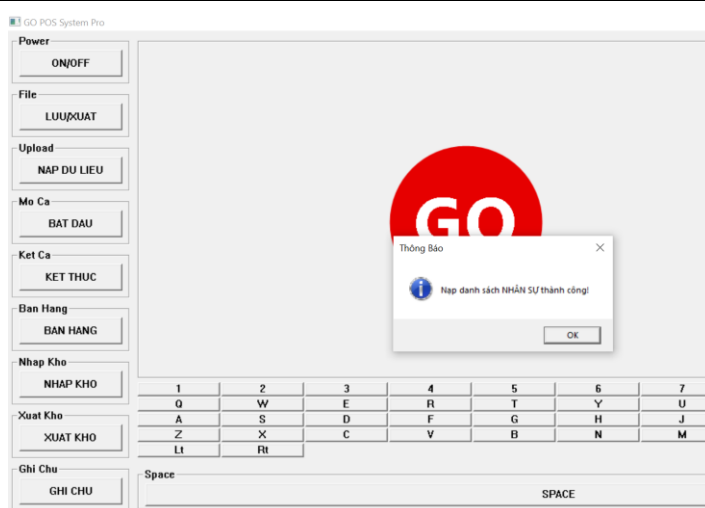
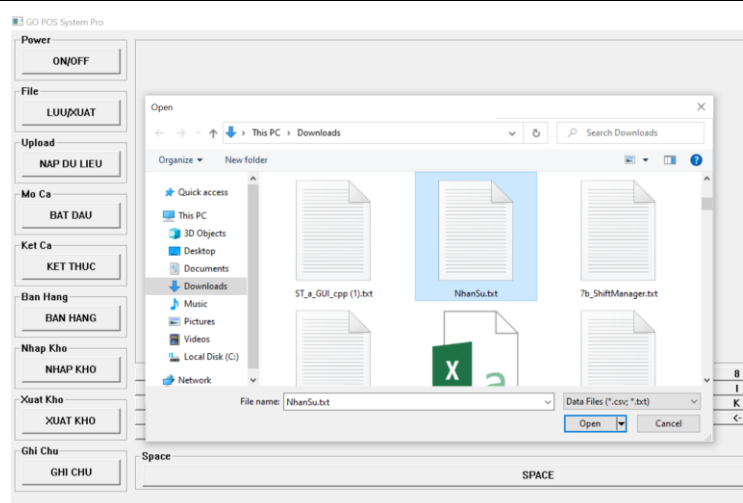
a. Ứng dụng đang tắt	b. Ứng dụng được bật (Power được chọn)
	
c. Tắt ứng dụng khi đang hoạt động, hiện thông báo “Đã lưu tự động tất cả các dữ liệu đang chạy”	d. Ứng dụng tự động hiện về trạng thái ban đầu



3.4.2.2. Test case 2: Kiểm thử với tính năng Upload dữ liệu *.txt đúng cấu trúc

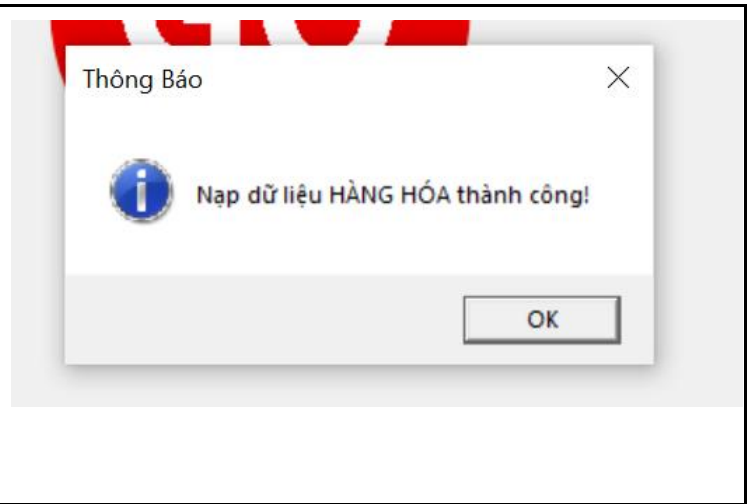
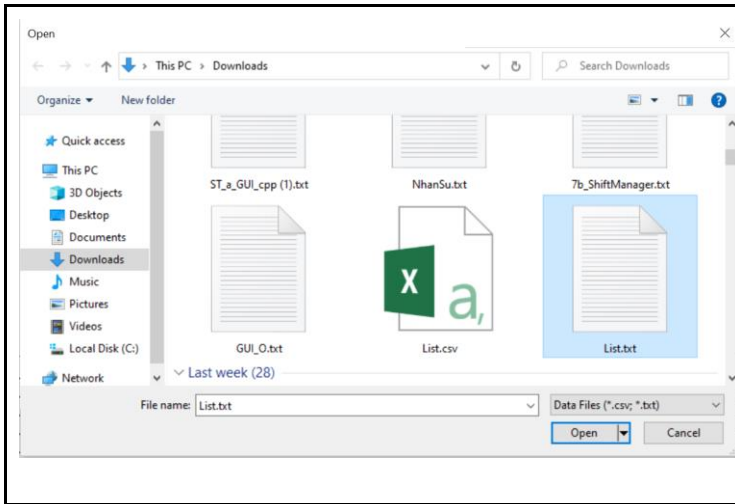
a. Chọn Upload > chọn NhanSu.txt để nạp dữ liệu mã quản lý và nhân viên để thực hiện mở ca, kết ca và các nghiệp vụ liên quan > Open

b. Hiện thông báo “Nạp danh sách NHÂN SỰ thành công!”



c. Chọn Upload > chọn List.txt để nạp dữ liệu danh sách hàng hoá để bán và dùng cho các nghiệp vụ liên quan > Open

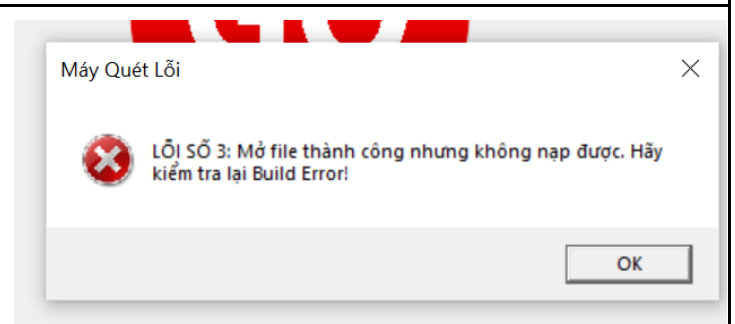
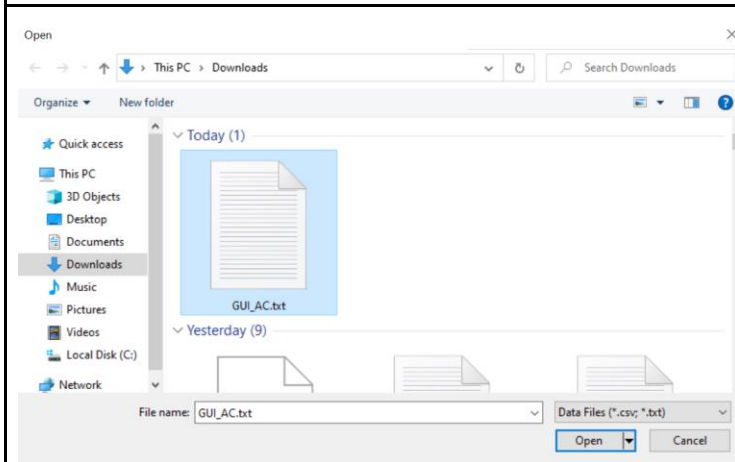
d. Hiện thông báo “Nạp dữ liệu HÀNG HOÁ thành công!”



3.4.2.3. Test case 3: Kiểm thử với tính năng Upload dữ liệu *.txt sai cấu trúc

Chọn Upload > chọn GUI_AC.txt để nạp dữ liệu mã quản lí và nhân viên để thực hiện mở ca, kết ca và các nghiệp vụ liên quan > Open

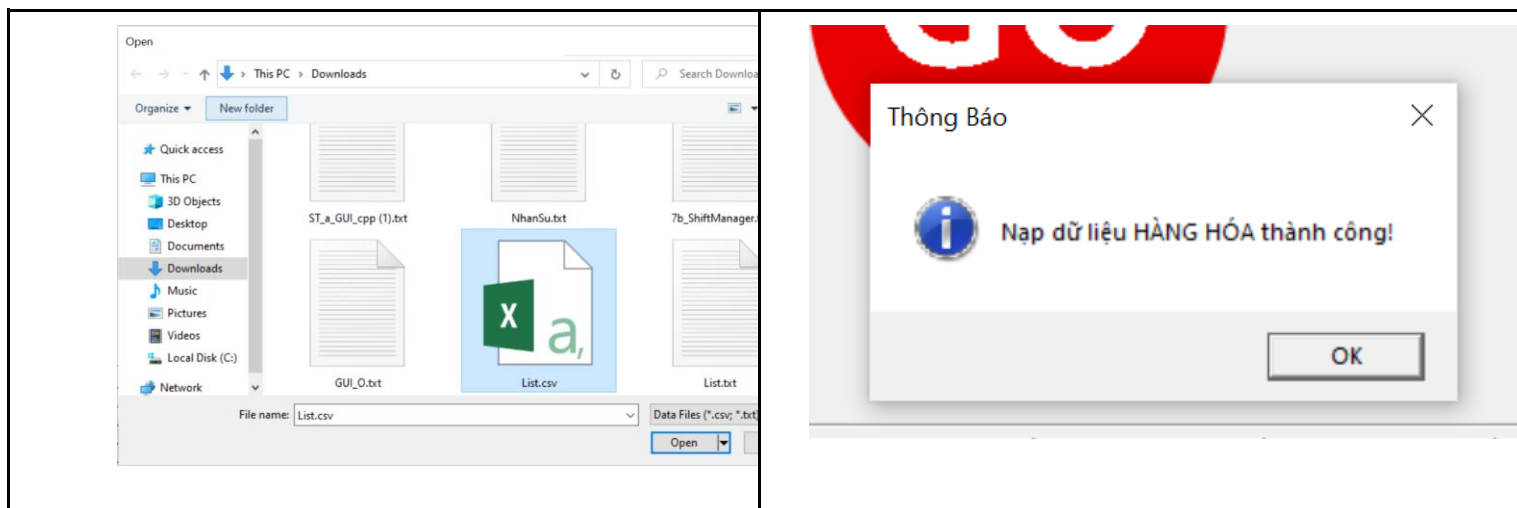
Hiện thông báo “LỖI SỐ 3: Mở file thành công nhưng không nạp được. Hãy kiểm tra lại Build Error!”



3.4.2.4. Test case 4: Kiểm thử với tính năng Upload dữ liệu *.csv đúng cấu trúc

a. Chọn Upload > chọn List.csv để nạp dữ liệu hàng hoá và các nghiệp vụ liên quan > Open

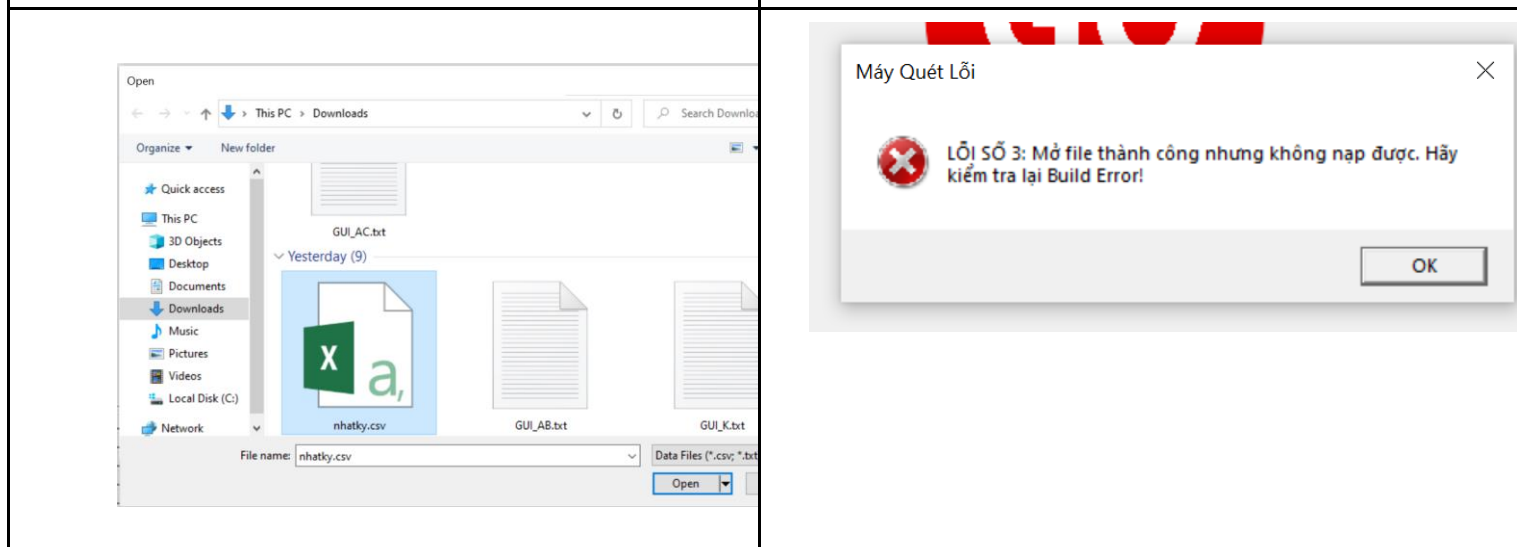
b. Hiện thông báo “Nạp dữ liệu HÀNG HOÁ thành công!”



3.4.2.5. Test case 5: Kiểm thử với tính năng Upload dữ liệu *.csv sai cấu trúc

c. Chọn Upload > chọn NhatKi.csv để nạp dữ liệu hàng hoá > Open

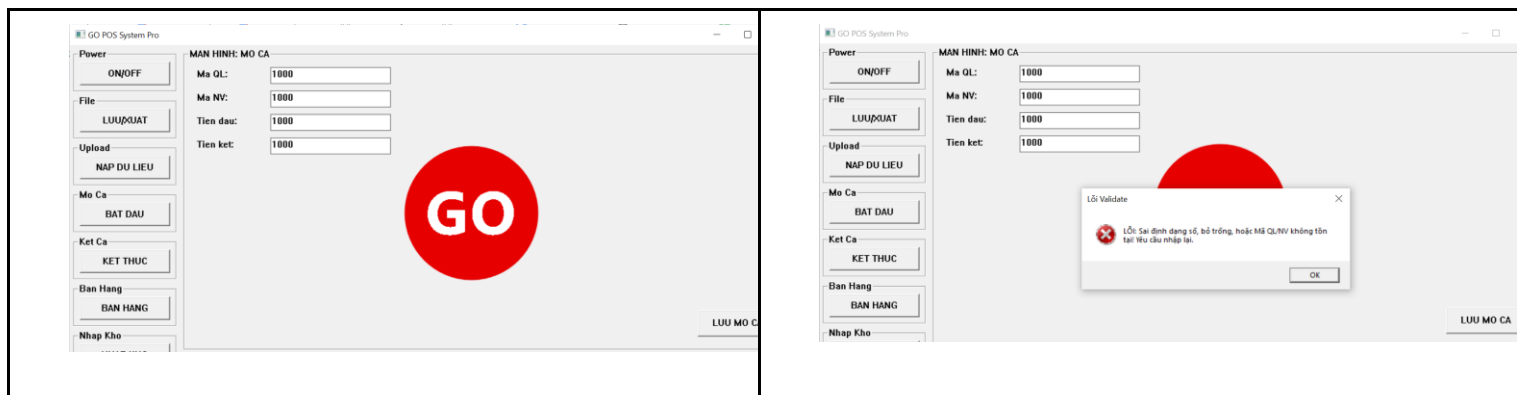
d. Hiện thông báo “Nạp dữ liệu HÀNG HOÁ thành công!”



3.4.2.6. Test case 6: Kiểm thử tính năng Mờ Ca với dữ liệu chưa được nạp

a. Chọn Mờ Ca > Nhập các thông tin bao gồm Mã quản lý, mã nhân viên, tiền đầu ca, tiền kết để thực hiện được các nghiệp vụ khác > LƯU MỜ CA

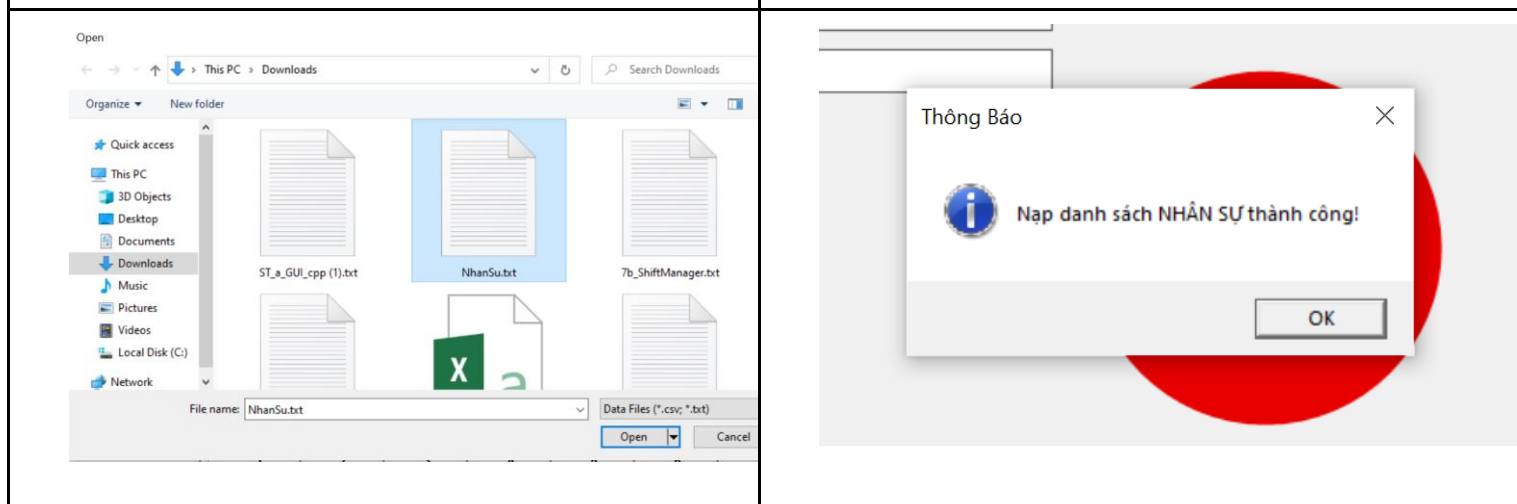
b. Hiện thông báo “LỖI: Sai định dạng số, bỏ trống hoặc mã QL/NV không tồn tại. Yêu cầu nhập lại”



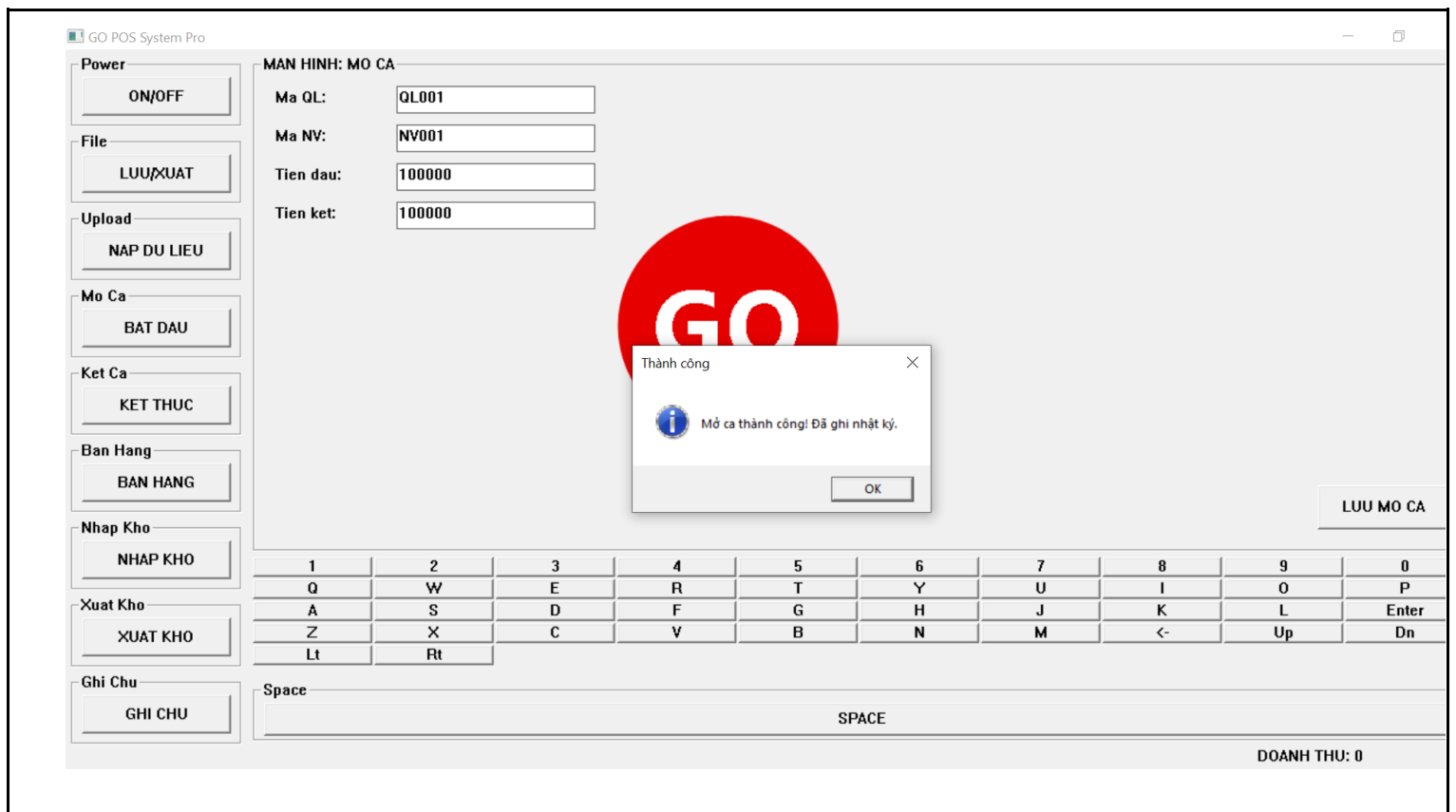
3.4.2.7. Test case 7: Kiểm thử tính năng Mở Ca với dữ liệu được nạp đúng

a. Chọn Chọn Upload> Chọn file chứa mã quản lí, nhân viên đúng cấu trúc> Open

b. Hiện thông báo “Nạp danh sách NHÂN SỰ thành công”



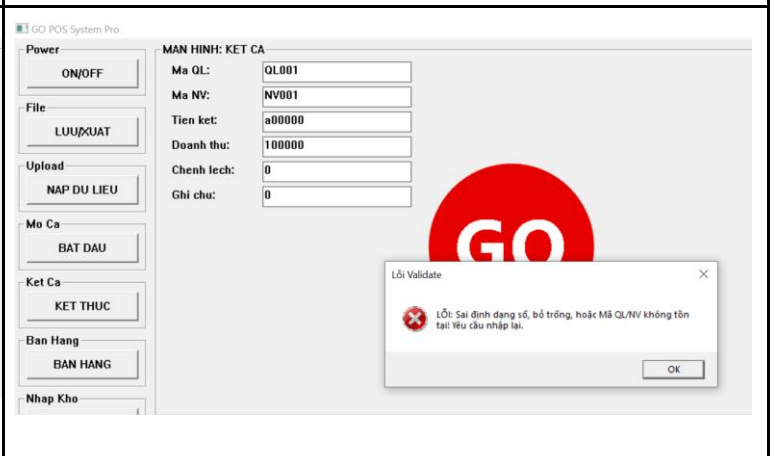
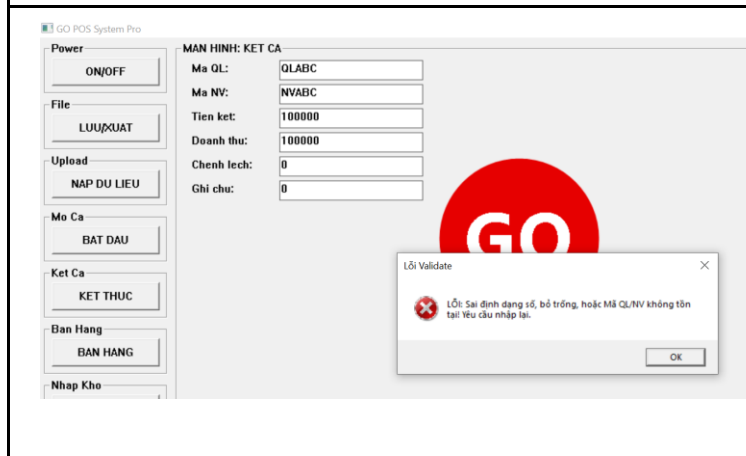
c. Nhập các thông tin bao gồm mã quản lí, mã nhân viên, tiền đầu ca (tiền đầu), tiền kết > LUU MO CA > Hiện thông báo “Mở ca thành công! Đã ghi nhật kí”



3.4.2.8. Test case 8: Kiểm thử tính năng Kết Ca

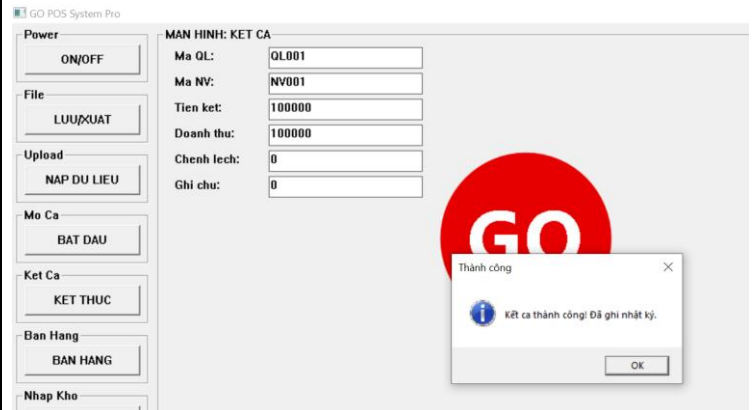
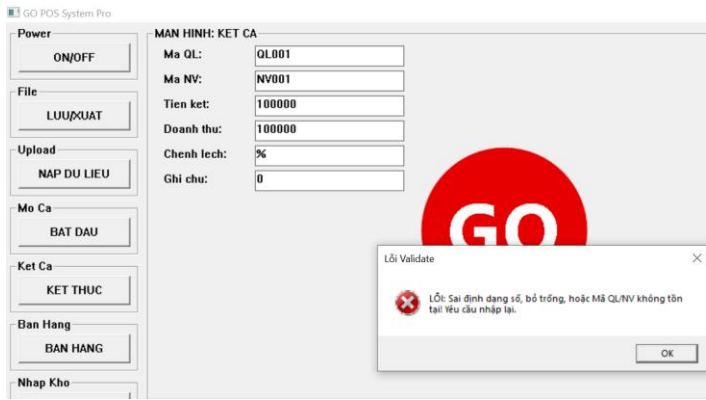
a. Chọn Ket Ca > Nhập sai thông tin mã quản lý và mã nhân viên

b. Chọn Ket Ca > Nhập sai thông tin Tiền kết > Hiện thông báo “LỖI: Sai định dạng số, bỏ trống hoặc Mã QL/NV không tồn tại! Yêu cầu nhập lại”



c. Chọn Ket Ca > Nhập sai thông tin Chênh lệch > Hiện thông báo “LỖI: Sai định dạng số, bỏ trống hoặc Mã QL/NV không tồn tại! Yêu cầu nhập lại”

d. Chọn Ket Ca > Nhập đúng tất cả thông tin > Hiện thông báo “Kết ca thành công! Đã ghi nhật kí”



3.4.2.9. Test case 9: Kiểm thử tính năng Bán Hàng

a. Sau khi nạp dữ liệu hàng hoá thành công > BÁN HÀNG > Nhập mã SKU > Nhập Số lượng > Enter > Các thông tin khác tự động được nhập bao gồm Tên Hàng, Đơn Giá, Khấu Trừ, HSD

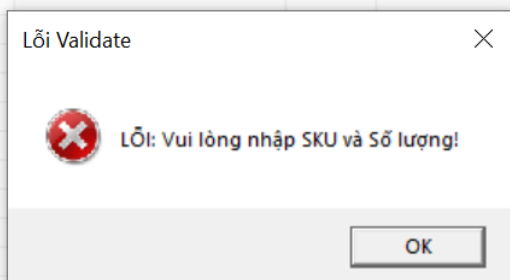
MAN HINH: BAN HANG								
Ma SKU:	STT	SKU	Ten Hang	SL	Gia	KT	HSD	Tong
SP001	1	SP001	Banh mi	20	10000.0000...	0.000...	2025-10-10	200000.000000
Ten Hang:	2	SP002	Nuoc ngot	10	12000.0000...	0.000...	2026-05-01	120000.000000
Banh mi	3	SP005	Banh quy	10	15000.0000...	0.000...	2026-02-20	150000.000000
So Luong:	4	SP009	Tra sua chai	5	25000.0000...	0.000...	2026-03-01	125000.000000
20	5	SP010	Banh bong lan	5	20000.0000...	0.000...	2025-08-20	100000.000000
Don Gia:								
10000.000000								
Khau Tru:								
0.000000								
HSD:								
2025-10-10								

b. Nhập cùng một mã SKU và Số Lượng khác nhau, ứng dụng cập nhật lại Số Lượng và Tổng > Enter

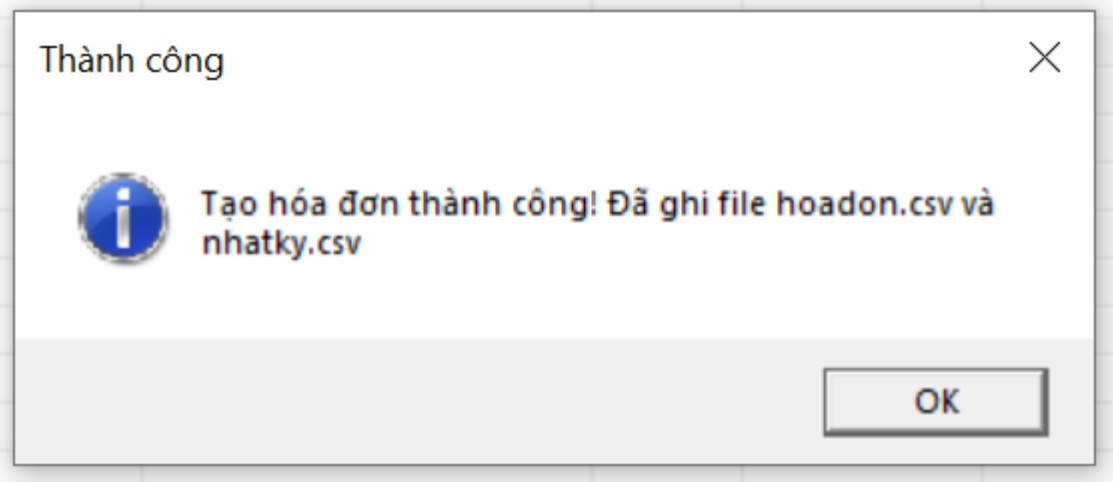
[illegible]

c. Nhập mã SKU không tồn tại > Enter > Thông báo lỗi: “LỖI: Vui lòng nhập SKU và Số lượng!”

d. Nhập số lượng vượt quá số lượng tồn tại > Enter > Thông báo lỗi: “LỖI: Vui lòng nhập SKU và Số lượng!”



e. Sau khi kiểm tra nội dung đơn hàng > LƯU KE TIẾP để thực hiện hoá đơn tiếp theo



f. File hoá đơn.csv được tạo trong mục Download, hiện đầy đủ thông tin

hoadon_1777803352.csv - Excel MAI LÝ KHẢI TRIỀU

File Home Insert Page Layout Formulas Data Review View Help Tell me

Clipboard Font Alignment Number Styles

POSSIBLE DATA LOSS Some features might be lost if you save this workbook in the comma-delimited (.csv) format. To preserve these features, save it in an Excel file format. Don't show again Save As

STT	SKU	Ten Hang	SL	Gia	Khau Tru	HSD	Thanh Tien
1	SP001	Banh mi	40	10000	0	10/10/2025	400000
2	SP002	Nuoc ngot	12	12000	0	5/1/2026	144000
3	SP005	Banh quy	10	15000	0	2/20/2026	150000
4	SP009	Tra sua ch	5	25000	0	3/1/2026	125000
5	SP010	Banh bong lan	5	20000	0	8/20/2025	100000
6	Tong Tien						919000

hoadon_1777803352.txt - Notepad

File Edit Format View Help

=== HOA DON BAN HANG ===

Ma Hoa Don: HD_1777803352

Thoi gian: 1777803352

-
1. Banh mi (SKU: SP001)
SL: 40 x Gia: 10000 = 400000
 2. Nuoc ngot (SKU: SP002)
SL: 12 x Gia: 12000 = 144000
 3. Banh quy (SKU: SP005)
SL: 10 x Gia: 15000 = 150000
 4. Tra sua chai (SKU: SP009)
SL: 5 x Gia: 25000 = 125000
 5. Banh bong lan (SKU: SP010)
SL: 5 x Gia: 20000 = 100000
-

TONG CONG: 919000 VND

STT	SKU	Ten Hang	SL	Gia	Khau Tru	HSD	Thanh Tien
1	SP099	Banh mi b	1	19000	0	#####	19000
2	SP088	Mi tom ly	1	7500	0	#####	7500
3	SP010	Banh bong	2	20000	0	#####	40000
4	SP011	Mi ly	3	12000	0	#####	36000
6	Tong Tien						102500

```

=== HOA DON BAN HANG ===
Ma Hoa Don: HD_1777803808
Thoi gian: 1777803808

-----
1. Banh mi bo (SKU: SP099)
   SL: 1 x Gia: 19000 = 19000
2. Mi tom ly dac biet (SKU: SP088)
   SL: 1 x Gia: 7500 = 7500
3. Banh bong lan (SKU: SP010)
   SL: 2 x Gia: 20000 = 40000
4. Mi ly (SKU: SP011)
   SL: 3 x Gia: 12000 = 36000

-----
TONG CONG: 102500 VND

```

3.4.2.10. Test case 10: Kiểm thử tính năng Nhập Kho

a. Nhập thông tin Mã SKU, Tên Hàng, SL Nhập, Giá Nhập, Nhà CC, HSD với các thông tin trên không tồn tại > Enter

b. Nhập thông tin Mã SKU khác cú pháp

MAN HINH: NHAP KHO

Ma SKU: SP104

Ten Hang: LapXuong

SLNhap: 50

GiaNhap: 10000

Nha CC: NguyenLong

HSD: 2027-12-2

STT	SKU	Ten Hang	SL	Gia	KT	HSD	Tong
1	SP102	OLong	1000	10000.00000...	0.000...	2027-12-10	10000000.000000
2	SP103	ButLong	10000	2000.0000000	0.000...	2030-12-10	20000000.000000
3	SP104	LapXuong	50	10000.00000...	0.000...	2027-12-2	500000.000000

MAN HINH: NHAP KHO

Ma SKU:

Ten Hang:

SLNhap:

GiaNhap:

Nha CC: Acecook

HSD:

STT	SKU	Ten Hang	SL	Gia	KT	HSD	Tong
1	HE01	MyKOKOMI	1000	4000.000000	0.000...	2030-10-12	4000000.000000

c. Nhập thông tin Mã SKU, Tên Hàng, SL Nhập, Giá Nhập, Nhà CC, HSD với các thông tin trên không tồn tại > Enter > LUU NHAP KHO

Today (12)

nhapkho_1777805555.csv

nhatky.csv

☒ **nhapkho_1777805260.csv**

nhapkho_1777804841.csv

hoadon_1777803808.csv

hoadon_1777803808.txt

nhapkho_1777805260.csv - Excel

Ma NK	SKU	Ten SP	SL	GiaNhap	HSD	Thanh Tien
1	SP102	OLong	1000	10000	12/10/2027	1.00E+07
2	SP103	ButLong	10000	2000	12/10/2030	2.00E+07
3	SP104	LapXuong	50	10000	12/2/2027	500000
Tong Tien						3.05E+07

d. Nhập thông tin số lượng sai > Enter > Thông báo lỗi: “LỖI: Số lượng và Giá nhập kho phải là chữ số”

e. Nhập Giá Nhập sai > Enter > “LỖI: Số lượng và Giá nhập kho phải là chữ số”

MAN HINH: NHAP KHO

Ma SKU: SP105
 Ten Hang: BanhDaHeo
 SL Nhap: \$100
 Gia Nhap: 20000
 Nha CC: ThiHuong
 HSD: 2030-12-12

STT	SKU	Ten Hang	SL	Gia	KT
1	HE01	MyKOKOMI	1000	4000.000000	0.000..

Lỗi Validate
 Lỗi: Số lượng và Giá nhập phải là chữ số!

LUU NHAP KHO

MAN HINH: NHAP KHO

Ma SKU: SP105
 Ten Hang: BanhDaHeo
 SL Nhap: 100
 Gia Nhap: %20000
 Nha CC: ThiHuong
 HSD: 2030-12-12

STT	SKU	Ten Hang	SL	Gia	KT	HSD	Tong
1	HE01	MyKOKOMI	1000	4000.000000	0.000...	2030-12-12	4000000.000000

Lỗi Validate
 Lỗi: Số lượng và Giá nhập phải là chữ số!

3.4.2.11. Test case 11: Kiểm thử với tính năng Xuất Kho

a. Nhập đúng mã SKU tồn tại, nhập số lượng xuất, Ngày xuất, Lý do: MUON HANG/ TIEU HUY/ THIEN NGUYEN > Enter

b. Nhập mã SKU không tồn tại, nhập số lượng xuất, Ngày xuất > Enter

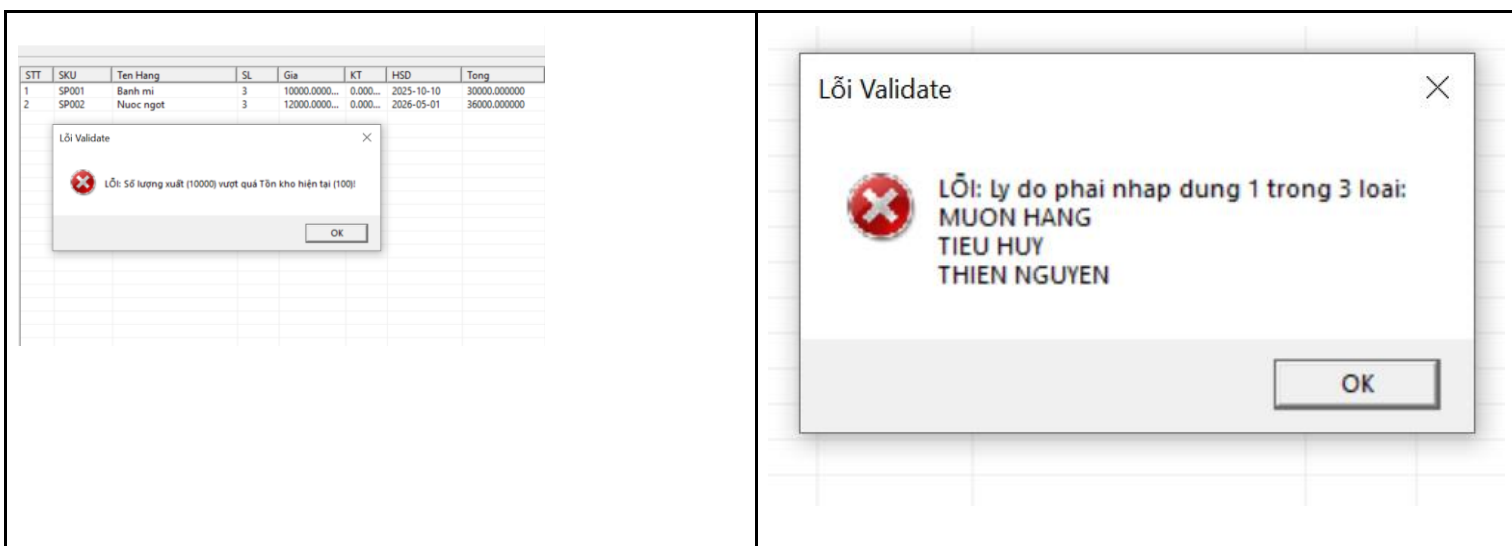
STT	SKU	Ten Hang	SL	Gia	KT	HSD	Tong
1	SP001	Banh mi	3	10000.0000...	0.000...	2025-10-10	30000.000000
2	SP002	Nuoc ngot	3	12000.0000...	0.000...	2026-05-01	36000.000000

STT	SKU	Ten Hang	SL	Gia	KT	HSD	Tong
1	SP001	Banh mi	3	10000.0000...	0.000...	2025-10-10	30000.000000
2	SP002	Nuoc ngot	3	12000.0000...	0.000...	2026-05-01	36000.000000

Lỗi Validate
 Lỗi: SKU không tồn tại trong kho!

c. Nhập số lượng xuất vượt số lượng tồn kho

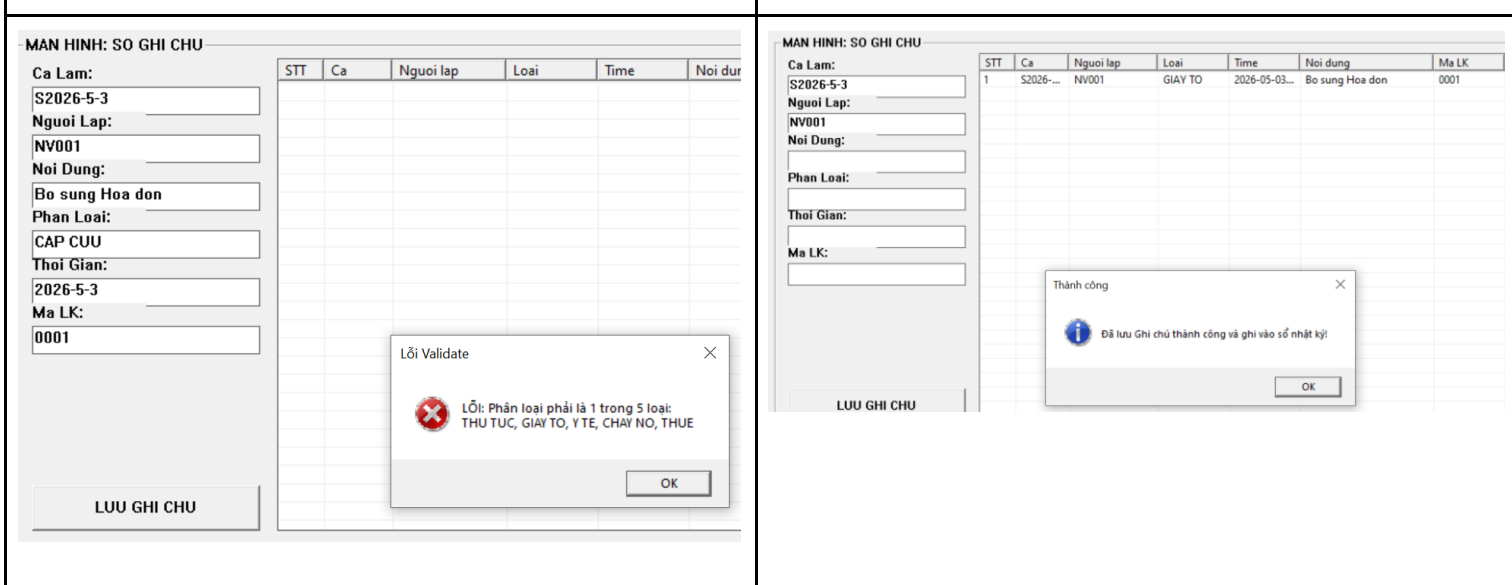
d. Nhập đúng mã SKU tồn tại, nhập số lượng xuất, Ngày xuất và sai lý do > Enter > LUU XUAT KHO



3.4.2.12. Test case 12: kiểm thử tính năng ghi chú

a. GHI CHU > Nhập sai Phân loại > LUU
GHI CHU > Hiện thông báo “LỖI: Phân loại phải là một trong 5 loại: THU TUC, GIAY TO, Y TE, CHAY NO, THUE”

b. GHI CHU > Nhập đúng tất cả thông tin > LUU GHI CHU



IV. KẾT LUẬN

Sau quá trình nghiên cứu, thiết kế và triển khai ứng dụng "Quản lý siêu thị quy mô vừa và nhỏ bằng ngôn ngữ C++", nhóm 8 đã hoàn thành các mục tiêu đề ra và đạt được các kết quả cụ thể như sau:

1. Kết quả đạt được:

Về mặt kỹ thuật: hệ thống vận hành ổn định trên nền tảng Win32 API, tách biệt rõ ràng giữa giao diện và logic xử lý thông qua kiến trúc MVC-lite. Nhóm đã áp dụng triệt để 4 tính chất của lập trình hướng đối tượng (OOP): đóng gói dữ liệu an toàn, kế thừa cấu trúc từ lớp Transaction, đa hình trong cơ chế lưu trữ và trừu tượng hóa các thuật toán xử lý phức tạp.

- Về mặt chức năng: chương trình đáp ứng đầy đủ các nghiệp vụ thực tế của một điểm bán hàng (POS) bao gồm: mở/kết ca, bán hàng, nhập/xuất kho và quản lý sổ ghi chú. Cơ chế tự động lưu (Auto-save) và nhật ký hệ thống (nhatky.csv) giúp đảm bảo tính toàn vẹn và khả năng truy vết dữ liệu cao.
- Về tính ứng dụng: giao diện trực quan với hệ thống bàn phím ảo mô phỏng máy POS thực tế, hỗ trợ nạp/xuất dữ liệu linh hoạt qua định dạng .csv và .txt, giúp người quản lý dễ dàng tích hợp với các công cụ văn phòng như Microsoft Excel.

2. Hạn chế và hướng phát triển:

Hạn chế: Vì lưu trữ dữ liệu trực tiếp trên các tệp tin văn bản, tốc độ xử lý có thể bị ảnh hưởng khi quy mô danh mục hàng hóa lên đến hàng chục nghìn dòng. Giao diện Win32 API mặc dù ổn định nhưng còn hạn chế về khả năng tùy biến thẩm mỹ hiện đại.

Hướng phát triển: Trong tương lai, nhóm dự kiến sẽ nâng cấp hệ thống sử dụng các hệ quản trị cơ sở dữ liệu (DBMS) như SQLite để tối ưu hiệu suất truy vấn. Đồng thời, nghiên cứu tích hợp thêm các tính năng nâng cao như quét mã vạch bằng camera và báo cáo thống kê doanh thu bằng biểu đồ trực quan.

Tổng kết: Đồ án không chỉ giúp nhóm củng cố vững chắc kiến thức về lập trình hướng đối tượng mà còn rèn luyện tư duy giải quyết vấn đề thực tiễn thông qua việc xây dựng một phần mềm quản lý hoàn chỉnh. Đây là nền tảng quan trọng để nhóm tiếp tục phát triển các hệ thống phần mềm phức tạp hơn trong tương lai.

V. TÀI LIỆU THAM KHẢO

[1]. Trần Đan Thư, Nguyễn Thanh Phương, Đinh Bá Tiến, Trần Minh Khiết, Giáo trình Nhập môn Kỹ thuật Lập trình, Khoa Công nghệ Thông tin, Trường Đại học Khoa học Tự nhiên – Đại học Quốc gia TP. Hồ Chí Minh, 2011.

[2]. Trần Đan Thư, Đinh Bá Tiến, và Nguyễn Tấn Trần Minh Khang, Phương pháp lập trình hướng đối tượng, Nhà xuất bản Khoa học và kỹ thuật, 2021.

[3]. Trần Đan Thư, Nguyễn Thanh Phương, Đinh Bá Tiến, Trần Minh Triết, và Đặng Bình Phương, Kỹ thuật lập trình, Nhà xuất bản Khoa học và kỹ thuật, 2021.

PHỤ LỤC

DANH MỤC BẢNG

Bảng 3.1. Chương trình GUI.cpp	30
Bảng 3.2. Chương trình DataManager.h	60
Bảng 3.3. Chương trình UploadManager.h	62
Bảng 3.4. Chương trình Controller.h	66
Bảng 3.5. Chương trình SystemManager.h	68
Bảng 3.6. Chương trình Transaction.h	70

DANH MỤC HÌNH

Hình 3.1. Lưu đồ Tổng quát Vận hành Hệ thống.....	11
Hình 3.2. Sơ đồ Tổ chức Phân tầng Mã nguồn Hệ thống.....	12
Hình 3.3. Lưu đồ Xử lý Giao diện toàn hệ thống.....	12
Hình 3.4. Lưu đồ Xử lý Sự kiện Giao diện.....	13
Hình 3.5. Lưu đồ Tab Power (ON/OFF).....	13
Hình 3.6. Lưu đồ File (LƯU/XUẤT).....	14
Hình 3.7. Lưu đồ Upload (NẠP DỮ LIỆU).....	14
Hình 3.8. Lưu đồ Mở Ca (BẮT ĐẦU).....	15
Hình 3.9. Lưu đồ Kết Ca (KẾT THÚC)	15
Hình 3.10. Lưu đồ Bán Hàng (BÁN HÀNG)	16
Hình 3.11. Lưu đồ Nhập Kho (NHẬP KHO)	16
Hình 3.12. Lưu đồ Xuất Kho (XUẤT KHO)	17
Hình 3.13. Lưu đồ Ghi Chú (GHI CHÚ)	17
Hình 3.14. Lưu đồ Quản trị Dữ liệu Trung tâm.....	18
Hình 3.15. Lưu đồ Giải thuật Nạp dữ liệu đầu vào.....	19
Hình 3.16. Lưu đồ Controller.....	21
Hình 3.17. Lưu đồ Quản lí hệ thống.....	21
Hình 3.18. Lưu đồ Cơ chế Giao dịch chung.....	22
Hình 3.19. Lưu đồ Nghiệp vụ Bán hàng.....	23

Hình 3.20. Lưu đồ Nghiệp vụ Nhập kho.....	24
Hình 3.21. Lưu đồ Nghiệp vụ Xuất kho.....	25
Hình 3.22. Lưu đồ Xử lý Sổ ghi chú thời gian thực.....	26
Hình 3.23. Lưu đồ Quản lý Ca làm việc.....	27
Hình 3.24. Lưu đồ xuất báo cáo.....	27
Hình 3.25. Giao diện khi tắt ứng dụng	87
Hình 3.26. Giao diện khi bật ứng dụng	87
Hình 3.27. Giao diện LƯU/XUẤT (.csv)	88
Hình 3.28. Giao diện LƯU/XUẤT (.txt)	88
Hình 3.29. Giao diện NẠP DỮ LIỆU (*.csv, *.txt)	89
Hình 3.30. Giao diện BẮT ĐẦU	89
Hình 3.31. Giao diện KẾT THÚC	90
Hình 3.32. Giao diện BÁN HÀNG	90
Hình 3.33. Giao diện NHẬP KHO	91
Hình 3.34. Giao diện XUẤT KHO	91
Hình 3.35. Giao diện GHI CHÚ	92